

版本信息:

版本 REV2026

时间 04/03/2026

米联客-安路 FPGA UDP 以太网通信方案

Milianke Anlogic FPGA Course

微信扫码注册米联客技术论坛 www.uisrc.com 免费享受更多资源



扫码关注微信公众平台“米联客(MILIANKE)”掌握更多信息动态



技术支持

客户可以通过以下链接获取米联客官方店铺的客服联系方式,加入米联客企业微信技术支持群,获得专业的技术支持和解答。感谢您的支持!

京东米联客旗舰店: <https://milianke.jd.com>

天猫米联客旗舰店: <https://milianke.tmall.com>

淘宝米联客旗舰店: <https://milianke.taobao.com>

售后

1: 7 天无理由退货(人为原因除外)

2: 质保期限: 本产品自快递签收之日起,提供一年质保服务(主芯片,比如 FPGA 或者 CPU 等除外)。

3: 维修换货,需提供淘宝订单编号或合同编号,联系销售/技术支持安排退回事宜。

售后维修请登录工单系统: https://www.uisrc.com/plugin.php?id=x7ree_service

4: 以下情形不属于质保范畴

A 由于用户使用不当造成板子的损坏:比如电压过高造成的开发板短路,自行焊接造成的焊盘脱落、铜线起皮等。

B: 用户日常维护不当造成板子的损坏: 比如放置不当导致线路板腐蚀、基板出现裂纹等

5: 质保范畴外(上方第 4 条)及质保期限以外的产品, 本公司提供有偿维修服务。维修仅收取器件材料成本, 往返运费全部由客户承担。

6: 寄回地址, 登录网页获取最新的售后地址: <https://www.uisrc.com/t-1982.html>

常州米联客信息科技有限公司 版权所有

常州溧阳总部: 常州溧阳市中关村吴潭渡路雅创高科制造谷 10-1 幢楼

南京研发基地: 南京市江宁区秣陵街道吉印大道 3118 号联东 U 谷·南京九龙湖智造创新港 3 号楼 9 楼

版本	时间	描述
Rev2024	2024-08-23	第一版
Rev2026	2026-04-03	第二版

写给即将开启 FPGA 学习之旅的用户

FPGA 编程又称为硬件编程语言和流行的各类软件编程语言 C/C++、JAVA、python 等相比，掌握基础的硬件编程语言不是难事，难点是 FPGA 在每个专业领域的应用，只有充分理解了 FPGA，并且具有对自己所处行业专业背景认知，才能真正理解 FPGA 应该用在什么场合更加合适。

很多人问我学习 FPGA 是否有前途，这个问题着实难以回答。我们可以一起来探讨以下几种情况，会是 FPGA 发挥作用的场合：

1)、数字 IC 设计工作

数字 IC 设计还是主要以硬件编程语言去设计数字 IC 芯片，通过硬件编程语言，软件可以把语言翻译成电路，自动布线工具可以完成布局布线，之后流片。由于流片费用非常贵，前期也可以用 FPGA 芯片模拟设计的数字 IC 芯片的功能。

2)、数字信号处理

高速的 ADC,DAC 的数模信号处理的领域也是必然需要用到 FPGA，在无线通信、雷达信号处理等领域也都会用到。

3)、数字信号高速通信

FPGA 具备的高速接口也非常适合用于高速通信，比如 PCIE 通信、光通信、以太网通信

4)、视频图像

包括图像的拼接、缩放、高效的实时传输等， 4K 视频 、8K 视频领域、OSD 菜单、高效的 FPGA 图像处理算法

5)、硬件加速算法

FPGA 硬件加速领域应用目前是热点，已经有很多公司利用 FPGA 的硬件加速实现了很好的经济效应。

6)、神经网络

利用 FPGA 的高效实时性实现的神经网络算法通常有比传统方法更高的实时性、更低的功耗、高性价比。难点是初期开发工作量较大。

7)、通用 CPU GPU 无法完成的工作

如果通用的 CPU 和 GPU 无法完成的工作任务，可以考虑下 FPGA 或者带 FPGA 的 SOC。

不能为了专门学习 FPGA 而学习 FPGA，FPGA 只是一个工具。如果不知道如何应用 FPGA，那么这是非常悲哀的一件事情，学习 FPGA 只是学习一门技能，而结合自己专业背景选择最合适的解决方案，解决问题才是最终的目的。

本章节课程介绍安路 TD 软件安装、JTAG 驱动安装、JTAG 下载程序、创建 FPGA 工程、添加代码、添加 IP、约束管脚、添加时序约束、使用在线逻辑分析仪、编译、下载、固化程序到 FLASH。同时也介绍了第三方 vscode 代码编辑软件的安装和 TD 调用 vscode 编辑代码的方法。

本章节课程适合初次使用 TD 软件的用户快速熟悉和掌握 TD 软件使用。

目录

米联客-安路 FPGA UDP 以太网通信方案	1
01 UDP RGMII 千兆以太网测试	1
1 概述	1
2 以太网协议	1
2.1 以太网介绍	1
2.2 物理层	2
2.3 以太网帧格式	3
3 程序设计	11
3.1 MAC 层	11
3.2 IP_ARP 层	33
3.3 IP 层	40
3.4 ARP 层	50
3.5 UDP 层	57
3.6 ICMP 层	61
4 仿真验证	67
4.1 数据通信仿真	68
4.2 PAUSE 流控仿真	74
4.3 ICMP 层仿真	77
5 上板调试	78
5.1 硬件连接	78
5.2 PC 设置	78
5.3 UDP 回环验证	81
5.4 FPGA 主动发送验证	86

02 UDP 三速以太网设计	87
1 以太网 PHY 芯片介绍	87
2 MDIO 接口设计	90
2.1 MDIO 时序介绍	90
2.2 MDIO 程序设计	91
3 RGMII 接口设计	101
3.1 RGMII 接口时序	101
3.2 RGMII 模块设计	104
3.3 数据转换模块设计	108
4 三速以太网回环测试	114
4.1 100M 以太网测试	114
4.2 10M 以太网测试	115
03 UDP 巨型帧设计	118
1 巨型帧介绍	118
2 巨型帧程序设计	118
2.1 巨型帧帧格式	118
2.2 发送端设计	119
2.3 接收端设计	128
3 巨型帧验证	131
3.1 仿真验证	131
3.2 上板验证	132
04 基于 UDP 的 AD7606 采集方案	136
1 系统框图	136
2 下载演示	137

2.1 硬件连接	137
2.2 运行结果	137
05 基于 UDP 的 AD9248 采集方案	139
1 系统框图	139
2 下载演示	140
2.1 硬件连接	140
2.2 运行结果	140
06 DDS 信号采集方案演示	142
1 系统框图	142
2 下载演示	143
2.1 硬件连接	143
2.2 运行结果	143
07 图像采集方案演示	146
1 程序设计	146
2 上板调试	147
2.1 硬件连接	147
2.1 实验结果	147
08 摄像头采集方案演示	148
1 系统框图	148
2 下载演示	149
2.1 硬件连接	149
2.2 运行结果	149

01 UDP RGMII 千兆以太网测试

软件版本: Anlogic -TD_6.2.2_Release_2026.1

操作系统: WIN10/WIN11 64bit

硬件平台: 适用安路(Anlogic)FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 概述

本文介绍了基于 FPGA 的米联客 UDP 协议栈的实现原理、内部逻辑、仿真测试、上板验证。该协议栈具有以下功能:

- (1) 实现了 ARP 请求和应答报文的发送, 完成 IP 地址和 MAC 地址的映射。
- (2) 实现了 ICMP 回显应答报文的发送。
- (3) 实现了 UDP 回环通信。
- (4) 实现了 PAUSE 帧对数据流量的控制。

2 以太网协议

2.1 以太网介绍

以太网是 IEEE 802.3 标准下的计算机局域网技术, 具有传输速率高、结构简单、工作可靠等优点, 是当今现有局域网采用的最通用的协议标准。

以太网最早由 Xerox 公司创建, 并由 Xerox、Intel 和 DEC 公司联合开发。之后 IEEE 802.3 标准定义了以太网的技术标准, 包括物理层的连线、电信号和介质访问层协议的内容。随着技术的发展, 以太网逐渐标准化, 并成为了当前应用最普遍的局域网技术。

以太网是一种传输规则, 收发双方必须遵循这些规则才能使数据有效传输。OSI (Open System Interconnection) 七层模型和 TCP/IP 四层模型是常用的数据传输模型, 本设计使用 OSI 七层模型设计以太网传输模块, 其各个子层的功能如下表所示。

层级	名称	作用	常用协议
7	应用层 (Application Layer)	为应用程序或用户请求提供各种服务 文件传输、电子邮件、虚拟终端	HTTP、FTP、SMTP、POP3、NNTP
6	表示层 (Presentation Layer)	数据编码、格式转换、数据加密	LLP、NBSSP
5	会话层 (Session Layer)	创建、管理和维护会话 建立或解除与其他接点的联系	SSL、DAP
4	传输层 (Transport Layer)	提供端对端的接口，数据通信	TCP、UDP
3	网络层 (Network Layer)	为数据包选择路由，IP 地址及路由选择	IP、ICMP
2	数据链路层 (Data Link Layer)	提供介质访问和链路管理。传输有地址的帧，错误检测功能	MAC、ARP
1	物理层 (Physical Layer)	管理通信设备和网络媒体之间的互联互通以二进制数据形式在物理媒体上传输数据	光纤、双绞线 (介质)

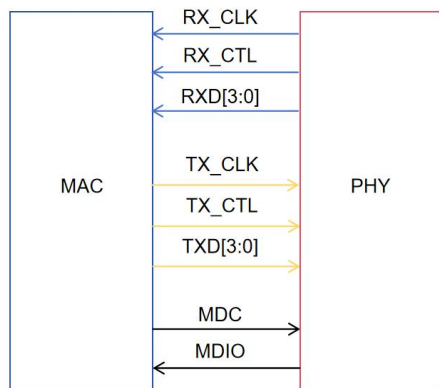
数据从上至下逐级封装，加入每层的头部信息，在物理层转换为比特流发送；接收端使用逆向顺序把数据逐级解封装，发送给应用层。FPGA 的 UDP 通信只做到了传输层，物理层一般由外部 PHY 芯片提供，没有应用层、表示层和会话层。

2.2 物理层

以太网物理层是 OSI 模型中的第一层，它负责数据的编码、解码、数模转换和数据在物理介质上的传输等。在 10Mbps、100Mbps 和 1Gbps 速率的以太网中，通常采用 RJ45 网口和双绞线作为物理介质传输数据，RJ45 连接器是一种 8 针连接器，提供 4 对提供信号输出。在 10Mbps./100Mbps 速率下，使用了其中 4 根线，在 1Gbps

速率下使用了全部 8 根线。

千兆以太网传输的数据由外部 PHY 芯片提供编解码和数模转换功能。在 1000BASE-T 标准中采用 4D_5PAM 编码方式,其带宽相比传统二电平编码相比提高了带宽利用率。千兆以太网的数据链路层和物理层之间一般采用 RGMII 接口连接,其接口连接示意图如下:



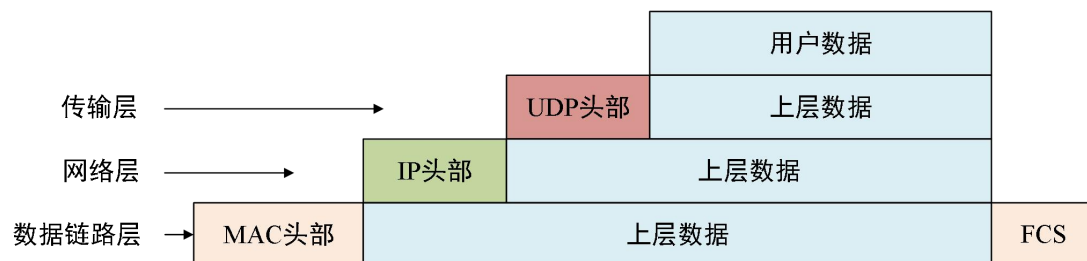
该接口采用双沿采样的方案,相比 GMII 接口减少了 I/O 数量,从而减小了 PCB 的面积和布线难度,降低了成本。RGMII 接口的控制信号被多路复用,CTL 信号复用了 ER 和 EN 信号,上升沿传输 EN 信号,下降沿传输 EN 信号与 ER 信号的异或结果。

万兆以太网的内部逻辑通过 XGMII 接口与 FPGA 芯片内部的 SERDES 连接。XGMII 接口的数据位宽为 32 位, SERDES 提供 PMA 和 PCS 子层,其结构包括锁相环、串并转换器、预加重电路、均衡器、时钟恢复电路等,万兆网的数据经过 SERDES 进行 64/66B 的编码,通过 10.3125Gbps 的速率发出,经过光模块进行光电转换后,采用光纤作为物理介质进行传输。

本文通过千兆以太网来验证 UDP 协议栈的功能。

2.3 以太网帧格式

数据在以太网中传输时需逐层封装或拆解,以添加或过滤上各层协议对应的首部。UDP 数据帧的结构和封装顺序如图所示,用户数据经过 UDP 层、IP 层、MAC 层,分别添加了 8 字节 UDP 头部、20 字节 IP 头部、22 字节 MAC 头部和 4 字节 CRC 校验结果,再经过 PHY 层传出完整的一帧数据。

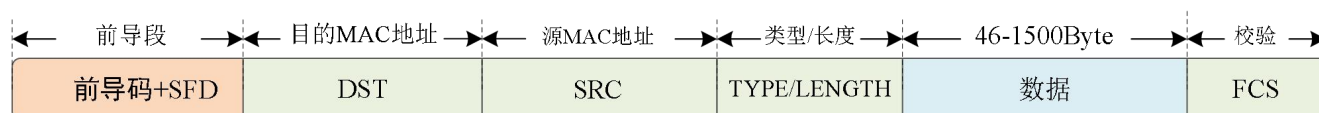


下面介绍各层协议的具体功能及其帧格式。

2.3.1 MAC 协议

以太网 MAC 层在网络通信中起着至关重要的作用，它负责数据的封装与拆包、传输控制、错误检测与纠正功能，确保数据在网络中正确传输和有效处理。

MAC 数据帧格式如下图 2.1 所示，其包头包含前导码、SFD、目的 MAC 地址、源 MAC 地址、类型/长度，最后 4 个字节为 CRC 校验位。



各区域具体功能如下：

前导码 (7Byte)：用于调整时钟使收发节点的时钟同步，接收端通过识别前导码来确定数据帧的边界，以确保接收端能够准确地提取出完整的数据帧。内容为连续 7 字节的 8'h55。

帧起始定界符 SFD (1Byte)：用于区分前导段和数据段，确保后续数据的正确解析。内容为 8'hd5。

MAC 地址 (6Byte)：MAC 地址由 48bit 数据组成，它是网卡的物理地址，一般固化在网卡的 ROM 中。在以太网传输的最底层就是根据 MAC 地址来发送数据，同一个网络里不能有两个相同的 MAC 地址。MAC 地址的前 3 个字节是组织唯一标识符，用于标识网络硬件制造商，后 3 个字节是制造商为网卡分配的序列号。

类型/长度 (2Byte)：该区域可以用来表示 MAC 数据包下一层的类型，也可以用来描述 MAC 数据包数据段的长度。该值小于 1536 表示长度，大于 1536 表示类型，IP 协议对应的数值为 0x0800，ARP 协议为 0x0806。

FCS (4Byte)：FCS (错误检测码) 是用于检测数据帧在传输过程中是否发生错误的一个字段，它通过 CRC (循环冗余校验) 算法计算得出，并附加在数据帧的尾部。发送数据时，会根据数据内容生成简短的校验和，并将其与数据一起发送。接收数据时，将再次生成的校验和并将其与发送的校验和进行比较。如果二者相等，则数据没

有损坏。

每一帧传输完成后，都必须等待 96bit 数据传输的时间，即最小帧间隔（IFG），才可以进行下一次以太网帧的传输。由于以太网每个时钟周期发送 8bit 数据，所以最小帧间隔为 12 个时钟周期。

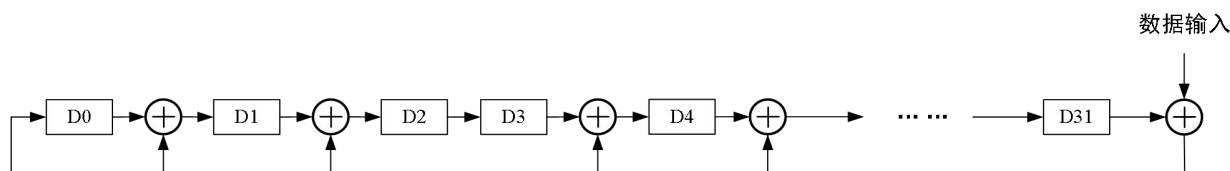
2.3.2 CRC 校验

CRC 校验的实现原理比较简单。设 $m(x)$ 为需要校验的数据， $g(x)$ 为多项式，先将数据左移 K 位（ K 为 $g(x)$ 的最大项系数），然后做模 2 除法运算，模二运算的本质是异或运算。

比如，当 $m(x)$ 为 101011， $g(x)$ 为 X^3+X+1 时，将 101011 左移 3 位得到 101011000，与 1011 进行模 2 除法，过程和结果如下图所示。计算得到的余数为 111，将其附加到 $m(x)$ 数据后得到 101011010，即为新的数据帧，该数据与 1011 进行模二除法得到的余数为 0，说明校验正确。

$$\begin{array}{r}
 10011 \\
 1011 \overline{) 101011000} \\
 \underline{1011} \\
 1110 \\
 \underline{1011} \\
 1010 \\
 \underline{1011} \\
 010
 \end{array}$$

硬件设计中，可以通过串行或并行的方式来实现 CRC 的计算。串行方式消耗的资源少，但速度较慢。串行处理的硬件电路实现实际上就是寄存器和异或门电路连接成线性反馈移位寄存器。CRC-32 生成的多项式为 $g(x)=x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$ ，其硬件实现如下图所示。



用 $c1[31:0]$ 表示移位寄存器的次态，用 $c[31:0]$ 表示移位寄存器的现态，用 $d[0:0]$ 表示需要校验的串行数据，得到的布尔表达式如下：

```

c1[0] = d[0] ^ c[31];
c1[1] = d[0] ^ c[0] ^ c[31];
c1[2] = d[0] ^ c[1] ^ c[31];
c1[3] = c[2];
c1[4] = d[0] ^ c[3] ^ c[31];

```

```
c1[5 ] = d[0] ^ c[4] ^ c[31];
c1[6 ] = c[5];
c1[7 ] = d[0] ^ c[6] ^ c[31];
c1[8 ] = d[0] ^ c[7] ^ c[31];
c1[9 ] = c[8];
c1[10] = d[0] ^ c[9] ^ c[31];
c1[11] = d[0] ^ c[10] ^ c[31];
c1[12] = d[0] ^ c[11] ^ c[31];
c1[13] = c[12];
c1[14] = c[13];
c1[15] = c[14];
c1[16] = d[0] ^ c[15] ^ c[31];
c1[17] = c[16];
c1[18] = c[17];
c1[19] = c[18];
c1[20] = c[19];
c1[21] = c[20];
c1[22] = d[0] ^ c[21] ^ c[31];
c1[23] = d[0] ^ c[22] ^ c[31];
c1[24] = c[23];
c1[25] = c[24];
c1[26] = d[0] ^ c[25] ^ c[31];
c1[27] = c[26];
c1[28] = c[27];
c1[29] = c[28];
c1[30] = c[29];
c1[31] = c[30];
```

并行方式消耗的资源多，但速度快，适合用于使用千兆以太网或万兆以太网的场景。得到 8 位并行运算的结果，需

要对 d[7:0]的每一位做迭代运算，对 d[0]计算得到的结果已经列出，对 d[1]做同样的运算得到的布尔表达式如下：

```
c2[0 ] = d[0] ^ c[30];
c2[1 ] = d[1] ^ d[0] ^ c[30] ^ c[31];
c2[2 ] = d[1] ^ d[0] ^ c[0] ^ c[30] ^ c[31];
c2[3 ] = d[1] ^ c[1] ^ c[31];
c2[4 ] = d[0] ^ c[2] ^ c[30];
... ..
c2[31] = c[29];
```

经过 8 次迭代运算即可得到 CRC-32 的 8 位并行运算公式。

2.3.3 PAUSE 流量控制帧格式

PAUSE 帧是一种用于控制数据流量的控制帧，当对端发来的数据量过大，为了防止缓冲区溢出导致的数据丢失，

MAC 层控制流量控制子层发出流量控制帧。接收方收到 PAUSE 帧后，会将其内容送到 MAC 层中的流量控制模块中

进行解析，得到需要暂停发送的时间。在一帧结束后开启暂停，暂停计数结束后，开始发送新一帧的数据。

PAUSE 帧的格式如下图所示。

前导码+SFD	目的MAC	源MAC	TYPE	OPCODE	TIME	补充数据	FCS
---------	-------	------	------	--------	------	------	-----

目的 MAC（6Byte）：PAUSE 帧的目的为保留的组播地址，其值为固定的 48' h01_80_c2_00_00_01。

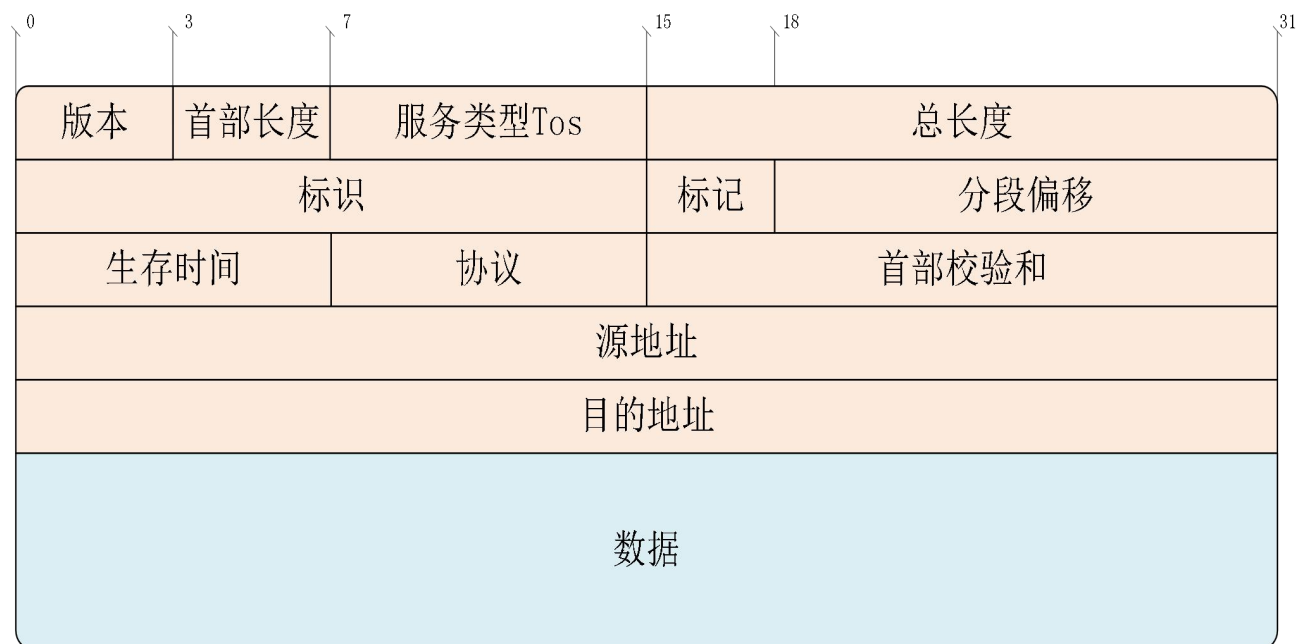
TYPE（2Byte）：固定值为 16' h8808，表示帧类型为 MAC 控制帧。

OPCODE（2Byte）：操作码，值为 16' h0001，表示 MAC 控制帧中的 PAUSE 帧。

TIME（2Byte）：时间参数，它的值表示以太网以当前速率传输 512bit 数据的时间，接收方实际暂停的时间为该字段数值左移 6 位得到的值乘时钟周期。

补充数据：有效信息后补 0，补齐 MAC 帧所需要的最小数据数量。

2.3.4 IP 协议



MAC 数据帧经过数据链路层传输到网络层时，前导码、SFD、MAC 地址、类型/长度以及校验字节均被过滤，IP 数据包传入了网络层。该数据包也不完全是有效数据，其还包含 20 字节的 IP 头部，具体见上图。IP 头部各区域功能如下：

版本（4bit）：定义 IP 协议版本，IPV4 为 4' h4。

首部长度（4bit）：定义 IP 数据包头部长度，表示具有 32 位字长数据的数量。最小值为 5，最大值为 15。

服务类型 (8bit) : 用于分配优先级、延迟、吞吐量及可靠性, 一般为 8' h00。

总长度 (16bit) : 定义整个 IP 数据包长度。

标识 (8bit) : 发完一包数据自动加 1。

标记 (3bit) : 最高位保留为 0; 中间位是否开启分段, 1 不开启, 0 开启; 最低位表示是否存在下一个分段, 0 表示为最后一个分段, 1 表示还存在下一个分段。一般默认为 3' b010。

分段偏移 (13bit) : 表示分段数据在源数据报中的相对位置。

生存时间 (8bit) : 表示以太网数据包可以中转进过多少个路由器, 每进过一个路由器, 该值就会减少 1, 直到该值变成 0, 丢包该包。WIN 系统默认为 8' h80。

协议 (8bit) : 指出 IP 处理过程完成后, 传输层的协议。UDP 为 8' d17, TCP 为 8' d6, ICMP 为 8' d1。

首部校验和 (16bit) : 该区域确保 IP 协议头部的完整性。将头部每 16 位划分为一部分, 将各部分相加取得的结果再取反码, 即为该区域数据。

源地址 (32bit) : 源主机 IP 地址。

目的地址 (32bit) : 目的主机 IP 地址。

2.3.5 IP 首部校验

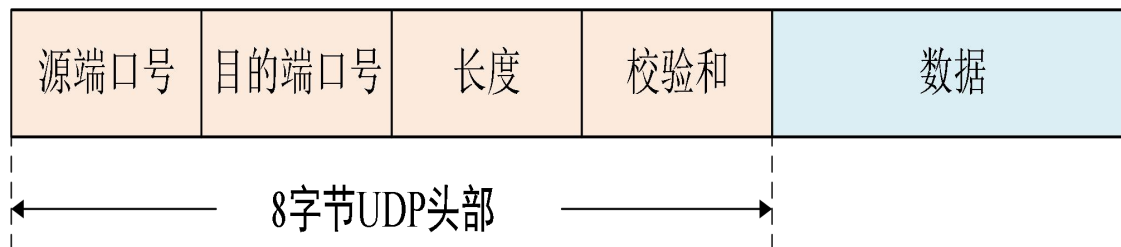
IP 首部校验的原理是将 IP 数据包头部每 16bit 的数据划分、校验位置 0, 进行累加操作, 得到的结果若有进位, 则将进位数据和低 16bit 数据进行相加, 直到不再产生进位, 把得到的结果取反, 就是 IP 首部校验和的值。

发送方对首部校验和计算, 并将其结果放入 IP 头部的首部校验和字段。接收方接收到 IP 数据包时, 将其所有的包头数据进行校验, 若得到的校验结果为全 1, 则数据没有发生改变。

2.3.6 UDP 协议

UDP 数据报文分为 UDP 头部和有效数据。UDP 头部由源端口号、目的端口号、长度以及校验和组成, 如下图所示。

相比 TCP 协议, UDP 的传输效率更高, 但其提供不可靠传输, 有可能丢包。各区域的功能如下:



UDP 数据包分为 UDP 头部和有效数据。UDP 头部由源端口号、目的端口号、长度以及校验和组成，如下图所示。相比 TCP 协议，UDP 的传输效率更高，但其提供不可靠传输，有可能丢包。各区域的功能如下：

源端口号（16bit）：源主机的应用程序使用的端口号。

目的端口号（16bit）：目的主机的应用程序使用的端口号。

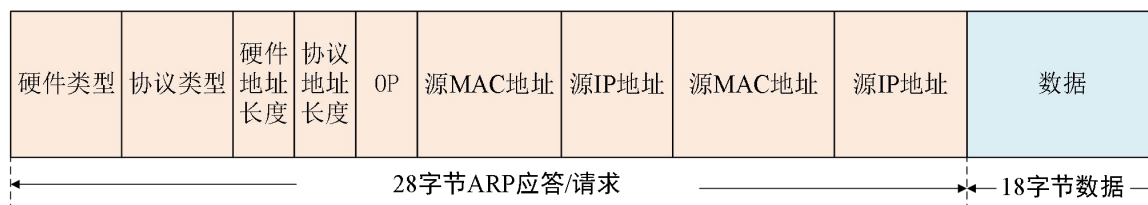
UDP 长度（16bit）：UDP 头部和 UDP 数据的字节长度之和。

UDP 校验和（16bit）：检查数据是否被正确接收，可以不使用。

2.3.7 ARP 协议

ARP 即地址解析协议的基本功能是通过目标设备的 IP 地址，查询目标设备的 MAC 地址。在发送 IP 数据包时，发送方首先查看自己的 ARP 链表，确定 IP 地址对应的 MAC 地址。如果找到了对应的 MAC 地址，则利用 ARP 链表中的 MAC 地址对 IP 数据包进行封装，将数据包发送出去；如果没有找到对应的 MAC 地址，则发送 ARP 广播，请求接收方将自己的 MAC 地址通过 ARP 应答发送到请求方，然后发送方将 ARP 应答包中的地址信息缓存进 ARP 链表中。

ARP 数据包的结构如下图：



硬件类型（2Byte）：硬件地址的类型，以太网的硬件类型为 1。

协议类型（2Byte）：上层协议类型，IP 的协议类型为 16' h0800。

硬件地址长度（1Byte）：硬件地址（MAC 地址）的长度，以字节为单位，其值固定为 8' h06。

协议地址长度（1Byte）：IP 地址的长度，以字节为单位，固定值 8' h04。

OPCODE (2Byte)：操作码，值为 16' h0001 表示 ARP 请求，值为 16' h0002 表示 ARP 应答。

源 MAC 地址 (6Byte)：发送方的 MAC 地址。

源 IP 地址 (4Byte)：发送方的 IP 地址。

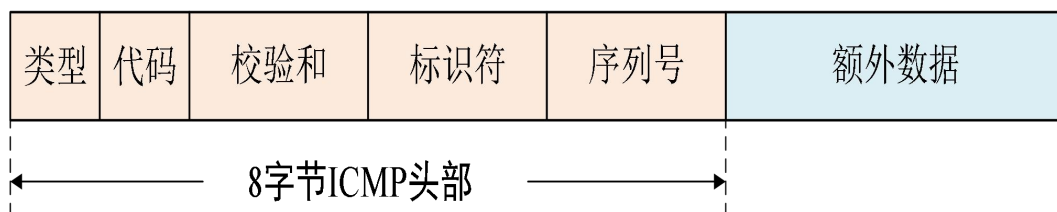
目的 MAC 地址 (6Byte)：发送 ARP 请求时为广播地址 48' hff_ff_ff_ff_ff_ff，发送 ARP 应答时为接收方的 MAC 地址。

目的 IP 地址 (4Byte)：接收端的 IP 地址。

2.3.8 ICMP 协议

ICMP 协议主要用于诊断网络连接故障和测试网络连接，PC 端通常会使用 ping 命令来测试网络，ping 命令就是基于 ICMP 协议实现的。

由于 UDP 协议提供不可靠传输，所以需要和 ICMP 协议配合使用以达到查询和纠错目的。ICMP 报文分为查询报文和差错报文两大类，本协议栈实现了对远程主机发送的 ping 请求回复一个 ping 应答包。ping 请求和 ping 应答都属于查询报文，查询报文的报文格式如下：



类型 (1Byte)：表示 ICMP 报文的类型，不同类型代表不同的功能，如：8' h08 表示回显请求，8' h00 表示回显应答。

代码 (1Byte)：和类型字段结合使用，共同标识了 ICMP 报文的详细类型，回显请求和回显应答是该值为 8' h00。

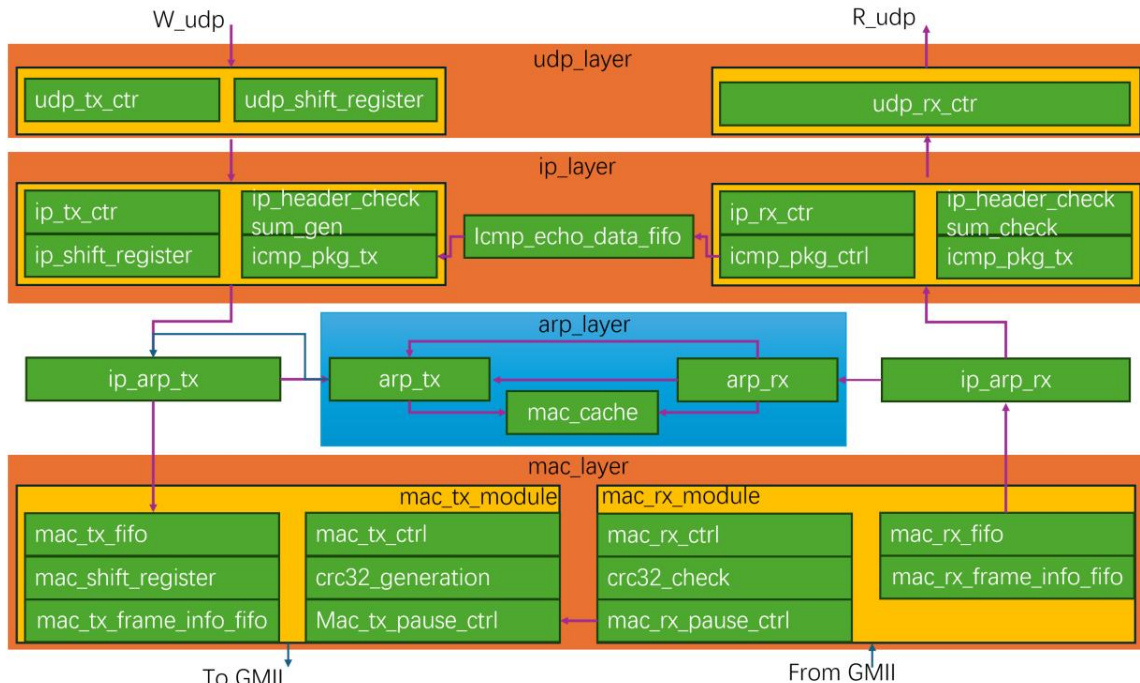
校验和 (2Byte)：用于判断数据传输时是否出了差错，与 IP 首部校验的方式相同。

标识符 (2Byte)：对数据报进行标识，用于区分不同的请求和应答。

序列号 (2Byte)：每次发送完就加 1，和标识符一起用于区分不同的请求和应答。

3 程序设计

前面我们介绍了以太网的基本概念，及涉及的各层协议格式，接下来我们通过设计 Verilog 程序来实现以太网各个子层的功能。程序整体架构图如下：



3.1 MAC 层

MAC 层一边连接 GMII 接口，一边连接上层协议传来的数据。该层将上层传来的数据包组帧发送出去，或者将接收到的数据帧解析，将信息和拆解出来的数据包传给上层协议。

3.1.1 MAC 接收模块

MAC 接收模块主要实现以下几个功能：

- (1) 对接收到的 MAC 帧进行解析，过滤前导码、帧起始定界符、MAC 地址、类型、CRC 校验位，将上层数据包提取出来，并缓存数据包类型，传给上层做判断。
- (2) 对每帧数据进行 CRC 校验，与帧末尾的 4 字节校验位做比较，判断数据的正确性
- (3) 识别接收到的流控帧，将有效信息发送给子层解析，把解析到的暂停时间和源 MAC 地址输出至 MAC 接收模块做进一步处理。

(4) 通过 FIFO 完成 PHY 接收时钟和用户接口时钟之间的时钟域的转换，并将数据包输出至上层。

```

/*****uimac_rx 模块*****/
--以下是米联客设计的 uimac_rx 模块
--本模块主要有以下几个功能
--1. 从外部 PHY 芯片接收 mac 帧，解析 mac 帧首部，进行 mac 地址过滤和帧类型过滤。
--2. 内部子模块 crc32_check 对每帧数据进行 crc32 值的计算，判断数据的正确性。
--3. 识别接收的 mac 流控帧，子模块 mac_frame_ctrl 提取流控帧中的暂停时间和源 mac 地址输出至 uimac_tx 模块。
--4. mac_rx_data_fifo 完成 phy 接收时钟和用户接口时钟之间的时钟域转换，将数据输出。
*****/
`timescale 1ns/1ps
module uimac_rx
(
    input wire [47:0] I_mac_local_addr, //本地 MAC 地址
    input wire I_crc32_en, //使能 CRC 校验
    input wire I_reset, //系统复位
    //MAC 接收数据发送给上层协议
    input wire I_mac_rclk, //接收时钟
    output wire O_mac_rvalid, //MAC 帧数据有效
    output wire [7:0] O_mac_rdata, //MAC 有效数据
    output wire [15:0] O_mac_rdata_type, //MAC 类型
    output wire O_mac_rdata_error,
    //发送 PAUSE 控制到 mac_send
    output wire O_mac_pause_en,
    output wire [21:0] O_mac_pause_time,
    output wire [47:0] O_mac_pause_addr,
    //从硬件层获取的裸 MAC 数据
    input wire I_gmii_rclk, //rgmii 接收时钟
    input wire I_gmii_rvalid, //gmii 接收数据有效使能信号
    input wire [7:0] I_gmii_rdata //gmii 接收数据
);

wire [7:0] mac_rdata;
reg mac_rdata_valid;
reg [15:0] mac_rdata_type;
reg mac_rdata_error;

assign O_mac_rdata = mac_rdata;
assign O_mac_rvalid = mac_rdata_valid;
assign O_mac_rdata_type = mac_rdata_type;
assign O_mac_rdata_error = mac_rdata_error;

reg [10:0] mac_rdata_cnt;
reg mac_wfifo_en; //FIFO 写入数据有效使能
reg mac_rfifo_en; //FIFO 读数据使能

reg crc_en; //crc 校验使能
reg [2:0] crc_cnt; //移位计数器

```

```

wire      [31:0]      crc_data_out;    //crc 校验结果输出
reg       [2:0]       STATE;           //写 FIFO 状态机
reg       [1:0]       S_RFIFO;         //读 FIFO 状态机

reg       [47:0]      dst_mac_addr;    //MAC 帧解析出的目的 MAC 地址（接收方的 MAC 地址）
reg       [47:0]      src_mac_addr;    //MAC 帧解析出的源 MAC 地址（发送方的 MAC 地址）
reg       [15:0]      mac_frame_type;  //上层数据包类型（0x0800 ip;0x0806 arp;0x8808 mac_ctrl）
reg       mac_pause_en;                //PAUSE 帧有效使能
reg       [3:0]       cnt;              //对 MAC 帧头部的字节数计数

reg       [10:0]      mac_wfifo_data_cnt_info; //写帧信息字节数到信息 FIFO
reg       mac_wfifo_en_info;              //写帧信息 FIFO 有效使能
reg       mac_rfifo_en_info;              //读帧信息 FIFO 有效使能
wire      [26:0]      mac_rfifo_data_info; //从信息 FIFO 读出帧信息
reg       [10:0]      mac_rdata_len;       //mac 帧长度
wire      mac_rfifo_empty_info;           //信息 FIFO 读空信号

reg       [7:0]       mac_rdata_r1, mac_rdata_r2, mac_rdata_r3, mac_rdata_r4; //打拍
reg       mac_rvalid_r1, mac_rvalid_r2, mac_rvalid_r3, mac_rvalid_r4; //打拍

localparam WAIT_SFD = 3'd0;
localparam CHECK_MAC_HEADER = 3'd1;
localparam WRITE_FIFO = 3'd2;
localparam RECORD_FRAME_LENGTH = 3'd3;
localparam WAIT_FRAME_END = 3'd4; //STATE

localparam WAIT_MAC_FRAME = 2'd0;
localparam READ_MAC_FRAME_DATA_LENGTH = 2'd1;
localparam READ_MAC_FRAME_DATA = 2'd2; //S_RFIFO

localparam ARP_TYPE = 16'h0806;
localparam IP_TYPE = 16'h0800;
localparam MAC_CONTROL_TYPE = 16'h8808;

```

由于用户接口时钟和 PHY 芯片接收时钟不同源，因此需要对输入的数据进行跨时钟域处理，再将数据传至上层。由于需要传输的数据量比较大，且用 FIFO 做跨时钟域比较简单，所以程序中使用异步 FIFO 做跨时钟域处理。

在 uimac_rx 和 uimac_tx 发送模块中，都使用了两个 FIFO 做跨时钟域处理，一个 FIFO 用来缓存数据，另一个 FIFO 用来缓存需要传递的信息，在一帧输入接收完成后，将信息写入帧信息 FIFO，通过 FIFO 空标志信号来控制一帧数据读出。

信号打 4 拍再写入数据 FIFO 是为了过滤掉 4 字节 CRC 校验位。

```

assign 0_mac_pause_addr = src_mac_addr;

always@(posedge I_gmii_rclk) begin

```

```

    mac_rdata_r1    <=  I_gmii_rdata;
    mac_rdata_r2    <=  mac_rdata_r1;
    mac_rdata_r3    <=  mac_rdata_r2;
    mac_rdata_r4    <=  mac_rdata_r3;
end

always@(posedge I_gmii_rclk) begin
    mac_rvalid_r1    <=  I_gmii_rvalid;
    mac_rvalid_r2    <=  mac_rvalid_r1;
    mac_rvalid_r3    <=  mac_rvalid_r2;
    mac_rvalid_r4    <=  mac_rvalid_r3;
end//打4拍，方便fifo只写入有效数据，而不写入crc校验位

mac_rx_data_fifo mac_rx_data_fifo (
    .rst              (I_reset),
    .wr_clk           (I_gmii_rclk),
    .din              (mac_rdata_r4),
    .wr_en            (mac_wfifo_en & I_gmii_rvalid), //mac_wfifo_en 控制只写入有效数据部分，
    I_gmii_rvalid 控制最后的CRC部分不写入

    .rd_clk           (I_mac_rclk),
    .rd_en            (mac_rfifo_en),
    .dout             (mac_rdata),
    .full             (),
    .empty            (),
    .rd_data_count    (),
    .wr_data_count    ()
);

mac_rx_frame_fifo mac_rx_frame_fifo (
    .rst              (I_reset),
    .wr_clk           (I_gmii_rclk),
    .din              ({mac_wfifo_data_cnt_info, mac_frame_type}),
    .wr_en            (mac_wfifo_en_info),

    .rd_clk           (I_mac_rclk),
    .rd_en            (mac_rfifo_en_info),
    .dout             (mac_rfifo_data_info),
    .full             (),
    .empty            (mac_rfifo_empty_info)
);

crc32_check crc32_check
(
    .reset            (I_reset),
    .clk              (I_gmii_rclk),
    .CRC32_en         (crc_en & I_crc32_en),
    .CRC32_init       (~mac_rvalid_r4),

```

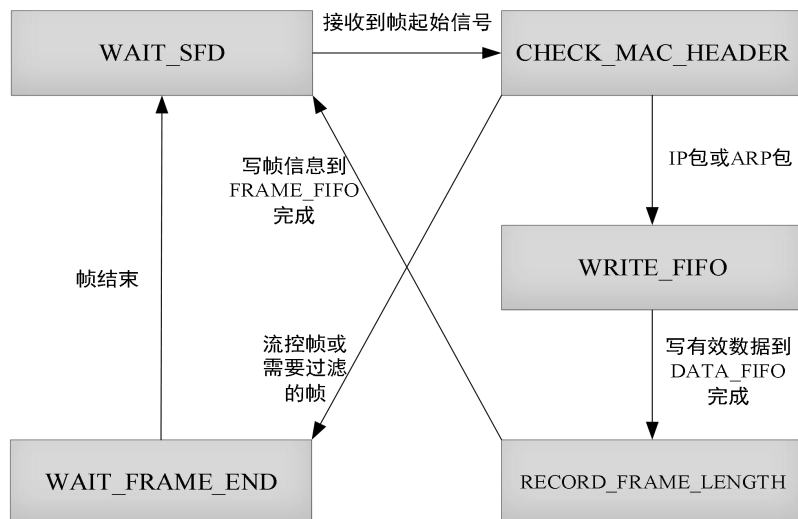
```

.data                (mac_rdata_r4),
.CRC_data            (crc_data_out)
);

//mac 帧控制，当接收方来不及处理接收数据，需要进行帧控制，通知发送模块。
uimac_tx_frame_ctrl mac_tx_frame_ctrl
(
    .I_clk            (I_gmii_rclk),
    .I_reset          (I_reset),
    .I_mac_pause_en   (mac_rvalid_r4 & mac_pause_en),
    .I_mac_data        (mac_rdata_r4),
    .O_mac_pause_en   (O_mac_pause_en),
    .O_mac_pause_time (O_mac_pause_time)
);

```

通过状态机分别对 FIFO 写入数据和 FIFO 读出数据的时序进行控制。FIFO 写入数据的状态机转换图如图所示。



WAIT_SFD：初始状态为 WAIT_SFD，等待接收到帧起始定界符 8' hd5 时，跳转到 CHECK_MAC_HEADER 帧头接收状态。

CHECK_MAC_HEADER：通过计数器将每一字节的帧头信息缓存，帧头信息接收完成后，判断接收到数据包的类型。如果接收到的 MAC 帧类型为 IP 包或者 ARP 包，跳转至 WRITE_FIFO 状态，并将数据 FIFO 写使能拉高，将有效数据写入数据 FIFO，如果接收到的 MAC 帧类型为流量控制帧，则将有效数据传入帧流控制模块，对信息进行解析，跳转至 FRAME_END 状态，状态如果以上情况都不是，丢弃该帧，跳转至 FRAME_END 状态。

WRITE_FIFO：当数据有效信号为高时，将有效数据写入写数据 FIFO，使用 I_gmii_rvalid 做为计数器计数的有效信号，是为了统计到有效数据的准确长度。CRC 校验完成后，将帧长度和帧类型写入帧信息 FIFO 中，进入 RECORD_FRAME_LENGTH 状态。

RECORD_FRAME_LENGTH：帧信息写入帧信息 FIFO 完成后，回到 WAIT_SFD 状态，等待接收下一帧。

WAIT_FRAME_END: 等待不传入上层的帧结束, 回到 WAIT_SFD 状态, 等待接收下一帧。

```

always@(posedge I_gmii_rclk or posedge I_reset) begin
    if(I_reset) begin
        dst_mac_addr        <= 48'd0;
        src_mac_addr        <= 48'd0;
        mac_frame_type      <= 16'd0;
        mac_wfifo_en        <= 1'b0;
        mac_wfifo_en_info   <= 1'b0;
        mac_wfifo_data_cnt_info <= 11'd0;
        cnt                 <= 4'd0;
        crc_en              <= 1'b0;
        crc_cnt             <= 3'd4;
        mac_rdata_error     <= 1'b1;
        mac_pause_en        <= 1'b0;
        STATE               <= WAIT_SFD;
    end
    else begin
        case(STATE)
            WAIT_SFD:begin
                if(mac_rvalid_r4 & (mac_rdata_r4 == 8'h55)) begin//以太网帧开始同步, 一个字节为
mac 字段
                    crc_en <= 1'b1;//使能 crc
                    STATE <= CHECK_MAC_HEADER;//进入帧头接收
                end
                else
                    STATE <= WAIT_SFD;
            end
            CHECK_MAC_HEADER:begin
                case(cnt)
                    4'd0:begin dst_mac_addr[47:40] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end
                    4'd1:begin dst_mac_addr[39:32] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end
                    4'd2:begin dst_mac_addr[31:24] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end
                    4'd3:begin dst_mac_addr[23:16] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end
                    4'd4:begin dst_mac_addr[15: 8] <= mac_rdata_r4; cnt <= cnt
+ 1'b1; end
                    4'd5:begin dst_mac_addr[ 7: 0] <= mac_rdata_r4; cnt <= cnt
+ 1'b1; end//目的 mac
                    4'd6:begin src_mac_addr[47:40] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end
                    4'd7:begin src_mac_addr[39:32] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end
                    4'd8:begin src_mac_addr[31:24] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end

```

```

4' d9:begin src_mac_addr[23:16] <= mac_rdata_r4;
cnt <= cnt + 1'b1; end
4' d10:begin src_mac_addr[15: 8] <= mac_rdata_r4; cnt <= cnt +
1'b1; end
4' d11:begin src_mac_addr[ 7: 0] <= mac_rdata_r4; cnt <= cnt +
1'b1; end
4' d12:begin mac_frame_type[15: 8] <= mac_rdata_r4; cnt <= cnt +
1'b1; end//源 mac
4' d13:begin
    mac_frame_type[7:0] <= mac_rdata_r4;
    cnt <= 4' d0;
    if(dst_mac_addr == I_mac_local_addr) begin//判断 mac 是否一致
        if({mac_frame_type[15:8], mac_rdata_r4} == IP_TYPE ||
{mac_frame_type[15:8], mac_rdata_r4} == ARP_TYPE) begin
            mac_wfifo_en <= 1'b1;//写 fifo 使能, 只写入数据有
效部分
            STATE <= WRITE_FIFO;
        end
        else begin//需要过滤的帧
            mac_wfifo_en <= 1'b0;//禁止写 fifo
            STATE <= WAIT_FRAME_END;//过滤该帧, 等
待帧结束
        end
    end
    else if(dst_mac_addr == 48'h01_80_c2_00_00_01) begin//如果目的地址
为 48'h0180c2000001 (固定值), mac 控制帧, 需要进行 PAUSE 流控制
        mac_wfifo_en <= 1'b0;
        STATE <= WAIT_FRAME_END;
        if({mac_frame_type[15:8], mac_rdata_r4} ==
MAC_CONTROL_TYPE)//报文类型字段, 需要进行 pause 流控制
            mac_pause_en <= 1'b1;//mac 控制帧有效
        else
            mac_pause_en <= 1'b0;
        end
    else if(dst_mac_addr == 48'hff_ff_ff_ff_ff_ff) begin//对于广播地址,
只接收 arp 包, 其余类型的广播包全部过滤
        if({mac_frame_type[15:8], mac_rdata_r4} == ARP_TYPE) begin
            mac_wfifo_en <= 1'b1;//写帧数据 fifo 使能, 只写入
有效数据部分
            STATE <= WRITE_FIFO;
        end
        else begin//需要过滤的帧
            mac_wfifo_en <= 1'b0;
            STATE <= WAIT_FRAME_END;
        end
    end
    else begin//需要过滤的帧
        mac_wfifo_en <= 1'b0;

```

```

STATE                                     <=  WAIT_FRAME_END;

end

end

endcase

end

WRITE_FIFO:begin//将去除首部后的 ip 数据包或者 arp 帧存入 mac_rx_frame_fifo 中,同时对当前数据包的长度进行统计

if(I_gmii_rvalid) begin//写帧信息 fifo
    mac_wfifo_data_cnt_info             <=  mac_wfifo_data_cnt_info + 1'b1;//有效数据计数器

STATE                                     <=  WRITE_FIFO;

end

else begin
    if(crc_cnt == 3'd0) begin//crc 校验
        if(crc_data_out != 32'h704dd7b)
            mac_rdata_error <=  1'b1;//校验正确
        else
            mac_rdata_error <=  1'b0;//校验错误

        mac_wfifo_en             <=  1'b0;
        mac_wfifo_en_info        <=  1'b1;//写帧信息 fifo 使能
        crc_en                   <=  1'b0;
        crc_cnt                  <=  3'd4;
        STATE                    <=  RECORD_FRAME_LENGTH;//写入帧信息到帧信息 fifo

end

else

        crc_cnt <=  crc_cnt - 1'b1;//crc 计算计数器

end

end

RECORD_FRAME_LENGTH:begin//写帧信息完成后,回到状态机 WAIT_SFD
    mac_wfifo_en_info             <=  1'b0;
    mac_wfifo_data_cnt_info <=  11'd0;
    STATE                         <=  WAIT_SFD;//回到帧探测状态机

end

WAIT_FRAME_END:begin//等待帧结束
    if(mac_rvalid_r4)
        STATE                     <=  WAIT_FRAME_END;
    else begin
        crc_en                   <=  1'b0;
        mac_pause_en             <=  1'b0;
        STATE                    <=  WAIT_SFD;

end

end

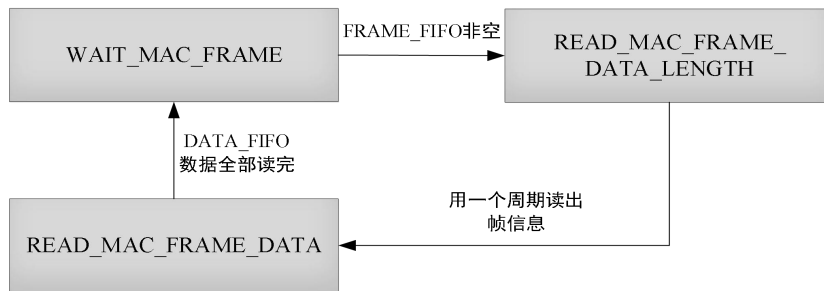
endcase

end

end

```

FIFO 读出数据的状态机转换图如图所示。



WAIT_MAC_FRAME: 当帧信息 FIFO 的空信号拉低, 表明一帧数据已经接收完毕, 将帧信息 FIFO 缓存的帧信息读出, 并跳转至 READ_MAC_FRAME_DATA_LENGTH 状态。

READ_MAC_FRAME_DATA_LENGTH: 该状态下开始读出数据 FIFO 缓存的有效数据, 并将数据有效信号拉高, 开始向上层传输数据包, 随即进入 READ_MAC_FRAME_DATA 状态。

READN_MAC_FRAME_DATA: 对读出的数据数量计数, 当读出的数据量为帧信息 FIFO 中读出的帧长度时, 停止读出数据, 返回 WAIT_MAC_FRAME 状态。

```

always@(posedge I_mac_rclk or posedge I_reset) begin
    if(I_reset) begin
        mac_rfifo_en_info    <= 1'b0;
        mac_rdata_len        <= 11'd0;
        mac_rdata_cnt        <= 11'd0;
        mac_rfifo_en         <= 1'b0;
        mac_rdata_type        <= 16'd0;
        mac_rdata_valid       <= 1'b0;
        S_RFIFO               <= WAIT_MAC_FRAME;
    end
    else begin
        case(S_RFIFO)
            WAIT_MAC_FRAME:begin
                if(!mac_rfifo_empty_info) begin//接收 mac 帧信息 fifo 非空
                    mac_rfifo_en_info    <= 1'b1;
                    S_RFIFO               <= READ_MAC_FRAME_DATA_LENGTH;
                end
            else
                S_RFIFO               <= WAIT_MAC_FRAME;
            end
            READ_MAC_FRAME_DATA_LENGTH:begin
                mac_rdata_len            <= mac_rfifo_data_info[26:16];//mac 帧长度
                mac_rdata_type            <= mac_rfifo_data_info[15:0];//mac 类型
                mac_rfifo_en_info         <= 1'b0;
                mac_rfifo_en              <= 1'b1;//读数据 fifo
                mac_rdata_valid           <= 1'b1;//数据有效
                S_RFIFO                  <= READ_MAC_FRAME_DATA;
            end
        endcase
    end
end
  
```

```

        READ_MAC_FRAME_DATA:begin
            if(mac_rdata_cnt < (mac_rdata_len - 1'b1)) begin//读完一帧数据
                mac_rdata_cnt    <=    mac_rdata_cnt + 1'b1;
                S_RFIFO          <=    READ_MAC_FRAME_DATA;
            end
            else begin
                mac_rfifo_en      <=    1'b0;
                mac_rdata_valid <=    1'b0;
                mac_rdata_cnt    <=    11'd0;
                mac_rdata_len    <=    11'd0;
                mac_rdata_type   <=    16'd0;
                S_RFIFO          <=    WAIT_MAC_FRAME;
            end
        end
    endcase
end
end
end

```

3.1.2 MAC 发送模块

MAC 发送模块主要实现以下功能：

- (1) 接收 IP 或 ARP 数据包，添加 MAC 帧首部，并对长度不足 64 字节的包进行补 0。
- (2) 通过 CRC 校验模块，生成 CRC 校验值添加在帧的末尾。
- (3) 通过流控模块，接收 MAC 接收模块发送的暂停信号，进行流量控制。
- (4) 通过 FIFO 完成 PHY 发送时钟和用户接口时钟之间的时钟域的转换，并将数据输出至外部 PHY 芯片。

该模块数据跨时钟域转换的方式和 uimac_rx 模块类似,使用两个 FIFO 对数据进行处理。通过 DR1_LOGIC_SHIFTER

原语将数据延时,将帧头插入,该方法在 uiip_tx 和 uiudp_tx 模块中也有所体现,该原语可在 TD 安装路径下的 arch

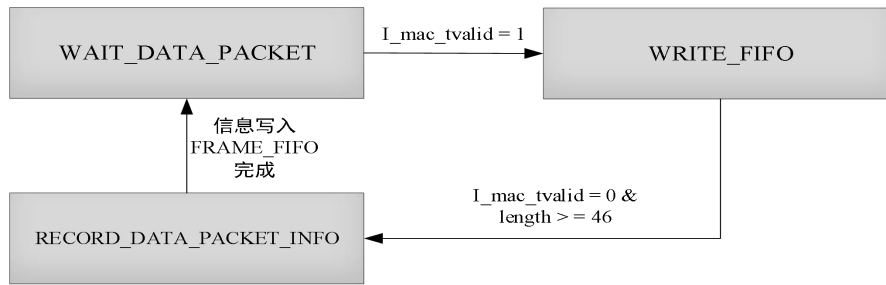
文件夹的 dr1_macro.v 文件中找到。

```

module DR1_LOGIC_SHIFTER (
    clk,
    datain,
    dataout,
    depth,
    en
);
parameter DATA_WIDTH = 16;    // >0.
parameter DEPTH_WIDTH = 2;    // >0.
parameter SHIFT_TYPE = "FIXED"; // "VARIABLE", "FIXED".
parameter DATA_DEPTH = 32;    // if SHIFT_TYPE is "FIXED", SHIFTER depth == DATA_DEPTH; if SHIFT_TYPE is "VARIABLE", SHIFTER depth == 2 ** DEPTH_WIDTH.
input      clk;
input      en;
input [DATA_WIDTH-1:0] datain;
input [DEPTH_WIDTH-1:0] depth;
output reg [DATA_WIDTH-1:0] dataout;
parameter INIT_FILE = "NONE";
endmodule

```

控制 FIFO 写入数据的状态机跳转图如所示。



WAIT_DATA_PACKET: 当数据发送过来后, 开始接收数据并缓存进 FIFO 中, 并通过寄存器缓存帧信息, 进入 WRITE_FIFO 状态。如果数据 FIFO 写计数器大于阈值 (本工程设置为 2500, 具体数值根据 FIFO 深度而定), 则停止接收该帧, 保持原状态不变。

WRITE_FIFO: 写数据到数据 FIFO 中, 并且对写入的数据长度进行计数, 如果数据长度小于 46, 对数据末尾进行补 0, 一帧数据写入完成后, 将长度、类型和地址信息写入帧信息 FIFO, 进入 RECORD_DATA_PACKET_INFO 状态。

RECORD_DATA_PACKET_INFO: 信息写入帧信息 FIFO 完成后, 回到 WAIT_DATA_PACKET 状态, 等待接收下一帧数据。

```

always@(posedge I_mac_tclk or posedge rst) begin
    if(rst) begin
        mac_wfifo_en_info          <= 1'b0;    //MAC 消息 FIFO, 把 MAC 的信息包括, 目的 MAC 地址、
        有效数据长度、帧类型写入到 info fifo 暂存
        mac_wfifo_data_addr_info   <= 48'd0;   //MAC 目的地址, 暂存 info fifo
        mac_wfifo_data_type_info   <= 16'd0;   //MAC 帧类型, 暂存 info fifo
        mac_wfifo_data_cnt_info    <= 11'd0;   //MAC 数据部分发送字节计数器
        mac_wfifo_en               <= 1'b0;    //将帧数据写入到 mac_tx_data_fifo 缓存
        mac_wfifo_data             <= 8'd0;    //将帧数据写入到 mac_tx_data_fifo 缓存
        O_mac_tbusy               <= 1'b1;    //通知外部模块, 非忙
        S_WFIFO                    <= WAIT_DATA_PACKET;
    end
    else begin
        case(S_WFIFO)
            WAIT_DATA_PACKET:begin
                if(mac_wfifo_data_cnt > SEND_PAUSE_THRESHOLD) begin//当 FIFO 写通道数据计数器大于
                SEND_PAUSE_THRESHOLD, 不进行新的一帧传输, O_mac_tbusy 为握手信号, 不进行握手(拉高)
                    O_mac_tbusy          <= 1'b0;
                    S_WFIFO               <= WAIT_DATA_PACKET;
                end
            end
            else begin
                if(I_mac_tvalid) begin//当有效数据发送过来后开始接收数据并且缓存到 FIFO

```

```

                                0_mac_tbusy          <=  1'b1;                                //uima
c_tx 忙
                                mac_wfifo_en          <=  1'b1;                                //将数
据写入 FIFO
                                mac_wfifo_data        <=  I_mac_tdata;                        //写入 FIFO
的数据
                                mac_wfifo_data_addr_info<=  I_mac_tdest_addr;    //目的 MAC 地址
                                mac_wfifo_data_type_info<=  {14'd0, I_mac_tdata_type};//数据类型
                                mac_wfifo_data_cnt_info <=  mac_wfifo_data_cnt_info + 1'b1;    //一帧数
据的长度, 以 BYTE 为单位
                                S_WFIFO              <=  WRITE_FIFO;    //进入下一个状态等待
写 FIFO
                                end
                                else begin
                                0_mac_tbusy          <=  1'b0;                                //uimac_tx
非忙
                                S_WFIFO              <=  WAIT_DATA_PACKET;
                                end
                                end
                                end
                                WRITE_FIFO:begin//写数据到 FIFO 该 FIFO 用于缓存 udp 协议发送过来的数据
                                if(I_mac_tvalid) begin//一帧数据接收过程中 0_gmii_tdata_valid 始终为高电平
                                mac_wfifo_en          <=  1'b1;    //继续写 FIFO
                                mac_wfifo_data        <=  I_mac_tdata;    //写入 FIFO 的数据
                                mac_wfifo_data_cnt_info <=  mac_wfifo_data_cnt_info + 1'b1;    //帧字节
计数器累加
                                S_WFIFO              <=  WRITE_FIFO;
                                end
                                else begin
                                if(mac_wfifo_data_cnt_info < 11'd46) begin//当一包/帧数据的长度小于 46 字节,
自动补 0 (一帧数据最小 64bytes, 其中数据部分最小 46bytes)
                                mac_wfifo_en          <=  1'b1;
                                mac_wfifo_data        <=  8'd0;
                                mac_wfifo_en_info      <=  1'b0;
                                mac_wfifo_data_cnt_info <=  mac_wfifo_data_cnt_info + 1'b1;
                                S_WFIFO              <=  WRITE_FIFO;
                                end
                                else begin//当一包/帧数据接收完, 写包/帧信息 到包/帧信息 FIFO
                                mac_wfifo_en          <=  1'b0;
                                mac_wfifo_data        <=  8'd0;
                                mac_wfifo_en_info      <=  1'b1;
                                S_WFIFO              <=  RECORD_DATA_PACKET_INFO;
                                end
                                end
                                end
                                RECORD_DATA_PACKET_INFO:begin//时序中, 该周期完成写包/帧信息 到包/帧信息 FIFO
                                mac_wfifo_en_info      <=  1'b0;
                                mac_wfifo_data_addr_info <=  48'd0;

```

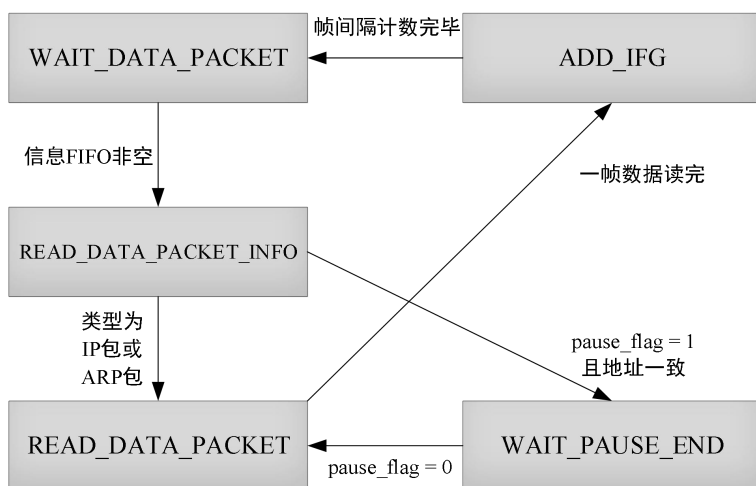
```

mac_wfifo_data_type_info    <= 16'd0;
mac_wfifo_data_cnt_info    <= 11'd0;
S_WFIFO                     <= WAIT_DATA_PACKET;

    end
endcase
end
end

```

控制 FIFO 读出数据的状态机转换图如图所示。



WAIT_DATA_PACKET: 帧信息 FIFO 非空, 说明有帧需要发送, 将帧信息 FIFO 读使能拉高一个时钟周期, 跳转至 READ_DATA_PACKET_INFO 状态。

READ_DATA_PACKET_INFO: 对读出的帧信息进行解析, 通过帧类型得到目地 MAC 地址, 并拉高数据 FIFO 读使能, 开始读出有效数据, 输入进移位寄存器中, 进入 READ_DATA_PACKET 状态。如果 PAUSE 标志信号为高, 且 PAUSE 帧的地址和目的 MAC 地址相同, 说明对方请求暂停发送, 则进入 WAIT_PAUSE_END 状态。

READ_DATA_PACKET: 当一帧数据读完后, 进入 WAIT_CRC_TRANS_DONE 状态。

WAIT_CRC_TRANS_DONE: 等待 CRC 计数器置零, 代表一帧数据发送完成, 进入 ADD_IFG 状态。

ADD_IFG: 等待最小帧间隔结束, 回到 WAIT_DATA_PACKET 状态, 等待接收下一帧。

WAIT_PAUSE_END: 等待 PAUSE 标志信号拉低, 暂停结束, 此时将数据 FIFO 读使能拉高, 继续读该帧的有效数据, 进入 READ_DATA_PACKET 状态。

READ_DATA_PACKET: 将 shift_ram 移位后的数据进行组帧, 通过计数器添加前导码、SFD、目的 MAC、源 MAC

和类型字段，并在帧末尾填充 4 字节 CRC 校验数据。

//完成 MAC 帧的发送，用到了前面的帧缓存 FIFO，信息缓存 FIFO，以及 SHIFT 寄存器(实现 MAC 帧头信息插入)

```
always@(posedge I_gmii_tclk or posedge I_reset) begin
    if(I_reset) begin
        mac_rfifo_data_en          <= 1'b0;
        mac_rfifo_en_info          <= 1'b0;
        mac_rfifo_data_cnt          <= 11'd0;
        mac_rfifo_data_length       <= 11'd0;
        mac_rfifo_data_addr         <= 48'd0;
        ether_type                  <= 16'd0;
        inter_gap_cnt               <= 4'd0;
        S_RFIFO                     <= WAIT_DATA_PACKET;
    end
    else begin
        case(S_RFIFO)
            WAIT_DATA_PACKET:begin
                if(!mac_rfifo_empty_info) begin//帧信息 FIFO 非空代表有帧需要发送
                    mac_rfifo_en_info          <= 1'b1;//FIFO 是设置的 FWT 模式，如果只有 1 帧数据，那么 FIFO 被读空，否则 FIFO 输出更新到下一帧
                    S_RFIFO                     <= READ_DATA_PACKET_INFO;
                end
                else
                    S_RFIFO                     <= WAIT_DATA_PACKET;
            end
            READ_DATA_PACKET_INFO:begin
                if(mac_rfifo_data_info[15:0] == 16'h0002) begin//发送的 ARP 包类型为应答包
                    ether_type                  <= ARP_PACKET;
                    mac_rfifo_data_addr         <= mac_rfifo_data_info[74:27];//MAC 地址
                end
                else if(mac_rfifo_data_info[15:0] == 16'h0003) begin
                    ether_type                  <= ARP_PACKET;
                    mac_rfifo_data_addr         <= 48'hff_ff_ff_ff_ff_ff;//广播地址
                end
                else begin
                    ether_type                  <= IP_PACKET;//IP 包
                    mac_rfifo_data_addr         <= mac_rfifo_data_info[74:27];//MAC 地址
                end

                mac_rfifo_data_length          <= mac_rfifo_data_info[26:16];//数据长度
                mac_rfifo_en_info              <= 1'b0;

                if(pause_flag && mac_rfifo_data_info[74:27] == pause_dst_mac_addr) begin//如果存在 PAUSE 帧需要发送，并且目的地址和当前目的地址一致
                    mac_rfifo_data_en          <= 1'b0;//PAUSE 帧阶段不从 FIFO 读数据
                    S_RFIFO                     <= WAIT_PAUSE_END;//等待 PAUSE 流控制结束
                end
            end
        end
    end
end
```

```

        mac_rfifo_data_en      <= 1'b1;
        S_RFIFO                <= READ_DATA_PACKET;

    end

end

READ_DATA_PACKET:begin
    if(mac_rfifo_data_cnt == (mac_rfifo_data_length - 1'b1)) begin//一帧数据从 FIFO 读完(上一个状态已经读出了一个周期, 所以这里少计一个数)
        mac_rfifo_data_en      <= 1'b0;
        mac_rfifo_data_length  <= 11'd0;
        mac_rfifo_data_cnt     <= 11'd0;
        mac_rfifo_data_addr    <= 48'd0;
        ether_type             <= 16'd0;
        S_RFIFO                <= WAIT_CRC_TRANS_DONE;

    end

    else begin
        mac_rfifo_data_en      <= 1'b1;
        mac_rfifo_data_cnt     <= mac_rfifo_data_cnt + 1'b1;
        S_RFIFO                <= READ_DATA_PACKET;

    end

end

WAIT_CRC_TRANS_DONE:begin//等待正在发送的 MAC 数据包 CRC 发送完成
    if(crc_cnt)
        S_RFIFO                <= WAIT_CRC_TRANS_DONE;

    else
        S_RFIFO                <= ADD_IFG;

    end

end

ADD_IFG:begin//数据包发送后, 插入帧间隔, 2 帧之间最少需要 IFGmini=96bit/speed, 比如 100M
96ns 100M 960ns 10M 9600ns
    if(inter_gap_cnt == (IFG - 4'd4)) begin//插入最小帧间隔周期, 在此状态机,
MAC_SEND_FLOW_CONTROL 流控制模可以发送 PAUSE 帧, 减去 4'd4 是本计数器结束后, 距离下一帧发送实际需要还要经过
4 个时钟周期

        inter_gap_cnt          <= 4'd0;
        S_RFIFO                <= WAIT_DATA_PACKET;//进入
WAIT_DATA_PACKET

    end

    else begin
        inter_gap_cnt          <= inter_gap_cnt + 1'b1;
        S_RFIFO                <= ADD_IFG;

    end

end

WAIT_PAUSE_END:begin//等待暂停结束后重新传输数据
    if(pause_flag) begin//pause 控制
        mac_rfifo_data_en      <= 1'b0;
        S_RFIFO                <= WAIT_PAUSE_END;

    end

    else begin
        mac_rfifo_data_en      <= 1'b1;//暂停结束后, 继续读帧 FIFO 中数据
        S_RFIFO                <= READ_DATA_PACKET;

    end
end

```

```

        end
    end
endcase
end
end

always@(posedge I_gmii_tclk or posedge I_reset) begin
    if(I_reset) begin
        O_gmii_tvalid      <= 1'b0;
        mac_tdata          <= 8'd0;
        mac_tdata_crc_en   <= 1'b0;
        data22_cnt         <= 5'd0;
        data22_shift_cnt   <= 5'd22;
        crc_cnt            <= 3'd4;
        crc_read           <= 1'b0;
    end
    else if(mac_rfifo_data_en) begin
        case(data22_cnt)//这个阶段移位寄存器进行数据的填充
            0 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1;
O_gmii_tvalid <= 1'b1; data22_shift_cnt <= 5'd22;end
            1 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1; end
            2 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1; end
            3 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1; end
            4 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1; end
            5 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1; end
            6 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1; end
            7 :begin mac_tdata <= 8'h55; data22_cnt <= data22_cnt + 1'b1; end

            8 :begin mac_tdata <= mac_rfifo_data_addr[47:40];
data22_cnt <= data22_cnt + 1'b1;
mac_tdata_crc_en <= 1'b1; end
            9 :begin mac_tdata <= mac_rfifo_data_addr[39:32];
data22_cnt <= data22_cnt + 1'b1; end
            10 :begin mac_tdata <= mac_rfifo_data_addr[31:24];
data22_cnt <= data22_cnt + 1'b1; end
            11 :begin mac_tdata <= mac_rfifo_data_addr[23:16];
data22_cnt <= data22_cnt + 1'b1; end
            12 :begin mac_tdata <= mac_rfifo_data_addr[15:8]; data22_cnt <= data22_
cnt + 1'b1; end
            13 :begin mac_tdata <= mac_rfifo_data_addr[7:0]; data22_cnt <= data22_
cnt + 1'b1; end
            14 :begin mac_tdata <= I_mac_local_addr[47:40]; data22_cnt <= data22
_cnt + 1'b1; end
            15 :begin mac_tdata <= I_mac_local_addr[39:32]; data22_cnt <= data22
_cnt + 1'b1; end
            16 :begin mac_tdata <= I_mac_local_addr[31:24]; data22_cnt <= data22
_cnt + 1'b1; end
            17 :begin mac_tdata <= I_mac_local_addr[23:16]; data22_cnt <= data22

```

3.1.3 CRC 校验模块

27

则校验正确, 该值由 CRC32 多项式 $32'h1cdf4421$ 经过位反演和字节反转得到。在发送和接收 CRC 校验和时, 其值是以低字节序进行传输的, 即最高位在末尾, 发送时需要注意将 CRC 校验结果转为低字节序。

```

`timescale 1ns / 1ps
module crc32_gen(
input    reset,
input    clk,
input    CRC32_en,           //CRC 校验使能信号
input    CRC32_init,        //CRC 校验值初始化信号
input    CRC_read,
        //input    CRC32_valid,      //CRC 校验值维持有效
input    [7:0]    data,
output    [7:0]    CRC_out
);
reg [31:0]    CRC_temp;
assign CRC_out = CRC_read ? {CRC_temp[24], CRC_temp[25], CRC_temp[26], CRC_temp[27],
                            CRC_temp[28], CRC_temp[29], CRC_temp[30], CRC_temp[31]} :
8'h00;

always@(posedge clk or posedge reset)
    if(reset)
        CRC_temp <= 32'hfffffff;
    else if(CRC32_init)
        CRC_temp <= 32'hfffffff;
    else if(CRC32_en)
        begin
CRC_temp[0]<=CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[1]<=CRC_temp[25]^CRC_temp[31]^data[0]^data[6]^CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[2]<=CRC_temp[26]^data[5]^CRC_temp[25]^CRC_temp[31]^data[0]^data[6]^CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[3]<=CRC_temp[27]^data[4]^CRC_temp[26]^data[5]^CRC_temp[25]^CRC_temp[31]^data[0]^data[6];
CRC_temp[4]<=CRC_temp[28]^data[3]^CRC_temp[27]^data[4]^CRC_temp[26]^data[5]^CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[5]<=CRC_temp[29]^data[2]^CRC_temp[28]^data[3]^CRC_temp[27]^data[4]^CRC_temp[25]^CRC_temp[31]^data[0]^data[6]^CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[6]<=CRC_temp[30]^data[1]^CRC_temp[29]^data[2]^CRC_temp[28]^data[3]^CRC_temp[26]^data[5]^CRC_temp[25]^CRC_temp[31]^data[0]^data[6];
CRC_temp[7]<=CRC_temp[31]^data[0]^CRC_temp[29]^data[2]^CRC_temp[27]^data[4]^CRC_temp[26]^data[5]^CRC_temp[24]^data[7];
CRC_temp[8]<=CRC_temp[0]^CRC_temp[28]^data[3]^CRC_temp[27]^data[4]^CRC_temp[25]^data[6]^CRC_temp[24]^data[7];
CRC_temp[9]<=CRC_temp[1]^CRC_temp[29]^data[2]^CRC_temp[28]^data[3]^CRC_temp[26]^data[5]^CRC_temp[25]^data[6];
CRC_temp[10]<=CRC_temp[2]^CRC_temp[29]^data[2]^CRC_temp[27]^data[4]^CRC_temp[26]^data[5]^CRC_temp[24]^data[7];

```

```

CRC_temp[11]<=CRC_temp[3]^CRC_temp[28]^data[3]^CRC_temp[27]^data[4]^CRC_temp[25]^data[6]^CRC_temp[24]^data[7];
CRC_temp[12]<=CRC_temp[4]^CRC_temp[29]^data[2]^CRC_temp[28]^data[3]^CRC_temp[26]^data[5]^CRC_temp[25]^data[6]^CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[13]<=CRC_temp[5]^CRC_temp[30]^data[1]^CRC_temp[29]^data[2]^CRC_temp[27]^data[4]^CRC_temp[26]^data[5]^CRC_temp[25]^CRC_temp[31]^data[0]^data[6];
CRC_temp[14]<=CRC_temp[6]^CRC_temp[31]^data[0]^CRC_temp[30]^data[1]^CRC_temp[28]^data[3]^CRC_temp[27]^data[4]^CRC_temp[26]^data[5];
CRC_temp[15]<=CRC_temp[7]^CRC_temp[31]^data[0]^CRC_temp[29]^data[2]^CRC_temp[28]^data[3]^CRC_temp[27]^data[4];
CRC_temp[16]<=CRC_temp[8]^CRC_temp[29]^data[2]^CRC_temp[28]^data[3]^CRC_temp[24]^data[7];
CRC_temp[17]<=CRC_temp[9]^CRC_temp[30]^data[1]^CRC_temp[29]^data[2]^CRC_temp[25]^data[6];
CRC_temp[18]<=CRC_temp[10]^CRC_temp[31]^data[0]^CRC_temp[30]^data[1]^CRC_temp[26]^data[5];
CRC_temp[19]<=CRC_temp[11]^CRC_temp[31]^data[0]^CRC_temp[27]^data[4];
CRC_temp[20]<=CRC_temp[12]^CRC_temp[28]^data[3];
CRC_temp[21]<=CRC_temp[13]^CRC_temp[29]^data[2];
CRC_temp[22]<=CRC_temp[14]^CRC_temp[24]^data[7];
CRC_temp[23]<=CRC_temp[15]^CRC_temp[25]^data[6]^CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[24]<=CRC_temp[16]^CRC_temp[26]^data[5]^CRC_temp[25]^CRC_temp[31]^data[0]^data[6];
CRC_temp[25]<=CRC_temp[17]^CRC_temp[27]^data[4]^CRC_temp[26]^data[5];
CRC_temp[26]<=CRC_temp[18]^CRC_temp[28]^data[3]^CRC_temp[27]^data[4]^CRC_temp[24]^CRC_temp[30]^data[1]^data[7];
CRC_temp[27]<=CRC_temp[19]^CRC_temp[29]^data[2]^CRC_temp[28]^data[3]^CRC_temp[25]^CRC_temp[31]^data[0]^data[6];
CRC_temp[28]<=CRC_temp[20]^CRC_temp[30]^data[1]^CRC_temp[29]^data[2]^CRC_temp[26]^data[5];
CRC_temp[29]<=CRC_temp[21]^CRC_temp[31]^data[0]^CRC_temp[30]^data[1]^CRC_temp[27]^data[4];
CRC_temp[30]<=CRC_temp[22]^CRC_temp[31]^data[0]^CRC_temp[28]^data[3];
CRC_temp[31]<=CRC_temp[23]^CRC_temp[29]^data[2];
    end
    else if(CRC_read)
        CRC_temp <= {CRC_temp[23:0], 8'hff};
endmodule

```

3.1.4 PAUSE 帧流控制模块

uimac_rx 模块接收到 PAUSE 帧后, 会将有效数据段送入 mac_tx_frame_ctrl 模块进行解析, 得到暂停时间和

PAUSE 帧发送方的 MAC 地址, 将其发送给 uimac_tx 模块的 mac_tx_pause_ctrl 子模块。

```

/*****mac_tx_frame_ctrl 模块*****/
--以下是米联客设计的 mac_tx_frame_ctrl 模块, 用于产生 MAC 发送模块的 PAUSE 暂停发送
1.
*****/
`timescale 1ns/1ps
module    uimac_tx_frame_ctrl
(
    input    wire          I_clk,

```

```

input  wire          I_reset,
input  wire          I_mac_pause_en,
input  wire          [7:0] I_mac_data,
output reg          O_mac_pause_en,
output reg          [21:0] O_mac_pause_time//发送 MAC 停止发送数据的时间
);

reg    [15:0]          opcode;
reg    [15:0]          pause_time;//pause_time 字段为发送 MAC 停止发送数据的时间，每单位为
512bit 传输时间，比如数值为 16' d1024 表示暂停时间为 MAC 传输 1024*512bit 数据所需要的时间
reg    [2:0]          cnt;
reg                               STATE;

localparam  READ_FRAME      = 0;
localparam  WAIT_FRAME_END  = 1;

localparam  PAUSE_FRAME     = 16'h0001;//操作码，固定值为 0x0001

always@(posedge I_clk or posedge I_reset) begin
    if(I_reset) begin
        cnt          <= 3'd0;
        opcode        <= 16'd0;
        pause_time    <= 16'd0;
        O_mac_pause_en <= 1'b0;
        O_mac_pause_time <= 22'd0;
        STATE         <= READ_FRAME;
    end
    else begin
        case (STATE)
            READ_FRAME:begin
                if(I_mac_pause_en)//帧流控制有效
                    case(cnt)
                        0:begin opcode[15: 8] <= I_mac_data; cnt <= cnt + 1'b1;end
                        1:begin
                            opcode[ 7: 0] <= I_mac_data;
                            if({opcode[15: 8], I_mac_data} == PAUSE_FRAME) begin//判断是
PAUSE 帧
                                STATE <= READ_FRAME;
                                cnt <= cnt + 1'b1;
                            end
                        end
                        else begin
                            STATE <= WAIT_FRAME_END;
                            cnt <= 3'd0;
                        end
                    end
                end
                2:begin pause_time[15: 8] <= I_mac_data; cnt <= cnt + 1'b1;end
                3:begin pause_time[ 7: 0] <= I_mac_data; cnt <= cnt + 1'b1;end//
需要暂停发送的时间

```

```

                                4:begin
                                    cnt                                <=    3'd0;
                                    opcode                            <=    16'd0;
                                    pause_time                        <=    16'd0;
                                    O_mac_pause_en                    <=    1'b1;//通知 MAC 发送控制器，接收到了
PAUSE 帧
                                    O_mac_pause_time                <=    {pause_time, 6'd0};//*512/8 = *64 =
*(2^6)
                                    STATE                            <=    WAIT_FRAME_END;//等待帧结束
                                end
                                endcase
                                else
                                    STATE    <=    READ_FRAME;
                                end
                                WAIT_FRAME_END:begin//等待帧结束
                                    O_mac_pause_time                <=    22'd0;
                                    O_mac_pause_en                    <=    1'b0;
                                    if(I_mac_pause_en)
                                        STATE    <=    WAIT_FRAME_END;
                                    else
                                        STATE    <=    READ_FRAME;
                                    end
                                end
                                endcase
                                end
                                end
                                endmodule

```

mac_tx_pause_ctrl 接收到 mac_pasue_en 信号为高时，将接收的信息寄存，状态机跳转，等待 MAC 发送端发送完一帧数据，进入帧间隔等待。当 MAC 发送模块进入帧间隔状态后，流控模块拉高 pause_flag 信号，等待暂停时间结束后将信号拉低。

```

/*****mac_tx_pause_ctrl 模块*****/
--以下是米联客设计的 mac_tx_pause_ctrl MAC 发送端，流控制器模块
1.
*****/
`timescale 1ns/1ps
module    uimac_tx_pause_ctrl
(
    input    wire                I_clk,
    input    wire                I_reset,
    input    wire    [2:0]       I_mac_state,
    input    wire                I_mac_pause_en,
    input    wire    [21:0]      I_mac_pause_time,
    input    wire    [47:0]      I_mac_pause_addr,
    output    reg    [47:0]      O_pause_dst_mac_addr,
    output    reg                O_pause_flag
)

```

```

);

reg      [21:0]      pause_clk_num;
reg      [21:0]      pause_clk_cnt;
reg      [1:0]       STATE;

localparam WAIT_PAUSE_FRAME      = 2'd0;
localparam WAIT_CURRENT_SEND_DONE = 2'd1;
localparam MAC_SEND_PAUSE        = 2'd2;

localparam ADD_IFG      = 3'd4;

always@(posedge I_clk or posedge I_reset) begin
    if(I_reset) begin
        pause_clk_num      <= 22'd0;
        pause_clk_cnt      <= 22'd0;
        O_pause_flag       <= 1'b0;
        O_pause_dst_mac_addr <= 48'd0;
        STATE               <= WAIT_PAUSE_FRAME;
    end
    else begin
        case(STATE)
            WAIT_PAUSE_FRAME:begin//等待 PAUSE 帧
                O_pause_flag      <= 1'b0;
                if(I_mac_pause_en) begin //MAC 接收模块接收到 PAUSE 帧
                    O_pause_dst_mac_addr <= I_mac_pause_addr;//MAC 发送模块需要发送 PAUSE
的目的 MAC 地址
                    pause_clk_num      <= I_mac_pause_time;//PAUSE 时间，在 MAC 接收
端已经换算好需要 PAUSE 的时钟周期个数
                    STATE               <= WAIT_CURRENT_SEND_DONE;
                end
            end
            else begin
                O_pause_dst_mac_addr <= 48'd0;
                pause_clk_num      <= 22'd0;
                STATE               <= WAIT_PAUSE_FRAME;
            end
        end
        WAIT_CURRENT_SEND_DONE:begin//等待当 MAC 发送状态机在 I_mac_state == ADD_IFG 状态的时候，
设置 O_pause_flag 标志
        if(I_mac_state == ADD_IFG) begin
            O_pause_flag <= 1'b1;//设置 O_pause_flag, 通知 MAC 帧发送
模块，暂停数据发送
            STATE        <= MAC_SEND_PAUSE;
        end
        else begin
            O_pause_flag <= 1'b0;
            STATE        <= WAIT_CURRENT_SEND_DONE;
        end
    end
end

```

```

        end
    end
    MAC_SEND_PAUSE:begin//暂停数据发送, 等待(pause_clk_num - 3)个时钟周期
        if(pause_clk_cnt == (pause_clk_num - 3)) begin
            O_pause_flag                <= 1'b0;
            O_pause_dst_mac_addr        <= 48'd0;
            pause_clk_cnt                <= 22'd0;
            pause_clk_num                <= 22'd0;
            STATE                        <= WAIT_PAUSE_FRAME;

        end
        else begin
            O_pause_flag                <= 1'b1;//设置 O_pause_flag, 通知 MAC 帧发送
            pause_clk_cnt                <= pause_clk_cnt + 1'b1;
            STATE                        <= MAC_SEND_PAUSE;

        end
    end
endcase
end
end
endmodule

```

3.2 IP_ARP 层

由于 IP 和 ARP 数据包送至 MAC 层要经过同一个通道, 需要对发送的数据包类型进行判断和仲裁, 这就需要额外增加一个 IP_ARP 层。

3.2.1 IP_ARP 接收模块

该模块接收到 MAC 帧经过 MAC 层解包得到的数据包, 通过类型字段判断该包是 IP 包还是 ARP 包, 将其送入对应的模块中处理。

```

/*****uiip_arp_rx 模块*****/
--以下是米联客设计的 uiip_arp_rx 模块
1. 该模块 1 用于区分接收数据是 IP 包还是 ARP 包
*****/
`timescale 1ns/1ps
module uiip_arp_rx
(
    input wire I_ip_arp_reset, //复位
    input wire I_ip_arp_rclk, //RX 接收时钟
    output wire O_ip_rvalid, //接收的有效 IP 信号
    output wire [7:0] O_ip_rdata, //接收的 IP 数据

```

```

output wire          0_arp_rvalid,          //接收的有效 ARP 信号
output wire          [7:0] 0_arp_rdata,      //接收的有效 ARP 数据

input  wire          I_mac_rvalid,          //MAC 接收到的数据有效信号
input  wire          [7:0] I_mac_rdata,      //MAC 接收的有效数据
input  wire          [15:0] I_mac_rdata_type //MAC 接收到的帧类型

);
reg          ip_rx_data_valid; //接收的有效 IP 信号
reg [7:0]    ip_rx_data;       //接收的 IP 数据
reg          arp_rx_data_valid; //接收的有效 ARP 信号
reg [7:0]    arp_rx_data;       //接收的有效 ARP 数据

assign 0_ip_rvalid    = ip_rx_data_valid;
assign 0_ip_rdata     = ip_rx_data;
assign 0_arp_rvalid   = arp_rx_data_valid;
assign 0_arp_rdata    = arp_rx_data;

localparam ARP_TYPE   = 16'h0806; //ARP 包类型
localparam IP_TYPE    = 16'h0800; //IP 包类型
always@(posedge I_ip_arp_rclk or posedge I_ip_arp_reset) begin
    if(I_ip_arp_reset) begin
        ip_rx_data_valid    <= 1'b0;
        ip_rx_data          <= 8'd0;
        arp_rx_data_valid   <= 1'b0;
        arp_rx_data         <= 8'd0;
    end
    else if(I_mac_rvalid) begin
        if(I_mac_rdata_type == IP_TYPE) begin//IP 包
            ip_rx_data_valid    <= 1'b1;
            ip_rx_data          <= I_mac_rdata;
        end
        else if(I_mac_rdata_type == ARP_TYPE) begin//ARP 包
            arp_rx_data_valid   <= 1'b1;
            arp_rx_data         <= I_mac_rdata;
        end
        else begin
            ip_rx_data_valid    <= 1'b0;
            ip_rx_data          <= 8'd0;
            arp_rx_data_valid   <= 1'b0;
            arp_rx_data         <= 8'd0;
        end
    end
    else begin
        ip_rx_data_valid    <= 1'b0;
        ip_rx_data          <= 8'd0;
        arp_rx_data_valid   <= 1'b0;
        arp_rx_data         <= 8'd0;
    end
end

```

```

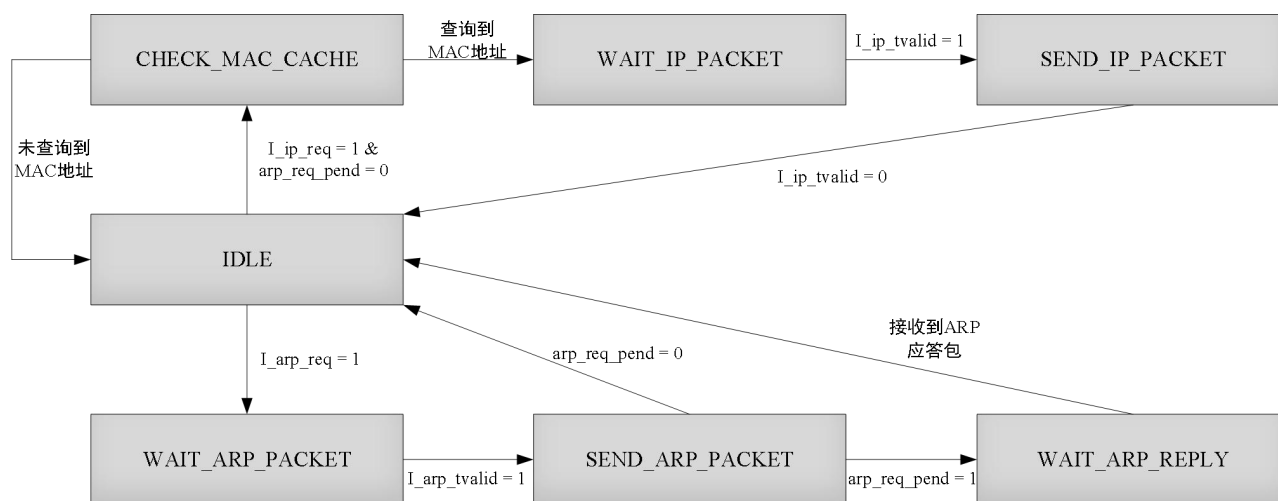
end
end
endmodule

```

3.2.2 IP_ARP 发送模块

该模块接收 IP 层和 ARP 层传来的发送请求，通过 busy 信号与上层协议模块进行握手，来发送对应的数据。

该模块的状态机转换图如图所示。



IDLE: 如果是 IP 层发送的请求，且 arp_req_pend 信号没有挂起时，进入 CHECK_MAC_CACHE 状态，进入 ARP 层查询 mac_cache 中缓存的 MAC 地址。如果是 ARP 层发送的请求，在 MAC 层非忙时，将 arp_tbusy 拉高，表示可以发送 ARP 包，进入 WAIT_ARP_PACKET 状态。

CHECK_MAC_CACHE: 若没有查询到 IP 地址对应的 MAC 地址，则使能 O_arp_treq_en 信号，请求 ARP 层发送 ARP 广播包，并且将 arp_req_pend 挂起（该信号挂起时不能发送 IP 包），回到 IDLE 状态，等待 ARP 层发送请求信号。若查询到 MAC 地址，进入 WAIT_IP_PACKET 状态，等待 IP 层将有效数据发送过来。

WAIT_IP_PACKET: 若 I_ip_valid 拉高，说明 IP 包有效数据开始传入，接收传来的数据并将其发送至 MAC 层，进入 SEND_IP_PACKET 状态。

SEND_IP_PACKET: 等待一帧 IP 包数据全部发送完成时，回到 IDLE 状态。

WAIT_ARP_PACKET: I_arp_valid 拉高，说明 ARP 包有效数据开始传入，接收传来的数据并将其发送至 MAC 层，进入 SEND_ARP_PACKET 状态。

SEND_ARP_PACKET: 如果发送的数据包是 ARP 应答包, 数据全部发送完成时, 回到 IDLE 状态。如果发送的数据包是 ARP 请求包, 则要进入 SEND_ARP_REPLY 状态, 等待接收到对方发送 ARP 应答包。

SEND_ARP_REPLY: 接收到 ARP 应答包后, 将 arp_req_pend 信号拉低, 回到 IDLE 状态。若超时未收到应答, 则回到 IDLE 状态, 此时由于 arp_req_pend 信号一直为高, 该模块会持续发送 ARP 请求直至收到应答。

```

always@(posedge I_ip_arp_clk or posedge I_ip_arp_reset)begin
    if(I_ip_arp_reset) begin
        O_mac_cache_ren          <= 1'b0;    //查询 MAC cache
        O_mac_cache_rip_addr     <= 32'd0;    //查询 MAC cache 地址
        O_arp_tbusy              <= 1'b0;    //ip_arp_tx arp 发送准备好
        O_arp_treq_en            <= 1'b0;    //ip_arp_tx arp 请求发送 ARP 包（当发送 IP 包，没有找
打 cache 中的 MAC 的时候发送）
        O_arp_treq_ip_addr       <= 32'd0;    //ARP 可以发送模块通过发送带有目的 IP 地址的 ARP 请求，
获取目的远程主机的 MAC 地址

        O_ip_tbusy               <= 1'b0;    //ip_arp_tx 可以发送 IP 包

        O_mac_tdata_type         <= 2'd0;    //MAC 发送数据类型
        O_mac_tvalid             <= 1'b0;    //MAC 发送数据有效
        O_mac_tdata              <= 8'd0;    //MAC 发送数据
        O_mac_tdest_addr         <= 48'd0;    //MAC 发送地址

        tmac_addr_temp           <= 48'd0;
        arp_req_pend              <= 1'b0;
        dst_ip_unreachable        <= 1'b0;
        arp_wait_time             <= 30'd0;
        STATE                     <= IDLE;
    end
    else begin
        case(STATE)
            IDLE:begin
                O_arp_treq_en      <= 1'b0;
                if(!I_mac_tbusy) begin//MAC 层不忙
                    if(I_arp_treq) begin//是否有 ARP 请求
                        O_arp_tbusy <= 1'b1;    //可以发送
                        ARP 包
                        O_ip_tbusy  <= 1'b0;
                        STATE       <= WAIT_ARP_PACKET;//等待 ARP 响应
                    end
                    else if(I_ip_treq && ~arp_req_pend) begin //如果是 IP 请求，并且之前的 ARP
                        请求没有 pend
                        O_arp_tbusy <= 1'b0;
                        O_ip_tbusy  <= 1'b0;
                        O_mac_cache_ren <= 1'b1;    //如果是
                        IP 请求，先从 mac cache 通过 IP 地址获取 MAC 地址
                    end
                end
            end
        endcase
    end
end

```

```

                                0_mac_cache_rip_addr    <=  I_ip_tdest_addr;    //通过 IP 地址查
询 MAC cache

                                STATE                  <=  CHECK_MAC_CACHE;

                                end
                                else begin
                                    0_arp_tbusy          <=  1'b0;
                                    0_ip_tbusy           <=
1'b0;

                                    STATE                <=

IDLE;

                                end
                                end
                                else begin
                                    0_arp_tbusy          <=  1'b0;
                                    0_ip_tbusy           <=  1'b0;
                                    0_mac_cache_ren       <=  1'b0;
                                    0_mac_cache_rip_addr  <=  48'd0;
                                    STATE                <=  IDLE;

                                end
                                end
                                CHECK_MAC_CACHE:begin//查询 MAC cache, 如果没有查到 MAC 会请求 ARP 层发送 ARP 请求
                                    0_mac_cache_ren      <=  1'b0;
                                    if(I_mac_cache_rdone) begin                    //MAC cache 查询完成
                                        if(I_mac_cache_rdest_addr == 48'd0) begin //如果没有查询到对应的 MAC, 请求
ARP 层发送 ARP 请求

                                            0_arp_treq_en    <=  1'b1;    //请求 ARP 层发送 ARP
                                            0_ip_tbusy      <=  1'b0;
                                            0_arp_treq_ip_addr <=  0_mac_cache_rip_addr;    //如果没有
查询到 MAC 需要根据提供的 IP 地址请求 ARP 层发送 ARP 包获取 MAC

                                            arp_req_pend    <=  1'b1;
//arp 请求 Pend 结束前不处理其他的 arp 请求
                                            STATE          <=  IDLE;

                                            //回到 IDLE 状态, 等待 ARP 层发送 ARP 包
                                            end
                                            else begin
                                                tmac_addr_temp <=  I_mac_cache_rdest_addr; //从 MAC
cache 查询到 MAC 地址

                                                0_ip_tbusy    <=  1'b1;

                                                //返回 IP 层的 ACK

                                                0_arp_treq_en    <=  1'b0;
                                                arp_req_pend    <=  1'b0;
                                                STATE          <=  WAIT_IP_PACKET;

                                            end
                                            end
                                            else
                                                STATE          <=  CHECK_MAC_CACHE;

                                            end
                                WAIT_ARP_REPLY:begin//等待远程主机的 ARP 响应(ARP 层的 recieve 模块会接收到 ARP 响应)

```

```

        if(I_arp_treply_done) begin//响应
            arp_req_pend          <= 1'b0;
            arp_wait_time         <= 30'd0;
            dst_ip_unreachable    <= 1'b0;
            STATE                 <= IDLE;
        end
    else begin
        if(arp_wait_time == ARP_TIMEOUT_VALUE) begin//超时, 未收到响应
            arp_req_pend          <= 1'b1;
            O_arp_tbusy           <= 1'b0;
            O_arp_treq_en         <= 1'b1;
            O_arp_treq_ip_addr    <= I_ip_tdest_addr;
            dst_ip_unreachable    <= 1'b1;
            arp_wait_time         <= 30'd0;
            STATE                 <= IDLE;

            end
        else begin
            arp_req_pend          <= 1'b1;
            O_arp_tbusy           <= 1'b1;
            dst_ip_unreachable    <= 1'b0;
            arp_wait_time         <= arp_wait_time + 1'b1;
            STATE                 <= WAIT_ARP_REPLY;
        end
    end
end

WAIT_ARP_PACKET:begin//ARP 包有效, 打拍后直接输出给 MAC 层
    if(I_arp_tvalid) begin
        O_mac_tdata_type        <= {1'b1,I_arp_tdata_type};//2'b10:arp reply;
        2'b11:arp request ;2'b01 ip
        O_mac_tvalid            <= 1'b1;
        O_mac_tdata              <= I_arp_tdata;
        O_mac_tdest_addr        <= I_arp_tdest_mac_addr;
        STATE                   <= SEND_ARP_PACKET;
    end
    else begin
        O_mac_tdata_type        <= 2'd0;
        O_mac_tvalid            <= 1'b0;
        O_mac_tdata              <= 8'd0;
        O_mac_tdest_addr        <= 48'd0;
        STATE                   <= WAIT_ARP_PACKET;
    end
end

SEND_ARP_PACKET:begin //继续打拍后输出给 MAC 层
    if(I_arp_tvalid) begin //如果 ARP 包有效
        O_mac_tvalid            <= 1'b1;
        O_mac_tdata              <= I_arp_tdata;
    end
end

```

```

STATE                                     <=  SEND_ARP_PACKET;

end
else begin
    O_arp_tbusy                           <=  1'b0;
    O_mac_tdata_type                      <=  2'd0;
    O_mac_tvalid                          <=  1'b0;
    O_mac_tdata                           <=  8'd0;
    O_mac_tdest_addr                      <=  48'd0;
    if(arp_req_pend)                      //如果该信号有效，代表 IP 层发送 IP 包的时候没有从本地
cache 查询到 MAC 地址，而发送的 ARP 请求包，因此下一步等待远程主机发送 ARP 响应
        STATE                             <=  WAIT_ARP_REPLY;
    else
        STATE                             <=  IDLE;    //如果是单纯的 ARP 层发送的
包，到此结束
    end
end
WAIT_IP_PACKET:begin                    //IP 包的传输
    if(I_ip_tvalid) begin
        O_mac_tdata_type                  <=  2'b01;
        O_mac_tvalid                      <=  1'b1;
        O_mac_tdata                       <=  I_ip_tdata;
        O_mac_tdest_addr                  <=  tmac_addr_temp;
        STATE                             <=  SEND_IP_PACKET;
    end
    else begin
        O_mac_tdata_type                  <=  2'd0;
        O_mac_tvalid                      <=  1'b0;
        O_mac_tdata                       <=  8'd0;
        O_mac_tdest_addr                  <=  48'd0;
        STATE                             <=  WAIT_IP_PACKET;
    end
end
SEND_IP_PACKET:begin                    //IP 包的传输
    if(I_ip_tvalid) begin
        O_mac_tvalid                      <=  1'b1;
        O_mac_tdata                       <=  I_ip_tdata;
        STATE                             <=  SEND_IP_PACKET;
    end
    else begin
        O_ip_tbusy                        <=  1'b0;
        O_mac_tdata_type                  <=  2'd0;
        O_mac_tvalid                      <=  1'b0;
        O_mac_tdata                       <=  8'd0;
        O_mac_tdest_addr                  <=  48'd0;
        STATE                             <=  IDLE;
    end
end
end
end

```

```

        endcase
    end
end

```

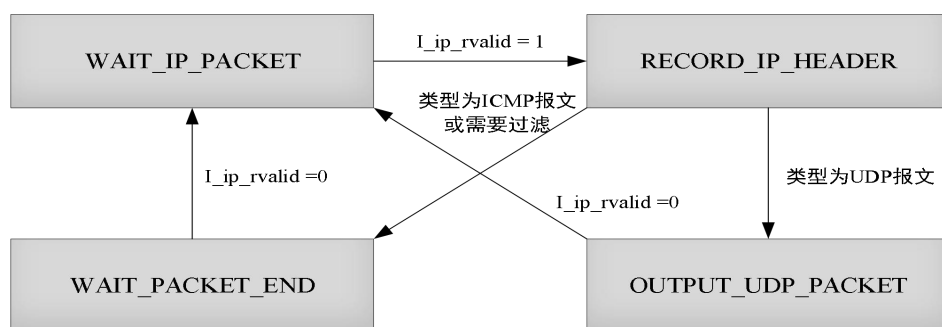
3.3 IP 层

ICMP 层数据和 UDP 层数据都要经过 IP 层打包或者解包，IP 层主要功能为判断数据报文类型，进行 IP 首部校验，添加包头或者过滤包头，处理 ICMP 请求。

3.3.1 IP 接收模块

该模块的主要功能是接收 uiip_arp_rx 传入的数据包，通过首部校验判断包头的正确性，对包头进行过滤，并且提取出 UDP 报文和 ICMP 报文。

该模块的状态机转换图如图所示。



WAIT_IP_PACKET: 等待接收 IP 包，若 I_ip_rvalid 拉高，代表数据传入，进入 RECORD_IP_HEADER 状态。

RECORD_IP_HEADER: 接收包头信息，20 字节的包头数据全部接收完时，判断收到的 IP 地址和本地 IP 地址是否匹配，若不匹配，丢弃该帧，进入 WAIT_PACKET_END 状态；若匹配，则判断数据包类型。如果收到的数据报文为 UDP 数据报文，进入 OUTPUT_UDP_PACKET 状态。如果收到的数据报文为 ICMP 报文，则将数据传入 ICMP 子层模块，等待一包数据传输完成后，进入 WAIT_PACKET_END 状态。

OUTPUT_UDP_PACKET: 将有效数据打包后传输至 UDP 层，一包数据传输完成后，回到 WAIT_IP_PACKET 状态。

WAIT_PACKET_END: 等待数据有效信号拉低，一帧数据传输完成，回到 WAIT_IP_PACKET 状态。

```

always@(posedge I_ip_clk or posedge I_reset) begin
    if(I_reset) begin
        cnt
        <= 5'd0;
    end
end

```

```

        ip_version                <= 4'd0;          //IP 首部-版本:4 位数据表示 IP 版本号, 为 4 时表示
IPv4, 为 6 时表示 IPv6, IPv4 使用较多。
        ip_header_len             <= 4'd0;          //IP 首部-首部长度:4 位数据表示 IP 首部一共有多少个
32 位 (4 个字节) 数据。没有可选字段的 IP 首部长度为 20 个字节, 故首部长度为 5
        ip_tos                    <= 8'd0;          //IP 首部-服务类型:8 位服务类型被划分成两个子字
段: 3 位优先级字段和 4 位 TOS 字段, 最后一位固定为 0。服务类型为 0 时表示一般服务。
        ip_pkg_len                <= 16'd0;         //IP 首部-总长度:16 位 IP 数据报总长度包括 IP 首部和
IP 数据部分, 以字节为单位。利用 IP 首部长度和 IP 数据报总长度可以计算出 IP 数据报中数据内容的起始位置和长度
        ip_packet_id              <= 16'd0;         //IP 首部-ID:16 位标识字段, 用来标识主机发送的每一
份数据报。每发送一份报文它的值就会加 1
        ip_packet_flag            <= 3'd0;          //IP 首部-标志字段:3 位标志字段的第 1 位是保留位, 第
2 位表示禁止分片 (1 表示不分片, 0 允许分片), 第 3 位标识更多分片, 通常为 010 不分片
        ip_fragment_offset        <= 13'd0;         //IP 首部-片偏移:13 位片偏移, 在接收方进行数据报重组时
用来标识分片的顺序。
        ip_packet_ttl             <= 8'd0;          //IP 首部-生存时间:      8 位生存时间防止丢失的数据
包在无休止的传播, 一般被设置为 64 或者 128
        ip_packet_protocol        <= 8'd0;          //IP 首部-协议:8 位协议类型表示此数据报所携带上层数据
使用的协议类型, ICMP 为 1, TCP 为 6, UDP 为 17
        ip_header_checksum        <= 16'd0;         //IP 首部-首部校验和:16 位首部校验和, 该字段只校验数据
报的首部, 不包含数据部分
        ip_src_address            <= 32'd0;         //IP 首部-源 IP 地址:32 位发送端的 IP 地址
        ip_dst_address            <= 32'd0;         //IP 首部-目的 IP 地址:32 位接收端的 IP 地址
        icmp_pkg_valid            <= 1'b0;          //icmp 数据报文有效信号
        0_udp_ip_rvalid           <= 1'b0;          //UDP 数据包有效
        0_udp_ip_rdata            <= 8'd0;          //UDP 数据包有效输出
        STATE                     <= WAIT_IP_PACKET;

    end
    else begin
        case(STATE)
            WAIT_IP_PACKET:begin
                if(I_ip_rvalid) begin
                    ip_version                <=  I_ip_rdata[7:4];          //IP 首部-版本:4 位数据
表示 IP 版本号
                    ip_header_len             <=  I_ip_rdata[3:0];          //IP 首部-首部长度:4 位数
据表示 IP 首部一共有多少个 32 位 (4 个字节) 数据
                    STATE                     <=  RECORD_IP_HEADER;        //下一状态,继续接收 IP
头部信息

                    end
                    else
                        STATE                     <=  WAIT_IP_PACKET;

                    end
                RECORD_IP_HEADER:begin
                    case(cnt)
                        0:  begin          ip_tos                    <=  I_ip_rdata; cnt
<=  cnt + 1'b1;end //IP 首部-服务类型:8 位服务类型被划分成两个子字段: 3 位优先级字段和 4 位 TOS 字段, 最后
一位固定为 0。服务类型为 0 时表示一般服务。
                        1:  begin          ip_pkg_len[15:8]            <=  I_ip_rdata; cnt
<=  cnt + 1'b1;end //IP 首部-总长度:16 位 IP 数据报总长度包括 IP 首部和 IP 数据部分, 以字节为单位。

```

```

2:   begin   ip_pkg_len[7:0]               <=   I_ip_rdata; cnt
<=   cnt + 1'b1;end   //IP 首部-总长度:16 位 IP 数据报总长度包括 IP 首部和 IP 数据部分, 以字节为单位。
3:   begin   ip_packet_id[15:8]            <=   I_ip_rdata; cnt
<=   cnt + 1'b1;end   //IP 首部-ID:16 位标识字段, 用来标识主机发送的每一份数据报。
4:   begin   ip_packet_id[7:0]             <=   I_ip_rdata; cnt
<=   cnt + 1'b1;end   //IP 首部-ID:16 位标识字段, 用来标识主机发送的每一份数据报。
5:   begin
      ip_packet_flag                       <=   I_ip_rdata[7:5];
      //IP 首部-标志字段:3 位标志字段的第 1 位是保留位, 第 2 位表示禁止分片 (1 表示不分片, 0 允许分片), 第 3
      位标识更多分片, 通常为 010 不分片
      ip_fragment_offset[12:8]             <=   I_ip_rdata[4:0];
      //IP 首部-片偏移:13 位片偏移, 在接收方进行数据报重组时用来标识分片的顺序。
      cnt <=   cnt + 1'b1;
   end
6:   begin   ip_fragment_offset[7:0]        <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-片偏移:13 位片偏移, 在接收方进行数据报重组时用来标识分片的顺序。
7:   begin   ip_packet_ttl[7:0]             <=   I_ip_rdata; cnt
<=   cnt + 1'b1;end   //IP 首部-生存时间:8 位生存时间防止丢失的数据包在无休止的传播, 一般被设置为 64 或者 128
8:   begin   ip_packet_protocol[7:0]        <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-协议:8 位协议类型表示此数据报所携带上层数据使用的协议类型, ICMP 为 1, TCP 为 6, UDP 为
17
9:   begin   ip_header_checksum[15:8]       <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-首部校验和:16 位首部校验和, 该字段只校验数据报的首部, 不包含数据部分
10:  begin   ip_header_checksum[7:0]        <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-首部校验和:16 位首部校验和, 该字段只校验数据报的首部, 不包含数据部分
11:  begin   ip_src_address[31:24]          <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-源 IP 地址:32 位发送端的 IP 地址
12:  begin   ip_src_address[23:16]          <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-源 IP 地址:32 位发送端的 IP 地址
13:  begin   ip_src_address[15:8]           <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-源 IP 地址:32 位发送端的 IP 地址
14:  begin   ip_src_address[7:0]            <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-源 IP 地址:32 位发送端的 IP 地址
15:  begin   ip_dst_address[31:24]          <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-目的 IP 地址:32 位接收端的 IP 地址
16:  begin   ip_dst_address[23:16]          <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-目的 IP 地址:32 位接收端的 IP 地址
17:  begin   ip_dst_address[15:8]           <=   I_ip_rdata; cnt <=   cnt +
1'b1;end   //IP 首部-目的 IP 地址:32 位接收端的 IP 地址
18:  begin
      ip_dst_address[7:0]                  <=   I_ip_rdata;
      //IP 首部-目的 IP 地址:32 位接收端的 IP 地址
      cnt                                  <=   5'd0;
      if({ip_dst_address[31:8], I_ip_rdata} == I_ip_local_addr)
begin   //如果收到的 IP 地址和本地 IP 地址匹配
      if(ip_packet_protocol == ICMP_TYPE)
begin   //如果是 ICMP 类型
      icmp_pkg_valid                       <=   1'b1;

```

```

//ICMP 包有效
STATE                                     <=  WAIT_PACKET_END;

//等待传输结束状态
end
else if(ip_packet_protocol == UDP_TYPE)
begin
//如果是 UDP 包有效
icmp_pkg_valid                           <=  1'b0;
STATE                                    <=  OUTPUT_UDP_PACKET;

//输出 UDP 包到 UDP 协议层
end
else begin
icmp_pkg_valid                           <=  1'b0;
STATE                                    <=  WAIT_PACKET_END;

//等待传输结束状态
end
end
else begin
icmp_pkg_valid                           <=  1'b0;
STATE                                    <=  WAIT_PACKET_END;

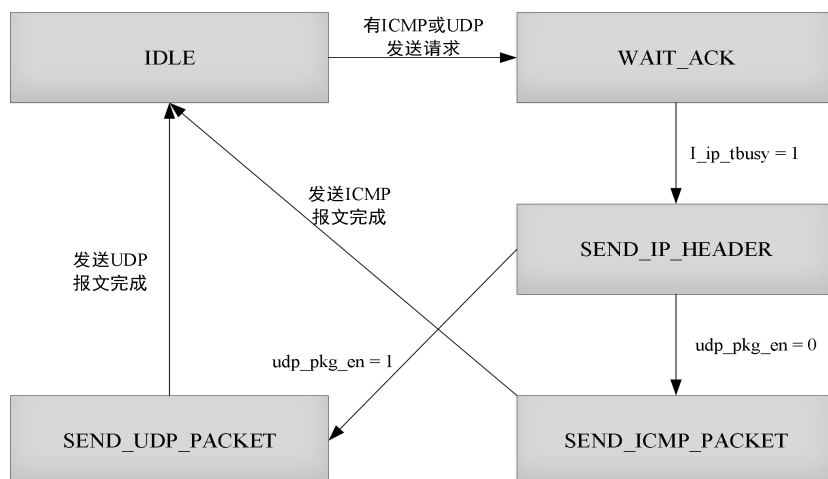
//等待传输结束状态
end
end
endcase
end
OUTPUT_UDP_PACKET:begin
if(I_ip_rvalid) begin
0_udp_ip_rvalid                         <=  1'b1;
0_udp_ip_rdata                         <=  I_ip_rdata;
STATE                                  <=  OUTPUT_UDP_PACKET;
end
else begin
0_udp_ip_rvalid                         <=  1'b0;
0_udp_ip_rdata                         <=  8'd0;
STATE                                  <=  WAIT_IP_PACKET;
end
end
end
WAIT_PACKET_END:begin
if(I_ip_rvalid)
STATE                                  <=  WAIT_PACKET_END;
else begin
icmp_pkg_valid                         <=  1'b0;
STATE                                  <=  WAIT_IP_PACKET;
end
end
endcase
end
end
end

```

3.3.2 IP 发送模块

该模块的主要功能有：接收 uiudp_tx 发送的 UDP 报文和 uiip_rx 发送的 ICMP 报文信息，将其封装成 IP 包并发送至下层 uiip_arp_tx 模块。

该模块的状态机转换图如图所示。



IDLE: uiip_tx 模块处于发送空闲时，接收 ICMP 和 UDP 报文发送请求，优先响应 ICMP 请求。当收到请求时，向 uiip_arp_tx 模块发送请求信号，通过 udp_pkg_en 信号标记报文类型，进入 WAIT_ACK 状态，等待握手响应信号。

WAIT_ACK: 接收到 I_ip_tbusy 信号，代表和 IP_ARP 层模块握手成功，可以发送数据，进入 SEND_IP_HEADER 状态。若要发送的是 UDP 报文，则会发送 O_ip_udp_tbusy 信号和 UDP 层进行握手。

SEND_IP_HEADER: 在有效数据报文前添加对应的 IP 包头，发送包头完毕后，若报文类型为 UDP 报文，进入 SEND_UDP_PACKET 状态；若为 ICMP 报文，则进入 SEND_ICMP_PACKET 状态。

SEND_UDP_PACKET 和 SEND_ICMP_PACKET: 发送完报文中的有效数据后，将标识加 1，回到 IDLE 状态。

```

case (STATE)
    IDLE:begin
        if (icmp_pkg_req & (~I_ip_tbusy)) begin           //如果有 ICMP 包需要发送，并
            //且 ip_arp_tx 模块处于空闲(I_ip_tbusy==0 代表 tbuf 空闲，不在发送数据)
            O_ip_treq                                     <= 1'b1;           //通知 ip_arp_tx 模
            //块，有 IP 包需要发送 (ICMP 包也是 IP 包)
            O_ip_udp_tbusy                               <= 1'b0;           //通知 udp_layer 模
            //块，目前不能发送 UDP 数据包
            O_ip_taddr                                    <= icmp_pkg_ip_addr; //保存 ip_rx 模块接收到的
            //icmp 请求的 IP 地址(因为发送 icmp 包需要通过 IP 地址获取远程主机的 MAC 地址)
            udp_pkg_en                                    <= 1'b0;           //标记是否是 udp
            //包，但 udp_pkg_en=1 代表 ip 层发送的数据为 UDP 包
        end
    end
  
```

```

STATE                                     <=  WAIT_ACK;

end
else if(I_ip_udp_treq & (~I_ip_tbusy)) begin //如果有 UDP 包需要发送
    0_ip_treq                             <=  1'b1; //输出 ip 包发送请求, 给 ip_arp_tx 模块

    0_ip_udp_tbusy                         <=  1'b0; //通知 udp_layer 模块, 目前不能发送 UDP 数据包

    0_ip_taddr                             <=  I_ip_dest_addr; //udp 层提供需要发送的目的主机的 IP 地址

    udp_pkg_en                             <=  1'b1; //标记是否是 udp 包, 但 udp_pkg_en=1 代表 ip 层发送的数据为 UDP 包

    STATE                                 <=  WAIT_ACK;

end
else begin
    0_ip_udp_tbusy                         <=  1'b0;
    0_ip_treq                             <=  1'b0;
    udp_pkg_en                             <=  1'b0;
    STATE                                 <=  IDLE;

end
end
WAIT_ACK:begin
    if(I_ip_tbusy) begin//当发送 0_ip_treq 后, 如果 ip_arp_tx 模块返回 I_ip_tbusy=1 代表 ip_layer 可以发送 IP 包(UDP 包和 ICMP 包)到 ip_arp_tx 模块

        0_ip_treq                         <=  1'b0;
        0_ip_udp_tbusy                     <=  udp_pkg_en ? 1 : 0; //如果 udp_pkg_en 有效代表发的是 UDP 包

        STATE                             <=  SEND_IP_HEADER; //发送 IP 帧头

    end
    else begin
        0_ip_treq                         <=  1'b1;
        0_ip_udp_tbusy                     <=  1'b0;
        STATE                             <=  WAIT_ACK;

    end
end
SEND_IP_HEADER:begin//发送 IP 包帧头
    case(cnt)
        0 :begin
            if(I_ip_udp_tvalid | (~udp_pkg_en)) begin //如果是 UDP 报文包需要发送或者 udp_pkg_en==0 是 ICMP 报文包

                0_ip_tdata                 <=  {VERSION, IHL}; //版本|首部长度(IP 首部一共有多少个 32bit 数据)

                0_ip_tvalid                 <=  1'b1; //通知 tbuf IP 数据有效

                packet_id                   <=  ID_BASE + datagram_cnt; //标识, 每发送 1 包该值加 1

                TTL                         <=  8'h80; //生存时间

                if(!udp_pkg_en) begin //如果是

```

```

ICMP 包
                                ip_tdata_length    <=    icmp_pkg_data_len + (IHL << 2);
//IP 包总长度 (IP 数据长度+IP 首部长度)
                                PROTOCOL            <=    8'h01;
                                //IP 包类型为 ICMP 包
                                end
                                else begin
                                ip_tdata_length    <=    I_ip_udp_tdata_len + (IHL <<
2);
                                PROTOCOL            <=    8'h11;
                                //IP 包类型为 UDP 包
                                end
                                cnt                <=    cnt + 1'b1;
                                end
                                else
                                cnt                <=    5'd0;
                                end
                                1 :begin    0_ip_tdata    <=    TOS;
                                cnt <=    cnt + 1'b1;end//服务类型
                                2 :begin    0_ip_tdata    <=    ip_tdata_length[15:8];                cnt
<=    cnt + 1'b1;end//IP 包总长度
                                3 :begin    0_ip_tdata    <=    ip_tdata_length[7:0];                cnt
<=    cnt + 1'b1;end//IP 包总长度
                                4 :begin    0_ip_tdata    <=    packet_id[15:8];                cn
t <=    cnt + 1'b1;end//IP 包标识符, 每发送一份报文, 其值加 1
                                5 :begin    0_ip_tdata    <=    packet_id[7:0];                cn
t <=    cnt + 1'b1;end//IP 包标识符, 每发送一份报文, 其值加 1
                                6 :begin    0_ip_tdata    <=    {FLAG, FRAGMENT_OFFSET[12:8]};    cnt
<=    cnt + 1'b1;end//标志字段 3bit|片偏移共 13bit
                                7 :begin    0_ip_tdata    <=    FRAGMENT_OFFSET[7:0];                cnt
<=    cnt + 1'b1;end//片偏移共 13bit
                                8 :begin    0_ip_tdata    <=    TTL;
                                cnt <=    cnt + 1'b1;end//生存时间
                                9 :begin    0_ip_tdata    <=    PROTOCOL;
                                cnt <=    cnt + 1'b1;end//协议
                                10 :begin    0_ip_tdata    <=    checksum[15:8];                cn
t <=    cnt + 1'b1;end//校验和
                                11 :begin    0_ip_tdata    <=    checksum[7:0];                c
nt <=    cnt + 1'b1;end//校验和
                                12 :begin    0_ip_tdata    <=    I_ip_local_addr[31:24];                cnt
<=    cnt + 1'b1;end//源 IP 地址 32bit
                                13 :begin    0_ip_tdata    <=    I_ip_local_addr[23:16];                cnt
<=    cnt + 1'b1;end//源 IP 地址 32bit
                                14 :begin    0_ip_tdata    <=    I_ip_local_addr[15:8];                cnt
<=    cnt + 1'b1;end//源 IP 地址 32bit
                                15 :begin    0_ip_tdata    <=    I_ip_local_addr[7:0];                cnt
<=    cnt + 1'b1;end//源 IP 地址 32bit
                                16 :begin
                                //目的 IP 地址(远端主机 IP

```

地址)

```

        if(!udp_pkg_en)                                //ICMP 报文包
            0_ip_tdata      <=  icmp_pkg_ip_addr[31:24];
        else                                              //UDP 报文包
            0_ip_tdata      <=  0_ip_taddr[31:24];
        cnt      <=  cnt + 1'b1;
    end
    17 :begin                                           //目的 IP 地址(远端主机 IP

```

地址)

```

        if(!udp_pkg_en)                                //ICMP 报文包
            0_ip_tdata      <=  icmp_pkg_ip_addr[23:16];
        else                                              //UDP 报文包
            0_ip_tdata      <=  0_ip_taddr[23:16];
        cnt      <=  cnt + 1'b1;
    end
    18 :begin                                           //目的 IP 地址(远端主机 IP

```

地址)

SEND_ICMP_PACKET 状态输出 ICMP 报文包

```

        if(!udp_pkg_en) begin
            0_ip_tdata      <=  icmp_pkg_ip_addr[15:8];
            icmp_pkg_busy   <=  1'b1; //icmp_pkg_tx 代码上必须时序上, 在
        end
        else begin
            0_ip_tdata      <=  0_ip_taddr[15:8];
            icmp_pkg_busy   <=  1'b0;
        end
        cnt      <=  cnt + 1'b1;
    end
    19 :begin                                           //目的 IP 地址(远端主机 IP

```

地址)

```

        cnt <=  5'd0;
        if(!udp_pkg_en) begin
            0_ip_tdata      <=  icmp_pkg_ip_addr[7:0];
            STATE           <=  SEND_ICMP_PACKET;
        end
        else begin
            0_ip_tdata      <=  0_ip_taddr[7:0];
            STATE           <=  SEND_UDP_PACKET;
        end
    end
    default:      cnt <=  5'd0;
endcase
end
SEND_UDP_PACKET:begin                                //发送 UDP 报文包
    if(trans_data_cnt == (ip_tdata_length - 16'd20)) begin //20 个字节为 IP 首部, 这
里相等代表数据发送结束
        0_ip_udp_tbusy     <=  1'b0;
        0_ip_tvalid        <=  1'b0;
    end
end

```

```

                                O_ip_tdata          <= 8'd0;
                                datagram_cnt          <= datagram_cnt + 16'h0001; //没发送完 1 帧报
文，该值加 1
                                trans_data_cnt        <= 16'd0;
                                STATE                  <= IDLE;
                                end
                                else begin //发送有效的 UDP 报文数据部分
                                    O_ip_tvalid         <= 1'b1;
                                    O_ip_tdata          <= shift_data_out; //从 shift 移位寄
存器移出数据
                                    trans_data_cnt      <= trans_data_cnt + 1'b1; //UDP 报文包有效数据部
分计数器
                                    STATE              <= SEND_UDP_PACKET;
                                end
                                end
                                SEND_ICMP_PACKET:begin //发送 ICMP 报文包
                                    if(icmp_pkg_valid) begin
                                        O_ip_tvalid      <= 1'b1;
                                        O_ip_tdata        <= icmp_pkg_data;
                                        STATE            <= SEND_ICMP_PACKET;
                                    end
                                    else begin
                                        O_ip_tvalid      <= 1'b0;
                                        O_ip_tdata        <= 8'd0;
                                        icmp_pkg_busy     <= 1'b0;
                                        datagram_cnt      <= datagram_cnt + 16'h0001; //每发送完 1 帧报文，该
值加 1
                                        STATE            <= IDLE;
                                    end
                                end
                                end
                                endcase

```

3.3.3 IP 首部校验模块

正常计算 IP 首部校验和时，将校验位置 0。由于每两字节数据相加得到的不再产生进位结果，为最终校验和的取反，所以当计算过程中加入校验位时，相加得到的结果应当为全 1。若结果不为全 1，则数据校验错误，将 O_check_error 信号置为高。

```

`timescale 1ns / 1ps

module ip_header_checksum(
input  wire      I_clk,
input  wire      I_reset,
input  wire      I_ip_rdata_valid,
input  wire [7:0] I_ip_rdata,
output wire      O_checksum_error

```

```

);
reg checksum_correct;
assign 0_checksum_error = ~checksum_correct;

reg [1:0] state;
reg [3:0] cnt;
wire [16:0] tmp_accum1;
reg [15:0] accum1, accum2;

assign tmp_accum1 = accum1 + accum2;

always @(posedge I_clk or posedge I_reset) begin
    if(I_reset) begin
        state                <= 2'd0;
        cnt                  <= 4'd0;
        accum1               <= 16'd0;
        accum2               <= 16'd0;
        checksum_correct     <= 1'b1;
    end
    else begin
        case(state)
            0: begin
                if(I_ip_rdata_valid) begin
                    accum1[15:8] <= I_ip_rdata;
                    state        <= 2'd1;
                end
                else begin
                    accum1[15:8] <= 8'd0;
                    state        <= 2'd0;
                end
            end
            1: begin accum1[7:0] <= I_ip_rdata; state <= 2'd2; end
            2: begin
                if(cnt == 4'd9) begin
                    if((tmp_accum1[15:0] + tmp_accum1[16]) != 16'hffff)
                        checksum_correct <= 1'b0;
                    cnt <= 4'd0;
                    state <= 2'd3;
                end
                else begin
                    accum2 <= tmp_accum1[15:0] + tmp_accum1[16];
                    accum1[15:8] <= I_ip_rdata;
                    cnt <= cnt +
1'b1;

                    state <= 2'd1;
                end
            end
            3: begin

```

```

        accum1                <= 16'd0;
        accum2                <= 16'd0;
        if(I_ip_rdata_valid)
            state              <= state;
        else
            state              <= 2'd0;
        end
    endcase
end
end
endmodule

```

3.4 ARP 层

该层具有接收 ARP 请求、发送 ARP 回复，和发送 ARP 请求、接收 ARP 回复的功能，并将接收到的对端的地址信息存入 cache 中。

3.4.1 ARP 接收模块

该模块通过计数器接收 28 字节 ARP 信息字段。信息接收完成后，通过接收到的操作码判断接收到的数据包是 ARP 请求包还是 ARP 应答包。如果是应答包，将接收到的地址信息写入 cache 中；如果是请求包，判断目的 IP 地址与本地 IP 地址是否一致，若一致，将地址信息写入 cache 中，且发送至 uiarp_rx 模块，若不一致，则过滤该包。

```

if(OPER == ARP_REQUEST) begin//如果是 ARP 请求 ARP_REQUEST = 16'h0001 (16'h01 ARP 请求包 ; 16'h02 ARP 应
答)
    if(TPA == I_ip_local_addr) begin //比较接收到的 ARP 包里面的 IP 地
址是否和本地 IP 地址一致
        O_arp_req_ip_addr    <= SPA; //发送 IP(远端源 IP 地址)
        O_arp_req_mac_addr   <= SHA; //发送方 MAC(远端源地址 MAC)
        O_arp_req_valid      <= 1'b1; //设置 ARP 请求有效(通知发送
ARP 发送模块发送一个 ARP 应答给远端主机),保存远端主机的 IP 地址和 MAC 地址到 cache
        O_arp_reply_done     <= 1'b0;
    end
    else begin
        O_arp_req_ip_addr    <= 32'd0;
        O_arp_req_mac_addr   <= 48'd0;
        O_arp_req_valid      <= 1'b0;
        O_arp_reply_done     <= 1'b0;
    end
end
end

```

3.4.2 ARP 发送模块

该模块对 ARP 应答和 ARP 请求同时进行处理，首先对这两种逻辑进行仲裁，以避免发送冲突。如果既没有 ARP 应答也没有 ARP 请求，且内部 ARP 包发送逻辑非忙，则处理缓存的有效信号；如果只有 ARP 应答，没有 ARP 请求，且发送逻辑非忙，则请求发送 ARP 应答包，若发送逻辑忙，则将发送 ARP 应答包的请求缓存至 reply_buffer_valid 中，等待非忙时发出；如果只有 ARP 请求，没有 ARP 应答，且发送逻辑非忙，则请求发送 ARP 请求包，若发送逻辑忙，则将发送 ARP 请求包的请求缓存至 request_buffer_valid 中，等待非忙时发出；若 ARP 应答和 ARP 请求同时为高，则优先发送 ARP 请求，缓存 ARP 应答。

```

case({I_arp_treq_en, I_arp_rreply_en})//本状态机实现了即便是同时有 ARP 应答或者 ARP 请求，都能确保完成
发送
    2'b00:begin
        if((!O_arp_treq) && (!O_arp_tvalid)) begin//没有 arp_treq 请求，并且 arp_tvalid 为
0 代表没有要发送的 ARP 数据
            if(request_buffer_valid) begin//如果有未发送完的 ARP 请求，则继续发送
                OPER                <=  ARP_REQUEST;
                TPA                  <=  ip_request_buffer;
                THA                   <=  48'd0;
                request_buffer_valid <=  1'b0;//清除 request_buffer_valid
                O_arp_treq           <=  1'b1;
            end
            else if(reply_buffer_valid) begin//如果有未发送完的 ARP 应答，则继续发送
                OPER                <=  ARP_REPLY;
                TPA                  <=  ip_reply_buffer;
                THA                   <=  mac_reply_buffer;
                reply_buffer_valid <=  1'b0;//清除 request_buffer_valid
                O_arp_treq           <=  1'b1;
            end
        end
    end
    2'b01:begin//发送 ARP 应答
        if((!O_arp_treq) && (!O_arp_tvalid)) begin
            OPER                <=  ARP_REPLY;
            TPA                  <=  I_arp_rreply_ip_addr;
            THA                   <=  I_arp_rreply_mac_addr;
            O_arp_treq           <=  1'b1;
        end
        else begin//需要 arp 应答
            ip_reply_buffer      <=  I_arp_rreply_ip_addr;//寄存目的地址 IP
            mac_reply_buffer     <=  I_arp_rreply_mac_addr;//寄存目的地址

```

MAC

```

        reply_buffer_valid      <=  1'b1; //需要发送 ARP 应答
    end
end
2'b10:begin //发送 ARP 请求, 当 ip_arp_tx 发送 IP 包查询 MAC 没有查询到, 执行 ARP 请求, 请求远
程主机提供 MAC

    if((!O_arp_treq) && (!O_arp_tvalid)) begin
        OPER                      <=  ARP_REQUEST;
        TPA                      <=  I_arp_tip_addr;
        THA                      <=  48'd0;
        O_arp_treq                <=  1'b1; //ARP 发送
    end
    else begin //arp 请求包
        ip_request_buffer         <=  I_arp_tip_addr;
        request_buffer_valid      <=  1'b1; //ARP 请求有效标志
    end
end
2'b11:begin //既有 ARP 请求, 又有 ARP 应答
    if((!O_arp_treq) && (!O_arp_tvalid)) begin
        OPER                      <=  ARP_REQUEST;
        TPA                      <=  I_arp_tip_addr;
        THA                      <=  48'd0;
        O_arp_treq                <=  1'b1; //ARP 发送
    end
    else begin
        ip_request_buffer         <=  I_arp_tip_addr;
        request_buffer_valid      <=  1'b1; //ARP 请求有效
    end
end
    ip_reply_buffer              <=  I_arp_rreply_ip_addr;
    mac_reply_buffer              <=  I_arp_rreply_mac_addr;
    reply_buffer_valid            <=  1'b1; //ARP 应答有效
end
endcase

```

若 O_arp_treq 和 uiip_arp_tx 模块发送的 I_arp_tbusy 完成握手, 则开始发送 ARP 包, 并且通过判断 ARP 包的类型来确定发送的目的地址。通过计数器 cnt 发送 28 字节有效数据, 再通过计数器 pad_cnt 在末尾补 18 个字节的 0, 即完成发送, 开始等待下一次握手。

```

case (STATE)
    WAIT_BUFFER_READY:begin
        if(O_arp_treq && I_arp_tbusy) begin
            O_arp_tdata          <=  HTYPE[15:8]; //硬件类型-以太网类型
            O_arp_tvalid          <=  1'b1; //ARP 数据有效
            cnt                   <=  cnt + 1'b1;
            if(OPER == ARP_REQUEST) begin //如果是 ARP 请求
                O_arp_tdest_mac_addr <=  48'hff_ff_ff_ff_ff_ff; //ARP 目的地
                O_arp_ttype         <=  1'b1;
            end
        end
    end
endcase

```

```

//通知 ip_arp_tx ARP 类型为 ARP 请求
    end
    else begin
        O_arp_tdest_mac_addr    <=  THA;
        O_arp_ttype             <=  1'b0;           //通知 ip_arp_tx
        ARP 类型为 ARP 应答

    end
    O_arp_treq                  <=  1'b0;
    STATE                       <=  SEND_ARP_PACKET;

    end
    else
        STATE                   <=  WAIT_BUFFER_READY;

    end
    SEND_ARP_PACKET:begin
        case(cnt)
            1:  begin    O_arp_tdata <=  HTYPE[7:0];                cnt <=
cnt + 1'b1;end
            2:  begin    O_arp_tdata <=  PTYPE[15:8];                cnt <=
cnt + 1'b1;end
            3:  begin    O_arp_tdata <=  PTYPE[7:0];                cnt <=
cnt + 1'b1;end
            4:  begin    O_arp_tdata <=  HLEN;                        cnt
<= cnt + 1'b1;end
            5:  begin    O_arp_tdata <=  PLEN;                        cnt
<= cnt + 1'b1;end
            6:  begin    O_arp_tdata <=  OPER[15:8];                cnt <=
cnt + 1'b1;end
            7:  begin    O_arp_tdata <=  OPER[7:0];                cnt <=
cnt + 1'b1;end
            8:  begin    O_arp_tdata <=  I_mac_local_addr[47:40];    cnt <= cnt +
1'b1;end
            9:  begin    O_arp_tdata <=  I_mac_local_addr[39:32];    cnt <= cnt +
1'b1;end
            10: begin    O_arp_tdata <=  I_mac_local_addr[31:24];    cnt <= cnt +
1'b1;end
            11: begin    O_arp_tdata <=  I_mac_local_addr[23:16];    cnt <= cnt +
1'b1;end
            12: begin    O_arp_tdata <=  I_mac_local_addr[15:8];    cnt <= cnt +
1'b1;end
            13: begin    O_arp_tdata <=  I_mac_local_addr[7:0];      cnt <= cnt +
1'b1;end
            14: begin    O_arp_tdata <=  I_ip_local_addr[31:24];    cnt <= cnt +
1'b1;end
            15: begin    O_arp_tdata <=  I_ip_local_addr[23:16];    cnt <= cnt +
1'b1;end
            16: begin    O_arp_tdata <=  I_ip_local_addr[15:8];    cnt <= cnt +
1'b1;end
            17: begin    O_arp_tdata <=  I_ip_local_addr[7:0];      cnt <= cnt +

```

```

1'b1;end

18: begin    0_arp_tdata <=  THA[47:40];                cnt <= cnt
+ 1'b1;end

19: begin    0_arp_tdata <=  THA[39:32];                cnt <= cnt
+ 1'b1;end

20: begin    0_arp_tdata <=  THA[31:24];                cnt <= cnt
+ 1'b1;end

21: begin    0_arp_tdata <=  THA[23:16];                cnt <= cnt
+ 1'b1;end

22: begin    0_arp_tdata <=  THA[15:8];                cnt <=
cnt + 1'b1;end

23: begin    0_arp_tdata <=  THA[7:0];                cnt <=
cnt + 1'b1;end

24: begin    0_arp_tdata <=  TPA[31:24];                cnt <= cnt
+ 1'b1;end

25: begin    0_arp_tdata <=  TPA[23:16];                cnt <= cnt
+ 1'b1;end

26: begin    0_arp_tdata <=  TPA[15:8];                cnt <=
cnt + 1'b1;end

27: begin    0_arp_tdata <=  TPA[7:0];                cnt <=
cnt + 1'b1;end

28: begin
    0_arp_tdata <=  8'd0;
    if(pad_cnt == 5'd17) begin //通过在末尾添加 0 以确保数据长度为 46
        cnt      <=  cnt + 1'b1;
        pad_cnt <=  5'd0;
    end
    else begin
        cnt      <=  cnt;
        pad_cnt <=  pad_cnt + 1'b1;
    end
end
29: begin
    0_arp_tdata      <=  8'd0;
    0_arp_tvalid      <=  1'b0;
    0_arp_tdest_mac_addr <=  48'd0;
    0_arp_ttype       <=  1'b0;
    cnt              <=  5'd0;
    STATE            <=  WAIT_BUFFER_READY;
end
default:begin
    0_arp_tdata      <=  8'd0;
    0_arp_tvalid      <=  1'b0;
    cnt              <=  5'd0;
    STATE            <=  WAIT_BUFFER_READY;
end
endcase
end

```

```

        endcase
    end
end

```

3.4.3 CACHE 模块

mac_cache 模块相当于一个 RAM，通过 IP 作为地址，写入或读出 MAC 地址。写入 mac_cache 模块的地址信息远程主机发送的 ARP 应答包或 ARP 请求包中的地址，在 IP 层发送 UDP 数据报文时，需要在 mac_cache 中读出和 IP 地址对应的 MAC 地址。

```

`timescale 1ns/1ps
module mac_cache
(
    input  wire      I_wclk,
    input  wire      I_reset,
    input  wire      I_wen,
    input  wire [31:0] I_wip_addr,
    input  wire [47:0] I_wmac_addr,

    input  wire      I_rclk,
    input  wire      I_ren,
    input  wire [31:0] I_rip_addr,
    output reg [47:0] O_rmac_addr,
    output reg      O_rmac_done
);
reg mac_cache_flag [0:3];
reg [31:0] ip_address_cache [0:3];
reg [47:0] mac_address_cache [0:3];
reg [1:0] index;

always@(posedge I_wclk or posedge I_reset) begin
    if(I_reset) begin
        mac_cache_flag[0]    <= 1'b0;
        ip_address_cache[0]  <= 32'd0;
        mac_address_cache[0] <= 48'd0;
        mac_cache_flag[1]    <= 1'b0;
        ip_address_cache[1]  <= 32'd0;
        mac_address_cache[1] <= 48'd0;
        mac_cache_flag[2]    <= 1'b0;
        ip_address_cache[2]  <= 32'd0;
        mac_address_cache[2] <= 48'd0;
        mac_cache_flag[3]    <= 1'b0;
        ip_address_cache[3]  <= 32'd0;
        mac_address_cache[3] <= 48'd0;
        index                <= 2'd0;
    end
end

```

```

else begin
    if(I_wen) begin
        if(mac_cache_flag[0] && ip_address_cache[0] == I_wip_addr)
            mac_address_cache[0] <= I_wmac_addr;
        else if(mac_cache_flag[1] && ip_address_cache[1] == I_wip_addr)
            mac_address_cache[1] <= I_wmac_addr;
        else if(mac_cache_flag[2] && ip_address_cache[2] == I_wip_addr)
            mac_address_cache[2] <= I_wmac_addr;
        else if(mac_cache_flag[3] && ip_address_cache[3] == I_wip_addr)
            mac_address_cache[3] <= I_wmac_addr;
        else begin
            mac_cache_flag[index] <= 1'b1;
            ip_address_cache[index] <= I_wip_addr;
            mac_address_cache[index] <= I_wmac_addr;
            index <= index + 1'b1;
        end
    end
end
else begin
    mac_cache_flag[0] <= mac_cache_flag[0];
    ip_address_cache[0] <= ip_address_cache[0];
    mac_address_cache[0] <= mac_address_cache[0];
    mac_cache_flag[1] <= mac_cache_flag[1];
    ip_address_cache[1] <= ip_address_cache[1];
    mac_address_cache[1] <= mac_address_cache[1];
    mac_cache_flag[2] <= mac_cache_flag[2];
    ip_address_cache[2] <= ip_address_cache[2];
    mac_address_cache[2] <= mac_address_cache[2];
    mac_cache_flag[3] <= mac_cache_flag[3];
    ip_address_cache[3] <= ip_address_cache[3];
    mac_address_cache[3] <= mac_address_cache[3];
end
end
end

always@(posedge I_rclk or posedge I_reset) begin
    if(I_reset) begin
        O_rmac_addr <= 48'd0;
        O_rmac_done <= 1'b0;
    end
    else begin
        if(I_ren) begin
            O_rmac_done <= 1'b1;
            if(mac_cache_flag[0] && I_rip_addr == ip_address_cache[0])
                O_rmac_addr <= mac_address_cache[0];
            else if(mac_cache_flag[1] && I_rip_addr == ip_address_cache[1])
                O_rmac_addr <= mac_address_cache[1];
            else if(mac_cache_flag[2] && I_rip_addr == ip_address_cache[2])
                O_rmac_addr <= mac_address_cache[2];
        end
    end
end

```

```

        else if(mac_cache_flag[3] && I_rip_addr == ip_address_cache[3])
            O_rmac_addr <= mac_address_cache[3];
        else
            O_rmac_addr <= 48'd0;
        end
    else begin
        O_rmac_addr <= O_rmac_addr;
        O_rmac_done <= 1'b0;
    end
end
end
endmodule

```

3.5 UDP 层

该层实现用户数据和 UDP 报文的互转，相比于其它层次的设计，该层的逻辑相对简单。

3.5.1 UDP 接收模块

通过计数器将 UDP 报文头部信息拆解，提取出端口号和长度信息。通过提取的长度对有效数据进行计数，从而得到有效数据并发送给用户端。

```

always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset) begin
        cnt <= 4'd0;
        O_R_udp_valid <= 1'b0;
        O_R_udp_data <= 8'd0;
        O_R_udp_len <= 16'd0;
        udp_src_port <= 16'd0;
        udp_dest_port <= 16'd0;
        udp_pkg_len <= 16'd0;
        udp_data_cnt <= 16'd0;
    end
    else if(I_udp_ip_rvalid) begin
        udp_data_cnt <= udp_data_cnt + 1'b1;
        case(cnt)
            0 :begin udp_src_port[15:8] <= I_udp_ip_rdata; cnt <= cnt + 1'b1;end //UDP
接收源端口(远程主机端口)
            1 :begin udp_src_port[7:0] <= I_udp_ip_rdata; cnt <= cnt + 1'b1;end //UDP
接收源端口(远程主机端口)
            2 :begin udp_dest_port[15:8] <= I_udp_ip_rdata; cnt <= cnt + 1'b1;end //UDP
接收目的端口(本地主机端口)
            3 :begin udp_dest_port[7:0] <= I_udp_ip_rdata; cnt <= cnt + 1'b1;end //UDP
接收目的端口(本地主机端口)
            4 :begin udp_pkg_len[15:8] <= I_udp_ip_rdata; cnt <= cnt + 1'b1;end //UDP

```

数据包长度

```
5 :begin udp_pkg_len[7:0] <= I_udp_ip_rdata; cnt <= cnt + 1'b1;end //UDP
```

数据包长度

```
6 :begin cnt <= cnt +
```

1'b1;end

//跳过检验和

```
7 :begin cnt <= cnt +
```

1'b1;end

//跳过校验和

```
8 :begin
```

```
    O_R_udp_valid <= 1'b1;
```

//UDP 接收数据有效

```
    O_R_udp_data <= I_udp_ip_rdata;
```

```
    O_R_udp_len <= udp_pkg_len - 16'd8;
```

```
    cnt <= cnt + 1'b1;
```

```
end
```

```
9 :begin
```

```
    if(udp_pkg_len < 16'd26) begin
```

```
        if(udp_data_cnt == udp_pkg_len) begin
```

```
            O_R_udp_valid <= 1'b0;
```

```
            O_R_udp_data <= 8'd0;
```

```
            cnt <= cnt + 1'b1;
```

```
        end
```

```
    else begin
```

```
        O_R_udp_valid <= 1'b1;
```

```
        O_R_udp_data <= I_udp_ip_rdata;
```

```
        cnt <= cnt;
```

```
    end
```

```
end
```

```
else begin
```

```
    O_R_udp_valid <= 1'b1;
```

```
    O_R_udp_data <= I_udp_ip_rdata;
```

```
    cnt <= cnt;
```

```
end
```

```
end
```

```
10 :begin
```

```
    O_R_udp_valid <= 1'b0;
```

```
    O_R_udp_data <= 8'd0;
```

```
    cnt <= cnt;
```

```
end
```

```
default: cnt <= 4'd0;
```

```
endcase
```

```
end
```

```
else if(!I_udp_ip_rvalid) begin
```

```
    udp_pkg_len <= 16'd0;
```

```
    udp_src_port <= 16'd0;
```

```
    udp_dest_port <= 16'd0;
```

```
    udp_data_cnt <= 16'd0;
```

```
    O_R_udp_len <= 16'd0;
```

```
    O_R_udp_data <= 8'd0;
```

```

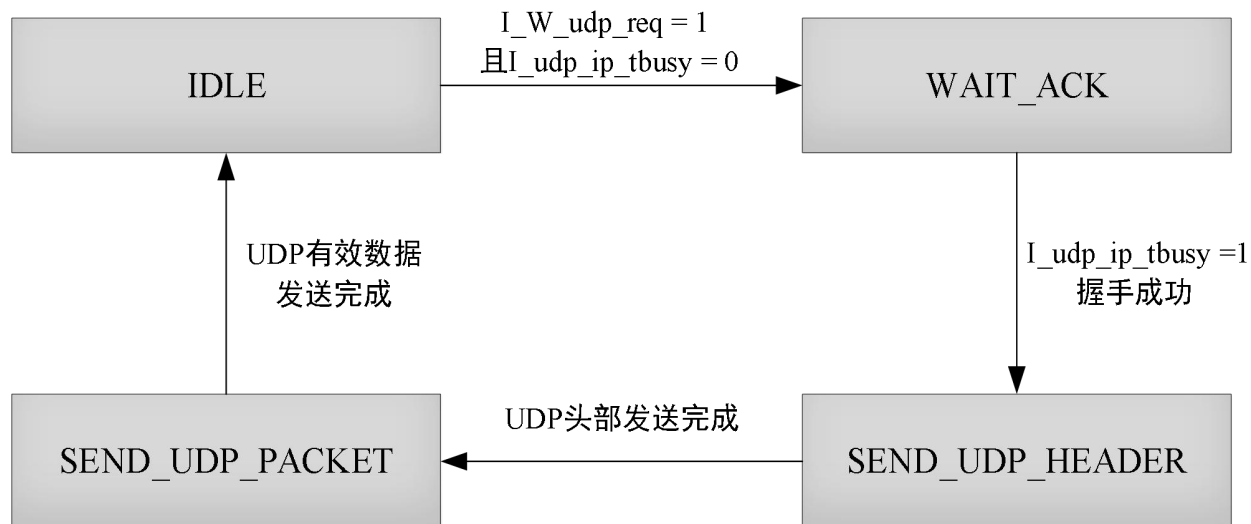
        O_R_udp_valid    <= 1'b0;
        cnt              <= 4'd0;

    end
end

```

3.5.2 UDP 发送模块

状态机转换图如下：



IDLE：当用户端发送写请求，且 uiip_tx 模块不忙时，进入 WAIT_ACK 状态。

WAIT_ACK：等待 uiip_tx 模块将 udp_ip_tbusy 信号拉高，完成握手，进入 SEND_UDP_HEADER 状态，开始发送 UDP 报文头部。

SEND_UDP_HEADER：将 8 个字节的 UDP 报文头部添加在有效数据之前，头部数据发送完成后，进入 SEND_UDP_PACKET 状态。

SEND_UDP_PACKET：将经过 shift_ram 移位 8 位的数据拼接在包头之后，通过计数器判断发送数据的数量，待数据发送完成后，回到 IDLE 状态，等待下一次发送请求。

```

always@(posedge I_W_udp_clk or posedge I_reset) begin

    if(I_reset) begin
        cnt              <= 4'd0;
        O_udp_ip_tvalid  <= 1'b0;
        O_udp_ip_tdata   <= 8'd0;
        O_udp_ip_tpkg_len <= 16'd0;
        trans_data_cnt   <= 16'd0;
        STATE            <= IDLE;
    end
end

```

```

else begin
    case (STATE)
        IDLE:begin
            if(I_W_udp_req & (~I_udp_ip_tbusy)) //当有写 UDP 请求, 并且 ip_tx 模块不忙(当
I_udp_ip_tbusy=1 代表正在 ip 层正在传输数据)
                STATE    <=  WAIT_ACK;                //进入 WAIT_ACK
            else
                STATE    <=  IDLE;
        end
        WAIT_ACK:begin
            if(I_udp_ip_tbusy)                //如果 ip_tx 模块准备好, 代表
udp_layer 可以发送数据
                STATE    <=  SEND_UDP_HEADER;
            else
                STATE    <=  WAIT_ACK;
        end
        SEND_UDP_HEADER:begin
            case(cnt)
                0 :begin
                    if(I_W_udp_valid) begin
                        O_udp_ip_tvalid    <=  1'b1;                //
udp 包数据有效

                        O_udp_ip_tdata    <=  I_udp_local_port[15:8]; //UDP 报文
源端口

                        O_udp_ip_tpkg_len    <=  I_W_udp_len + 16'h0008; //UDP 报文长
度, 其中 8bytes 为 udp 首部

                        cnt                <=  cnt + 1'b1;
                    end
                else
                    cnt                <=  0;
            end
                1 :begin
                    O_udp_ip_tdata    <=  I_udp_local_port[7:0]; //UDP 报文
源端口

                    cnt                <=  cnt + 1'b1;
                end
                2 :begin
                    O_udp_ip_tdata    <=  I_udp_dest_port[15:8]; //UDP 报文
目的端口

                    cnt                <=  cnt + 1'b1;
                end
                3 :begin
                    O_udp_ip_tdata    <=  I_udp_dest_port[7:0]; //UDP 报文
目的端口

                    cnt                <=  cnt + 1'b1;
                end
                4 :begin

```

```

                                0_udp_ip_tdata      <=  0_udp_ip_tpkg_len[15:8]; //UDP 报文
长度
                                cnt                  <=  cnt + 1'b1;
                                end
                                5 :begin
                                0_udp_ip_tdata      <=  0_udp_ip_tpkg_len[7:0]; //UDP 报文
长度
                                cnt                  <=  cnt + 1'b1;
                                end
                                6 :begin
                                0_udp_ip_tdata      <=  CHECKSUM[7:0];           //校
验和
                                cnt                  <=  cnt + 1'b1;
                                end
                                7 :begin
                                0_udp_ip_tdata      <=  CHECKSUM [7:0];           //校验
和
                                cnt                  <=  0;
                                STATE                <=  SEND_UDP_PACKET;
                                end
                                default: cnt          <=  0;
                                endcase
                                end
                                SEND_UDP_PACKET:begin
                                if(trans_data_cnt != (0_udp_ip_tpkg_len - 16'd8)) begin
                                0_udp_ip_tvalid      <=  1'b1;
                                0_udp_ip_tdata      <=  shift_data_out;
                                trans_data_cnt      <=  trans_data_cnt + 1'b1;
                                STATE                <=  SEND_UDP_PACKET;
                                end
                                else begin
                                trans_data_cnt      <=  16'd0;
                                0_udp_ip_tdata      <=  8'd0;
                                0_udp_ip_tvalid      <=  1'b0;
                                0_udp_ip_tpkg_len    <=  16'd0;
                                cnt                  <=  0;
                                STATE                <=  IDLE;
                                end
                                end
                                end
                                default: STATE        <=  IDLE;
                                endcase
                                end
                                end

```

3.6 ICMP 层

该层在程序中为 IP 层的子层，设计了接收 ICMP 请求包并回复的功能。

3.6.1 ICMP 接收模块

该模块首先过滤掉 ICMP 报文头部，并且对头部数据进行校验，校验方式和 IP 首部校验相同。

```
//ICMP 报文的校验和
always@(posedge I_clk or posedge I_reset) begin
    if(I_reset) begin
        accum1            <= 16'd0;
        accum2            <= 32'd0;
        checksum_state     <= 2'd0;
        checksum_correct   <= 1'b1;
    end
    else begin
        case(checksum_state)
            0:begin
                if(I_icmp_pkg_valid) begin
                    accum1[15:8] <= I_icmp_pkg_data;
                    checksum_state <= 2'd1;
                end
                else begin
                    accum1[15:8] <= 8'd0;
                    checksum_state <= 2'd0;
                end
            end
            1:begin
                accum1[7:0] <= I_icmp_pkg_data;
                checksum_state <= 2'd2;
            end
            2:begin
                if(!I_icmp_pkg_valid) begin
                    checksum_state <= 2'd3;
                    if((tmp_accum1[15:0] + tmp_accum1[31:16]) != 16'hffff)
                        checksum_correct <= 1'b0;
                    else
                        checksum_correct <= 1'b1;
                end
                else begin
                    accum2 <= tmp_accum1;
                    accum1[15:8] <= I_icmp_pkg_data;
                    checksum_state <= 2'd1;
                end
            end
            3:begin
                accum1 <= 16'd0;
                accum2 <= 32'd0;
                checksum_state <= 2'd0;
            end
        endcase
    end
end
```

```

end
end

```

将报文的附加数据存入 FIFO 中，并向 icmp_pkg_tx 模块发送 ping 应答信息，请求发送 ping 应答包。

```

//以下模块完成 ICMP 报文包 echo ping 应答的请求，并且先缓存到 ip_layer 的 FIFO 中
always@(posedge I_clk or posedge I_reset) begin
    if(I_reset) begin
        cnt                                <= 4'd0;
        type1                             <= 8'd0;
        code                              <= 8'd0;
        echo_data_cnt                     <= 10'd0;
        checksum                          <= 16'd0;
        O_icmp_req_en                     <= 1'b0;
        O_icmp_req_id                     <= 16'd0;
        O_icmp_req_sq_num                  <= 16'd0;
        O_icmp_ping_echo_data_valid <= 1'b0;
        O_icmp_ping_echo_data             <= 8'd0;
        O_icmp_req_checksum                <= 16'd0;
        STATE                             <= RECORD_ICMP_HEADER;
    end
    else begin
        case (STATE)
            RECORD_ICMP_HEADER:begin
                O_icmp_req_en              <= 1'b0;
                echo_data_cnt              <= 10'd0;
                if(I_icmp_pkg_valid)       //ICMP 报文有效
                    case(cnt)
                        0: begin type1 <= I_icmp_pkg_data;
cnt <= cnt + 1'b1;end //ICMP 报文首部-类型:8 位数表示错误类型的差错报文或者查询类型的报告报文,一般是查询报文 (0 代表回显应答(ping 应答); 1 代表查
                        1: begin code <= I_icmp_pkg_data;
cnt <= cnt + 1'b1;end //ICMP 报文首部-类型:代码占用 8 位数据,根据 ICMP 差错报文的类型,进一步分析错误的原因
                        2: begin checksum[15:8] <= I_icmp_pkg_data;
cnt <= cnt + 1'b1;end //ICMP 报文首部-校验和:16 位校验和的计算方法与 IP 首部校验和计算方法一致,该校验和需要对 ICMP 首部和 ICMP 数据做校验
                        3: begin checksum[7:0] <= I_icmp_pkg_data;
cnt <= cnt + 1'b1;end //ICMP 报文首部-校验和:16 位校验和的计算方法与 IP 首部校验和计算方法一致,该校验和需要对 ICMP 首部和 ICMP 数据做校验
                        4: begin O_icmp_req_id[15:8] <= I_icmp_pkg_data; cnt
nt <= cnt + 1'b1;end //ICMP 报文首部-标识符:16 位标识符对每一个发送的数据报进行标识
                        5: begin O_icmp_req_id[7:0] <= I_icmp_pkg_data;
cnt <= cnt + 1'b1;end //ICMP 报文首部-标识符:16 位标识符对每一个发送的数据报进行标识
                        6: begin O_icmp_req_sq_num[15:8] <= I_icmp_pkg_data; cnt
<= cnt + 1'b1;end //ICMP 报文首部-序列号:16 位对发送的每一个数据报文进行编号
                        7: begin O_icmp_req_sq_num[7:0] <= I_icmp_pkg_data; cn
t <= cnt + 1'b1;end //ICMP 报文首部-序列号:16 位对发送的每一个数据报文进行编号
                        8: begin

```

```

                                if(type1 == PING_REQUEST && code == 8'h00) begin //
如果是远端主机发的 ping 请求包，那么本地主机需要返回一个 ping 应答包
                                0_icmp_ping_echo_data_valid
<= 1'b1;                                //ping 应答有效
                                0_icmp_ping_echo_data      <=  I_icmp_pkg_data;
                                end
                                else begin
                                0_icmp_ping_echo_data_valid <= 1'b0;
                                0_icmp_ping_echo_data      <=  8'd0;
                                end
                                cnt      <= 4'd0;
                                STATE    <= WAIT_PACKET_END;
                                end
                                default: STATE <= RECORD_ICMP_HEADER;
                                endcase
                                else
                                STATE <= RECORD_ICMP_HEADER;
                                end
WAIT_PACKET_END:begin
                                if(I_icmp_pkg_valid) begin //继续接收 ICMP 报文
                                if(0_icmp_ping_echo_data_valid) //ping 应答有效
                                echo_data_cnt <= echo_data_cnt + 1'b1; //ICMP 包计数器
                                else
                                echo_data_cnt <= 10'd0;
                                0_icmp_ping_echo_data_valid <= 0_icmp_ping_echo_data_valid;
                                0_icmp_ping_echo_data      <=  I_icmp_pkg_data;
                                STATE <= WAIT_PACKET_END;
                                end
                                else begin
                                if(0_icmp_ping_echo_data_valid) begin
                                0_icmp_req_en <= 1'b1; //通知 ip_tx 模块接收
到 ICMP 报文包 ping 请求，并且发送一个 echo ping 应答
                                0_icmp_req_checksum <= checksum_temp; //输出校验和
                                end
                                else begin
                                0_icmp_req_en <= 1'b0;
                                0_icmp_req_checksum <= 16'd0;
                                end
                                echo_data_cnt <= echo_data_cnt;
                                0_icmp_ping_echo_data_valid <= 1'b0;
                                0_icmp_ping_echo_data      <=  8'd0;
                                STATE <= RECORD_ICMP_HEADER;
                                end
                                end
                                endcase
                                end
                                end
end

```

3.6.2 ICMP 发送模块

该模块收到 icmp_pkg_ctrl 模块发送的 ping 应答包使能信号，寄存接收到的信息包括标识符、序列号、校验和、地址和数据长度，通过计数器添加报文头部，再将附加数据从 FIFO 中读出，组成 ping 应答包发送到 ip_tx 模块。

```

always@(posedge I_clk or posedge I_reset) begin
    if(I_reset) begin
        cnt1                                <= 4'd0;
        cnt2                                <= 10'd0;
        request_id                           <= 16'd0;
        request_sq_num                       <= 16'd0;
        request_ip_taddress                  <= 32'd0;
        checksum                             <= 16'd0;
        echo_data_length                     <= 10'd0;
        O_icmp_pkg_req                       <= 1'b0;
        O_icmp_pkg_valid                     <= 1'b0;
        O_icmp_pkg_data                       <= 8'd0;
        O_icmp_ping_echo_ren                 <= 1'b0;
        STATE                                <= WAIT_ICMP_PACKET;
    end
    else begin
        case (STATE)
            WAIT_ICMP_PACKET:begin
                if(I_icmp_req_en) begin//当接收到 ICMP echo ping 包,先保存该包的基本信息到寄存器
                    request_id              <= I_icmp_req_id;                //I
CMP 包的标识符

                    request_sq_num          <= I_icmp_req_sq_num;            //ICMP
包的序列号

                    request_ip_taddress     <= I_icmp_req_ip_addr;          //ICMP 包的
地址

                    checksum                 <= I_icmp_req_checksum;        //ICM
P 包的校验和

                    echo_data_length        <= I_icmp_ping_echo_data_len;  //ICMP 包的长
度

                    O_icmp_pkg_req          <= 1'b1;                        /
/请求 ip_tx 模块发送部分，发送 ICMP 报文

                    STATE                    <= WAIT_PACKET_SEND;        //
发送 ICMP 包状态

                end
            else begin
                request_id                  <= 16'd0;
                request_sq_num              <= 16'd0;
                request_ip_taddress         <= 32'd0;
                checksum                     <= 16'd0;
                echo_data_length             <= 10'd0;
            end
        endcase
    end
end

```

```

0_icmp_pkg_req      <= 1'b0;
STATE               <= WAIT_ICMP_PACKET;

end

end

WAIT_PACKET_SEND:begin
    if(I_icmp_pkg_busy) begin//该信号来自 ip_tx 模块，当有效代表 ip_tx 模块已经开始准备
发送 ICMP 包，这里需要对 ip_tx 代码部分时序逻辑确保数据正确给到 ip_tx 模块
        0_icmp_pkg_req      <= 1'b0;
        0_icmp_pkg_valid    <= 1'b1;
        0_icmp_pkg_data     <= PING_REPLY_TYPE;//回显应答(ping 应答)的类
型

        STATE              <= SEND_PACKET;

    end
else begin
        0_icmp_pkg_req      <= 1'b1;
        0_icmp_pkg_valid    <= 1'b0;
        0_icmp_pkg_data     <= 8'd0;
        STATE              <= WAIT_PACKET_SEND;

    end
end

SEND_PACKET:begin
    case(cnt1)
        0 :begin 0_icmp_pkg_data
<= 8'h00; cnt1 <= cnt1 + 1'b1;end//回显应答(ping 应答)的代码
        1 :begin 0_icmp_pkg_data
<= checksum[15:8]; cnt1 <= cnt1 + 1'b1;end//ICMP 报文包校验和，直接获取远程主机发送的
Ping 包校验和
        2 :begin 0_icmp_pkg_data
<= checksum[7:0]; cnt1 <= cnt1 + 1'b1;end//ICMP 报文包校验和，直接获取远程主机发送
的 Ping 包校验和
        3 :begin 0_icmp_pkg_data
<= request_id[15:8]; cnt1 <= cnt1 + 1'b1;end//ICMP 报文标识符，直接获取远程主机发送的 Ping
包标识符
        4 :begin 0_icmp_pkg_data
<= request_id[7:0]; cnt1 <= cnt1 + 1'b1;end//ICMP 报文标识符，直接获取远程主机发送的
Ping 包标识符
        5 :begin 0_icmp_pkg_data
<= request_sq_num[15:8]; cnt1 <= cnt1 + 1'b1;end//ICMP 报文编码，直接获取远程主机发送的 Ping
序列号
        6 :begin //从 echo FIFO 中读取 ICMP echo 报文的数据部分
            0_icmp_pkg_data <= request_sq_num[7:0];
            cnt1 <= cnt1 + 1'b1;
            0_icmp_ping_echo_ren <= 1'b1;
        end
        7 :begin//ICMP 报文包的数据有效部分
            0_icmp_pkg_valid <= 1'b1;
            0_icmp_pkg_data <= I_icmp_ping_echo_data;
    end
end

```

```

        if(cnt2 == (echo_data_length - 1)) begin
            cnt2                <= 10'd0;
            0_icmp_ping_echo_ren <= 1'b0;
            cnt1                <= cnt1 + 1'b1;
        end
        else begin
            cnt2                <= cnt2 + 1'b1;
            0_icmp_ping_echo_ren <= 1'b1;
            cnt1                <= cnt1;
        end
    end

    end
    8 :begin
        cnt1                <= 4'd0;
        0_icmp_pkg_data      <= 8'd0;
        0_icmp_pkg_valid     <= 1'b0;
        STATE               <= WAIT_ICMP_PACKET;
    end
    default::;
endcase
end
endcase
end
end

```

4 仿真验证

仿真代码的顶层如下:

```

`timescale 1ns / 1ps
module sim_top;
    reg          I_reset;
    reg          I_clk;
    wire         b_r_udp_valid;
    wire [7:0] b_r_udp_data;
    wire [15:0] b_r_udp_data_len;
    wire [15:0] b_r_udp_src_port;
    wire         0_a_ip_rx_error;
    wire         0_a_mac_rx_error;
    wire         0_b_ip_rx_error;
    wire         0_b_mac_rx_error;
    test_udp_loop test_udp_loop_u(
        .I_reset          (I_reset),
        .I_clk            (I_clk),

        .b_r_udp_valid    (b_r_udp_valid),
        .b_r_udp_data     (b_r_udp_data),
        .b_r_udp_data_len (b_r_udp_data_len),

```

```

.b_r_udp_src_port      (b_r_udp_src_port),

.0_a_ip_rx_error       (0_a_ip_rx_error),
.0_a_mac_rx_error      (0_a_mac_rx_error),
.0_b_ip_rx_error       (0_b_ip_rx_error),
.0_b_mac_rx_error      (0_b_mac_rx_error)
);
initial begin
    I_clk = 0;
    I_reset = 1;
    #2000;
    I_reset = 0;
end
always #4 I_clk <= ~I_clk;
endmodule

```

4.1 数据通信仿真

例化两个 UDP 协议栈模块，分别为主机 A 和主机 B，主机 A 将用户端输入的数据打包，通过 GMII 接口发送给主机 B，主机 B 将数据解包后输出至用户端，观察各层的信号变化。

4.1.1 仿真代码编写

初始状态机为 WAIT_UDP_RDY，等待主机 A 的 O_W_udp_busy 信号为低时，用户端将 I_W_udp_req 信号拉高，状态机进入 WAIT_UDP_ACK 状态。待主机 A 将 O_W_udp_busy 拉高时，用户端将 I_W_udp_req 拉低，状态机跳转至 SEND_DATA 状态，计数器开始计数，发送的数据为计数器的低 8 位。当数据都发送完毕时，状态机跳回 WAIT_UDP_RDY 状态，等待下一次数据的发送。仿真代码如下：

```

`timescale 1ns / 1ps

module test_udp_loop(
input  wire      I_reset,
input  wire      I_clk,

output wire      b_r_udp_valid,
output wire [7:0] b_r_udp_data,
output wire [15:0] b_r_udp_data_len,
output wire [15:0] b_r_udp_src_port,

output wire      0_a_ip_rx_error,
output wire      0_a_mac_rx_error,
output wire      0_b_ip_rx_error,

```

```

output wire          0_b_mac_rx_error
);

wire          a_w_udp_rdy;
reg           a_w_udp_req;
reg           a_w_udp_valid;
wire [7:0]    a_w_udp_data;
reg  [15:0]   a_w_udp_len;
reg           a_w_udp_data_read;

wire          b_w_udp_rdy;
reg           b_w_udp_req;
reg           b_w_udp_valid;
wire [7:0]    b_w_udp_data;
reg  [15:0]   b_w_udp_len;
reg           b_w_udp_data_read;

wire          a_r_udp_valid;
wire [7 :0]   a_r_udp_data;
wire [15:0]   a_r_udp_data_len;
wire [15:0]   a_r_udp_src_port;

reg  [9 :0]   test_data;
reg  [1 :0]   STATE;

parameter    WAIT_UDP_READY = 0;
parameter    WAIT_UDP_ACK   = 1;
parameter    SEND_DATA      = 2;

always@(posedge I_clk or posedge I_reset)begin
    if(I_reset) begin
        a_w_udp_req          <= 1'b0;
        a_w_udp_valid        <= 1'b0;
        a_w_udp_len          <= 16'd0;
        test_data            <= 10'd0;
        STATE                 <= WAIT_UDP_READY;

    end
    else begin
        case(STATE)
            WAIT_UDP_READY:begin
                if(~a_w_udp_rdy) begin
                    a_w_udp_req          <= 1'b1;
                    STATE                 <= WAIT_UDP_ACK;
                end
            else begin
                a_w_udp_req          <= 1'b0;

```

```

        STATE                                     <= WAIT_UDP_READY;

    end

end

WAIT_UDP_ACK:begin
    if(a_w_udp_rdy) begin
        a_w_udp_len                               <= 16'd768;
        a_w_udp_valid                             <= 1'b1;
        a_w_udp_req                               <= 1'b0;
        STATE                                     <= SEND_DATA;

    end

    else

        STATE <= WAIT_UDP_ACK;

    end

SEND_DATA:begin
    if(test_data == 10'd767) begin
        a_w_udp_valid                             <= 1'b0;
        a_w_udp_len                               <= 16'd0;
        test_data                                 <= 0;
        STATE                                     <= WAIT_UDP_READY;

    end

    else begin
        a_w_udp_valid                             <= 1'b1;
        test_data                                 <= test_data + 1'b1;
        STATE                                     <= SEND_DATA;

    end

end

endcase

end

end

//以下实现 A 发送,B 接收的仿真测试
wire [7:0]  a_gmii_tdata;
wire        a_gmii_tvalid;
wire [7:0]  b_gmii_tdata;
wire        b_gmii_tvalid;

assign  a_w_udp_data = test_data[7:0];

udp_stack #
(
    .CRC_GEN_EN          (1'b1),
    .INTER_FRAME_GAP    (4'd12)
)
A_udp_stack
(
    .I_uclk              (I_clk),
    .I_reset             (I_reset),

```

```

.I_mac_local_addr      (48'h0123456789a2), //本地 MAC 地址
.I_udp_local_port      (16'd6002          ), //本地端口号
.I_ip_local_addr       (32'hc0a88902      ), //本地 ip 地址

.I_udp_dest_port       (16'd6001          ), //目的端口
.I_ip_dest_addr        (32'hc0a88901      ), //目的 IP 地址

.O_W_udp_busy          (a_w_udp_rdy),
.I_W_udp_req           (a_w_udp_req),
.I_W_udp_valid         (a_w_udp_valid),
.I_W_udp_data          (a_w_udp_data),
.I_W_udp_len           (a_w_udp_len),

.O_R_udp_valid         (),
.O_R_udp_data          (),
.O_R_udp_len           (),
.O_R_udp_src_port      (),

.I_gmii_rclk           (I_clk),
.I_gmii_rvalid         (b_gmii_tvalid),
.I_gmii_rdata          (b_gmii_tdata),

.I_gmii_tclk           (I_clk),
.O_gmii_tvalid         (a_gmii_tvalid),
.O_gmii_tdata          (a_gmii_tdata),
.O_ip_rerror           (0_a_ip_rx_error),
.O_mac_rerror          (0_a_mac_rx_error)
);

udp_stack #
(
.CRC_GEN_EN            (1'b1),
.INTER_FRAME_GAP       (4'd12)
)
B_udp_stack
(
.I_uclk                (I_clk),
.I_reset               (I_reset),

.I_mac_local_addr      (48'h0123456789a1), //本地 MAC 地址
.I_udp_local_port      (16'd6001          ), //本地端口号
.I_ip_local_addr       (32'hc0a88901      ), //本地 ip 地址

.I_udp_dest_port       (16'd6002          ), //目的端口
.I_ip_dest_addr        (32'hc0a88902      ), //目的 IP 地址

.O_W_udp_busy          (),
.I_W_udp_req           (0),

```

```

.I_W_udp_valid          (0),
.I_W_udp_data           (0),
.I_W_udp_len            (0),

.O_R_udp_valid          (b_r_udp_valid),
.O_R_udp_data           (b_r_udp_data),
.O_R_udp_len            (b_r_udp_data_len),
.O_R_udp_src_port       (b_r_udp_src_port),

.I_gmii_rclk            (I_clk),
.I_gmii_rvalid          (a_gmii_tvalid),
.I_gmii_rdata           (a_gmii_tdata),

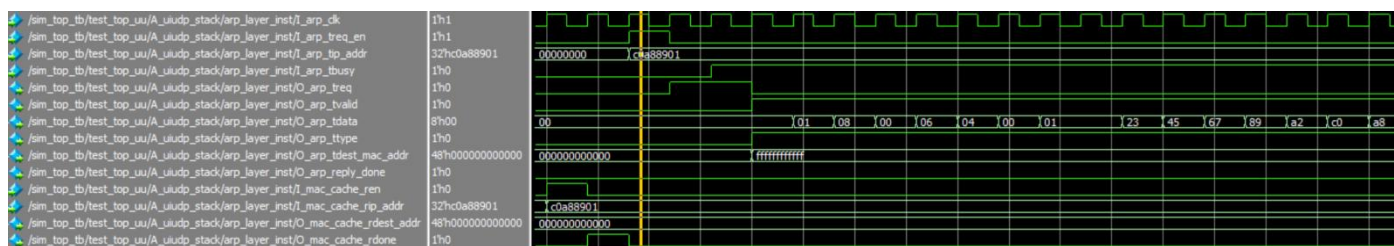
.I_gmii_tclk            (I_clk),
.O_gmii_tvalid          (b_gmii_tvalid),
.O_gmii_tdata           (b_gmii_tdata),
.O_ip_rerror             (0_b_ip_rx_error),
.O_mac_rerror            (0_b_mac_rx_error)
);

endmodule

```

4.1.2 ARP 请求仿真结果

主机 A 将数据打包发送给主机 B 时，由于主机 A 的 cache 中查询不到主机 B 的 MAC 地址，主机 A 会先发送一个 ARP 请求包给主机 B。



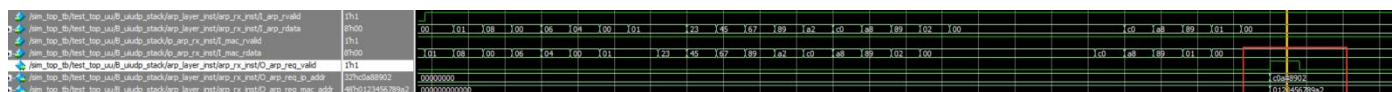
当数据帧发送至 ip_arp_tx 模块时,该模块向 ARP 层的 cache 发送一个 MAC 地址查询请求 l_mac_cache_ren 和需要查询的 IP 地址 l_mac_cache_ip_addr,查询结束后 O_mac_cache_rdone 拉高,返回的 MAC 地址为 48'h0,说明未查询到 MAC 地址, ip_arp_tx 模块将 l_arp_treq_en 拉高,准备发送 ARP 广播。arp_tx 模块接收到 l_arp_treq_en 高电平时,发送 ARP 请求 O_arp_req, ip_arp_tx 模块将 l_arp_busy 拉高,表示握手成功, arp_tx 模块开始组 ARP 广播包。下图为 GMII 接口发送 ARP 广播仿真波形图。



最后信号经过 MAC 层组帧后通过 GMII 接口发送至主机 B。

4.1.3 ARP 应答仿真结果

主机 B 接收到 ARP 请求包后，会将 ARP 包中的 MAC 地址解析，并将自己的 MAC 地址通过 ARP 应答包发送给主机 A。主机 A 收到 ARP 应答包后，将主机 B 的 MAC 地址存入 cache 中。下图为解包后发送至 arp_rx 模块的数据。



MAC 层解析数据的类型为 ARP 包，将该包数据通过 ip_arp_rx 模块发送给 arp_rx 模块，解析出 IP 地址、MAC 地址，并将该包识别为 ARP 请求包。arp_rx 发送 arp_req_valid，信号给 arp_tx 模块，请求发送 ARP 应答包，同时将 ARP 请求包中的主机 A 的 MAC 地址写入 cache 中。arp_tx 模块接收到应答请求并寄存，向 ip_arp_tx 模块发送请求，ip_arp_tx 模块将 busy 信号拉高以表示握手成功。busy 信号为高后，arp_tx 模块组 ARP 应答包，将本地 MAC 地址等信息发送至上层协议。

ARP 层发出的应答数据包经 MAC 层组帧，通过 GMII 接口发送至主机 A。下图为主机 A GMII 接口接收到的 ARP 请求包数据。

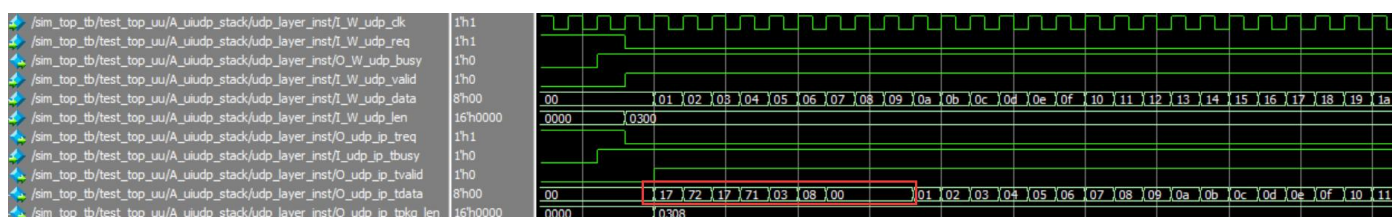


主机 A 的 MAC 层识别数据为 ARP 数据包类型，发送至 arp_rx 模块中，arp_rx 模块解析对方发送的 IP 地址和 MAC 地址，将 MAC 地址存入 cache 中。至此，地址解析完成，两机之间可以完成正常数据通信。

4.1.4 UDP 发送仿真结果

udp_tx 模块与下层模块握手成功后，将数据通过移位寄存器延迟 8 个周期，组成 UDP 包，发送至 ip_tx 模块，组成 IP 数据包，然后数据经 ip_arp_tx 模块仲裁，发送至 mac_tx 模块，组成 MAC 包后通过 GMII 接口发送。

下图为 UDP 层组 UDP 包的仿真波形图

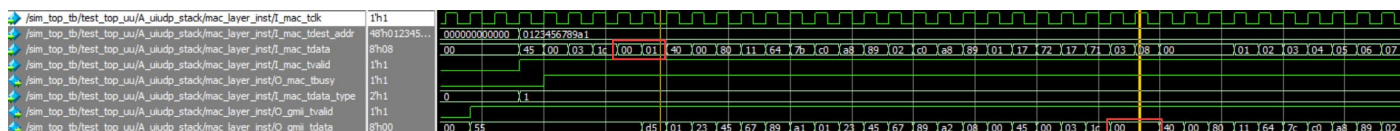


下图为 IP 层组 IP 包的仿真波形图



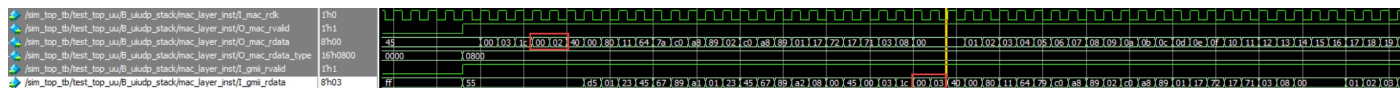
由于数据都写入了 data_fifo，当数据有效数据全部写入完成后，才会将数据读出，所以图中组的 MAC 包是上一个数据包正在发送的数据包 IP 头部的标识为 16'h0000，而从上层传入的 IP 数据包包头标识为 16'h0001。

下图为 MAC 层组 MAC 包的仿真波形图

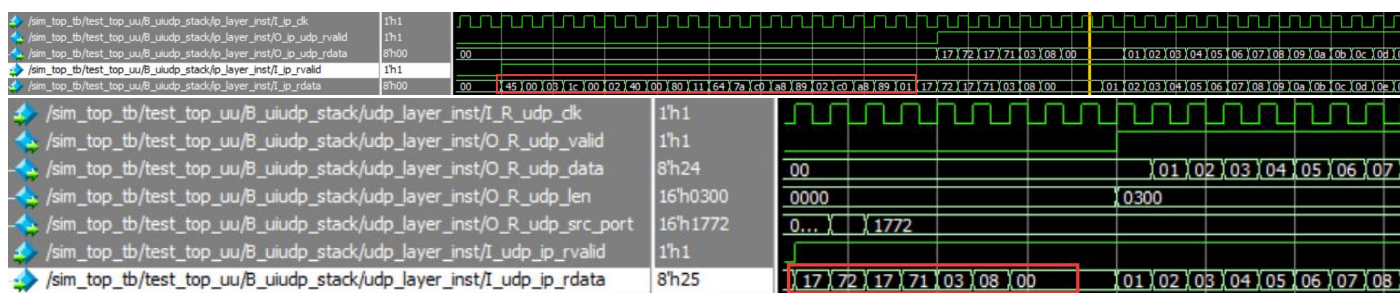


4.1.5 UDP 接收仿真结果

主机 B 的 GMII 接口接收到主机 A 发送的数据包，会将数据先存入 FIFO 做跨时钟域，当数据全部接收完成后，才会将数据读出，图中正在接收的数据包 IP 头部标识为 16'h0003，发送至上层协议的数据包标识为 16'h0002，即发送至 IP 层的数据包为上一个帧。下图为过滤 MAC 头部的仿真波形图



通过计数器去掉 IP 头部和 UDP 头部，最终得到有效数据，和发送的数据一致，如图所示。



4.2 PAUSE 流控仿真

发送数据的内容和速率与上一节的仿真中一致，每隔一定的时间，在用户端组一个 PAUSE 帧发送至主机 A，暂停时间设置为 16'h007F，即主机 A 实际暂停时间为 8128 个时钟周期 (7F (h) << 6 = 8128 (d))。

4.2.1 仿真代码编写

通过状态机和计数器，控制 PAUSE 帧发送到主机 A 的时序和间隔，同时不断地向主机 A 的用户端写数据。仿

真代码如下：

```

always@(posedge I_clk or posedge I_reset)begin
    if(I_reset) begin
        pause_data      <=      8'd0;
        pause_vld        <=      1'b0;
        cnt1              <=      'd0;
        data_cnt          <=      'd0;
        STATE2            <=      WAIT_PAUSE_RDY;
    end
    else begin
        case(STATE2)
            WAIT_PAUSE_RDY:begin
                data_cnt      <=      'd0;
                if(cnt1 == 16'd5000) begin
                    cnt1      <=      'd0;
                    STATE2    <=      SEND_PAUSE_DATA;
                end
                else begin
                    cnt1      <=      cnt1 + 1'b1;
                    STATE2    <=      WAIT_PAUSE_RDY;
                end
            end
            SEND_PAUSE_DATA:begin
                data_cnt      <=      data_cnt + 1'b1;
                case(data_cnt)
                    0      :begin pause_data <= 8'h55; pause_vld <= 1'b1;end
                    1      :begin pause_data <= 8'h55;end
                    2      :begin pause_data <= 8'h55;end
                    3      :begin pause_data <= 8'h55;end
                    4      :begin pause_data <= 8'h55;end
                    5      :begin pause_data <= 8'h55;end
                    6      :begin pause_data <= 8'h55;end
                    7      :begin pause_data <= 8'hd5;end

                    8      :begin pause_data <= 8'h01;end
                    9      :begin pause_data <= 8'h80;end
                    10     :begin pause_data <= 8'hc2;end
                    11     :begin pause_data <= 8'h00;end
                    12     :begin pause_data <= 8'h00;end
                    13     :begin pause_data <= 8'h01;end//pause 广播地址
                    14     :begin pause_data <= 8'h01;end
                    15     :begin pause_data <= 8'h23;end
                    16     :begin pause_data <= 8'h45;end
                    17     :begin pause_data <= 8'h67;end
                    18     :begin pause_data <= 8'h89;end
                    19     :begin pause_data <= 8'ha1;end
                end
            end
        endcase
    end
end

```

```

20  :begin    pause_data    <=    8'h88;end
21  :begin    pause_data    <=    8'h08;end

22  :begin    pause_data    <=    8'h00;end
23  :begin    pause_data    <=    8'h01;end

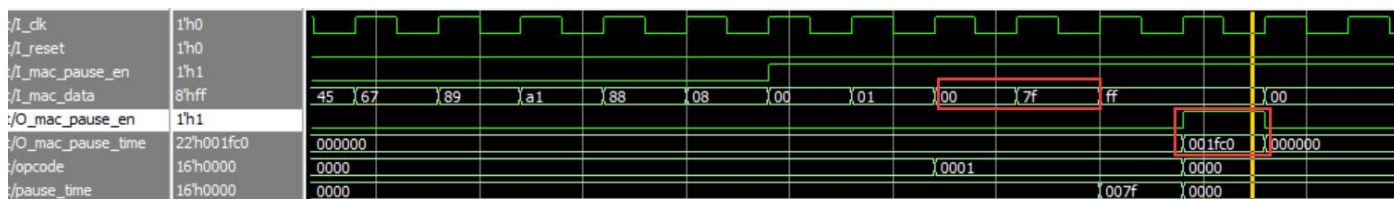
24  :begin    pause_data    <=    8'h00;end
25  :begin    pause_data    <=    8'h7f;end

26  :begin    pause_data    <=    8'hff;end
27  :begin    pause_data    <=    8'hff;end
36  :begin    pause_data    <=    8'h15;end
37  :begin    pause_data    <=    8'hbf;end
38  :begin    pause_data    <=    8'h4b;end
39  :begin    pause_data    <=    8'h6c;end
40  :begin
        pause_data    <=    8'h00;
        pause_vld      <=    1'b0;
        STATE2         <=    WAIT_PAUSE_RDY;
    end
    default:pause_data    <=    8'h00;
endcase
end
default: begin
    pause_data          <=    8'd0;
    pause_vld           <=    1'b0;
    cnt1                <=    'd0;
    data_cnt            <=    'd0;
    STATE2              <=    WAIT_PAUSE_RDY;
end
endcase
end
end
end

```

4.2.2 PAUSE 流控仿真结果

主机 A 接收到 PAUSE 帧后，会将 PAUSE 帧里的信息送入 mac_tx_frame_ctrl 模块中进行解析，得到暂停时间，并发送给定至 mac_tx 模块，如图所示。



当 mac_tx 模块中的读状态机进入帧间隔状态时，通过计数器将 pause_flag 拉高一定的时间。检测到下一帧

发送方的 MAC 地址与 PAUSE 帧发送方的 MAC 地址相同，均为 48'h0123456789a1，进入暂停状态。

仿真波形图如图，由下图可知，pause_flag 拉高的时间为 65024ns，即为 8128 个时钟周期。



4.3 ICMP 层仿真

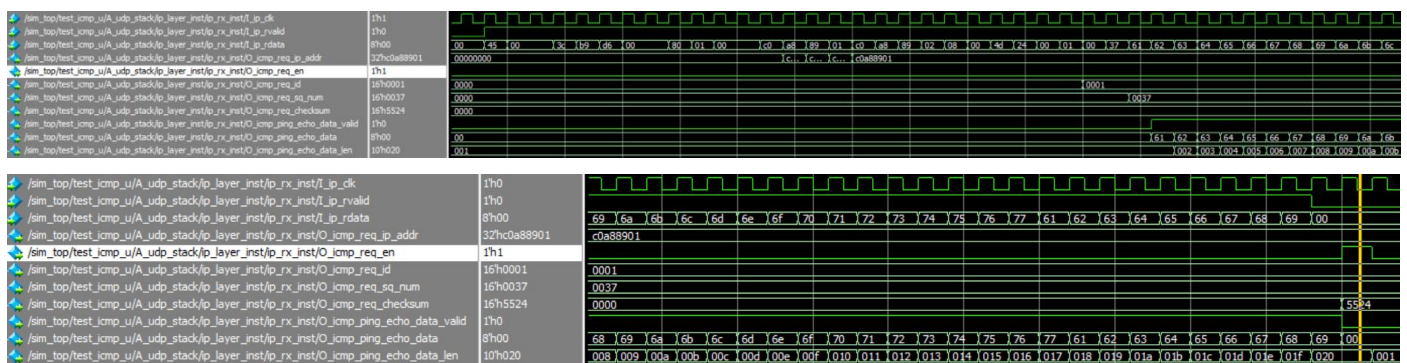
向主机 A 发送一个 ping 请求包，主机 A 成功接收到 ping 请求包后，发送一个 ping 应答包，其中的额外数据与请求包相同。

4.3.1 仿真代码编写

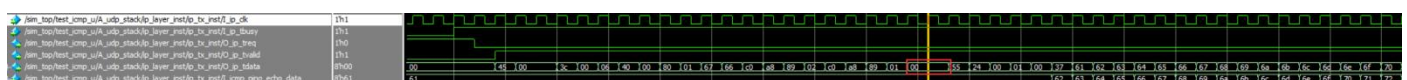
组一个 ping 请求包给主机 A，仿真代码与组 pause 帧类似，其中 IP 头部的协议类型为 8'h01，ICMP 头部的 type 字段为 8'h08，code 字段为 8'h00，表示主动请求。

4.3.2 ICMP 回显应答仿真结果

如图所示，ip_rx 模块接收到 ICMP 数据报文会将标识符、序列号、校验和等信息拆解出来，并将有效数据存入 FIFO。一帧数据接收完成后，将发送 ping 应答请求信号给 ip_tx 模块。



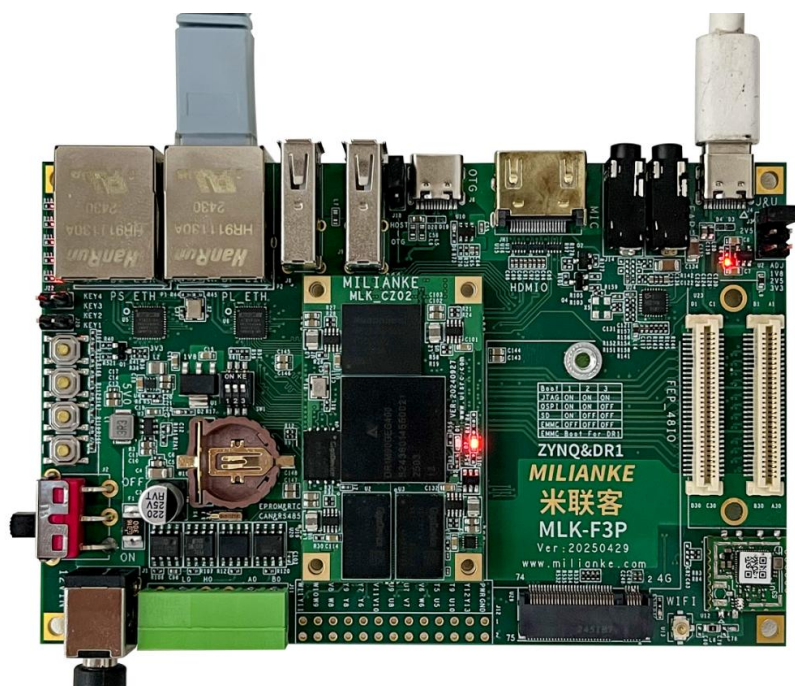
请求信号 icmp_req_en 拉高后，icmp_pkg_tx 模块中的 icmp_pkg_req 信号也拉高，并发送给 ip_tx 模块，等待 ip_tx 模块将 icmp_pkg_req 拉高，表示握手成功，开始发送 icmp 回显应答数据。ping 应答包的 type 字段为 8' h00，code 字段为 8' h00，表示回显应答，其有效数据和接收到的 ping 请求包额外数据保持一致。



5 上板调试

5.1 硬件连接

本次实验的物理层选择千兆以太网 RJ45 网口来验证该 UDP 协议栈的功能。首先将开发板的 RJ45 网口通过网线和 PC 连接，插入开发板的电源和下载器，硬件连接图如图 5.1 所示。打开电源开关，等待网口的黄灯亮起，PHY 芯片自协商成功。可以进行下一步操作。



5.2 PC 设置

5.2.1 PC 地址查询

为实现以太网的各种通信测试，我们需要先查找到 PC 端以太网网卡的物理地址和 IP 地址，以进行组包发送和对结果进行验证。

在 windows 开始菜单界面的搜索框中输入 cmd，找到命令提示符并打开，如图 5.2 所示。



打开命令提示符界面后，在界面中输入” ipconfig -all”，得到网卡的各种信息，如图所示，其中” CC-28-AA-06-33-56”为网卡的 MAC 地址，其数据格式为 16 进制，” 192.168.137.1”为网卡的 IP 地址，其数据格式为 10 进制。子网掩码” 255.255.255.0”表示若要使 FPGA 和 PC 之间建立正常的以太网通信，FPGA 与 PC 的 IP 地址的高位的三个字节必须相等，如” 192.168.137.xxx”。

```
C:\Users\Administrator>ipconfig -all

Windows IP 配置

   主机名 . . . . . : BF-202405161123
   主 DNS 后缀 . . . . . :
   节点类型 . . . . . : 混合
   IP 路由已启用 . . . . . : 否
   WINS 代理已启用 . . . . . : 否

以太网适配器 以太网 2:

   媒体状态 . . . . . : 媒体已断开连接
   连接特定的 DNS 后缀 . . . . . :
   描述 . . . . . : Mellanox ConnectX-3 Ethernet Adapter
   物理地址. . . . . : 24-8A-07-DD-54-60
   DHCP 已启用 . . . . . : 是
   自动配置已启用. . . . . : 是

以太网适配器 以太网:

   连接特定的 DNS 后缀 . . . . . :
   描述 . . . . . : Realtek Gaming 2.5GbE Family Controller
   物理地址. . . . . : CC-28-AA-06-33-56
   DHCP 已启用 . . . . . : 否
   自动配置已启用. . . . . : 是
   本地链接 IPv6 地址. . . . . : fe80::eb52:fd1:a302:25a1%23(首选)
   IPv4 地址 . . . . . : 192.168.137.1(首选)
   子网掩码 . . . . . : 255.255.255.0
   默认网关 . . . . . :
   DHCPv6 IAID . . . . . : 332146858
   DHCPv6 客户端 DUID . . . . . : 00-01-00-01-2E-0B-1C-BF-CC-28-AA-06-33-56
   TCP/IP 上的 NetBIOS . . . . . : 已启用
```

接下来输入” arp -a” 查询 PC 网卡中 ARP 链表缓存的地址，通过将 IP 地址和 MAC 地址绑定到 ARP 静态链表上，可以不使用 ARP 功能就能实现以太网 UDP 通信，由于本 UDP 协议栈实现了 ARP 应答和主动请求功能，不需要绑定地址。可以看到，网卡上并没有缓存其它动态地址。

```
C:\Users\Administrator>arp -a

接口: 192.168.101.209 --- 0x7
Internet 地址      物理地址      类型
192.168.101.1      58-f2-fc-ee-33-5e 动态
192.168.101.4      b4-8c-9d-b2-01-ab 动态
192.168.101.7      90-09-d0-4a-e4-01 动态
192.168.101.17     04-cf-4b-23-07-30 动态
192.168.101.36     78-2b-46-2c-e8-0e 动态
192.168.101.231    78-2b-46-2b-fa-b6 动态
192.168.101.255    ff-ff-ff-ff-ff-ff 静态
224.0.0.2          01-00-5e-00-00-02 静态
224.0.0.22        01-00-5e-00-00-16 静态
224.0.0.251       01-00-5e-00-00-fb 静态
224.0.0.252       01-00-5e-00-00-fc 静态
239.192.152.143    01-00-5e-40-98-8f 静态
239.255.255.250    01-00-5e-7f-ff-fa 静态
255.255.255.255    ff-ff-ff-ff-ff-ff 静态

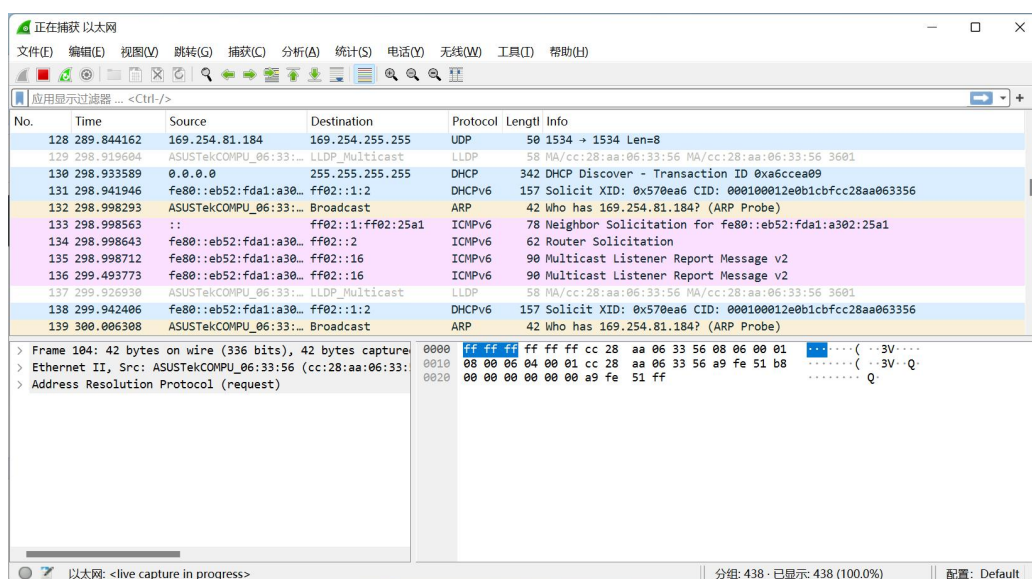
接口: 192.168.137.1 --- 0x17
Internet 地址      物理地址      类型
169.254.81.188     12-34-56-78-9a-bc 静态
224.0.0.22        01-00-5e-00-00-16 静态
224.0.0.251       01-00-5e-00-00-fb 静态
239.255.255.250    01-00-5e-7f-ff-fa 静态
255.255.255.255    ff-ff-ff-ff-ff-ff 静态

C:\Users\Administrator>
```

5.2.2 网络调试软件

本次实验需要用到三种网络调试软件，分别为 wireshark、netassist、科来数据包生成器，接下来介绍这几款软件的界面和功能。

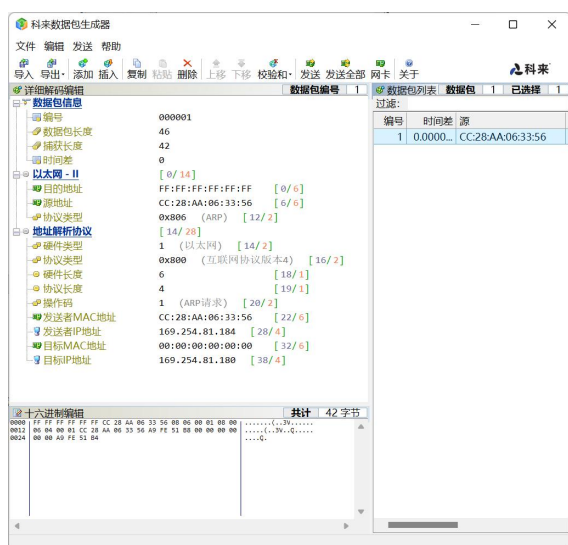
wireshark 的界面如图所示，它可以抓取到不同类型的数据包如 ARP 包、ICMP 包、UDP 包，并显示除帧头和校验位以外的数据包的全部内容。



Netassist 的界面如图所示，该工具可以通过绑定 IP 地址，对 UDP 数据进行发送或接收，并且显示接收的具体内容和收发数据的计数。



科来数据包生成器的界面如图所示，它可以组任意类型的数据包并可以将不同的数据包按照一定的间隔顺序发送。



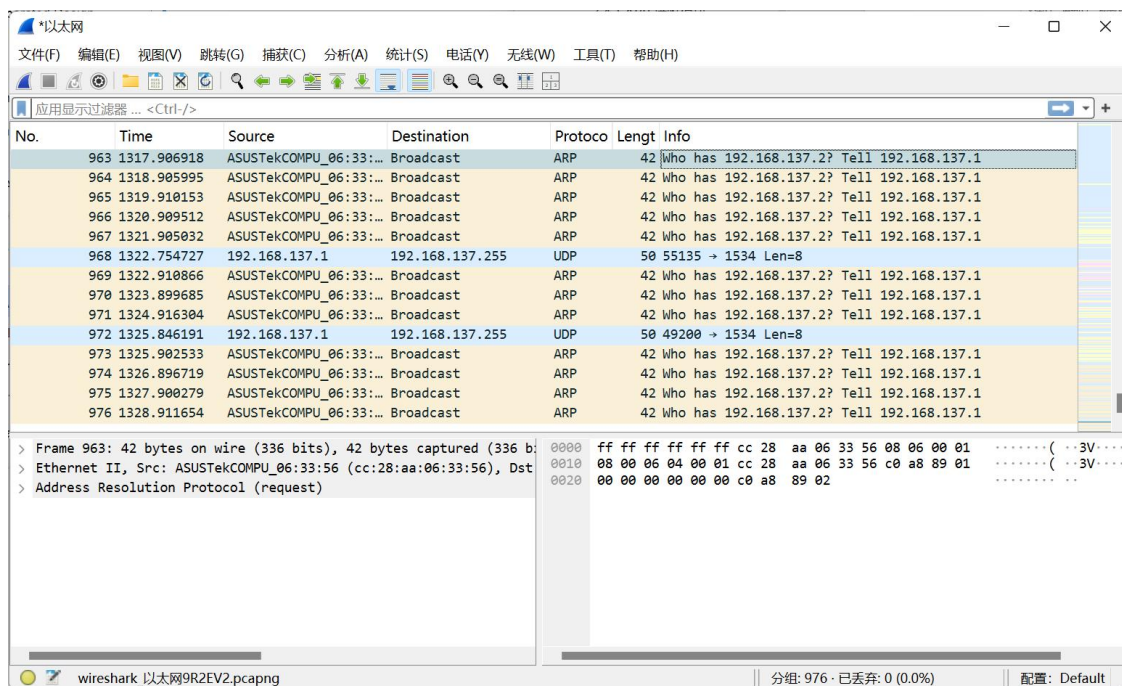
5.3 UDP 回环验证

netassist 连续发送数据给 FPGA, FPGA 将接收到的数据传回, 以此测试 UDP 协议栈的 ARP 应答功能以及 UDP 通信功能。

在代码顶层文件中将接收到的 UDP 数据包有效数据缓存中, 待 udp_tx 的 tbusy 信号为低时, 发送一次请求, 握手成功后, 发送接收到的一包数据, 这样就能实现 UDP 数据的回环。在顶层中设置 FPGA 的 MAC 地址为 48'h01_23_45_67_89_ab, IP 地址为 192.168.137.1。

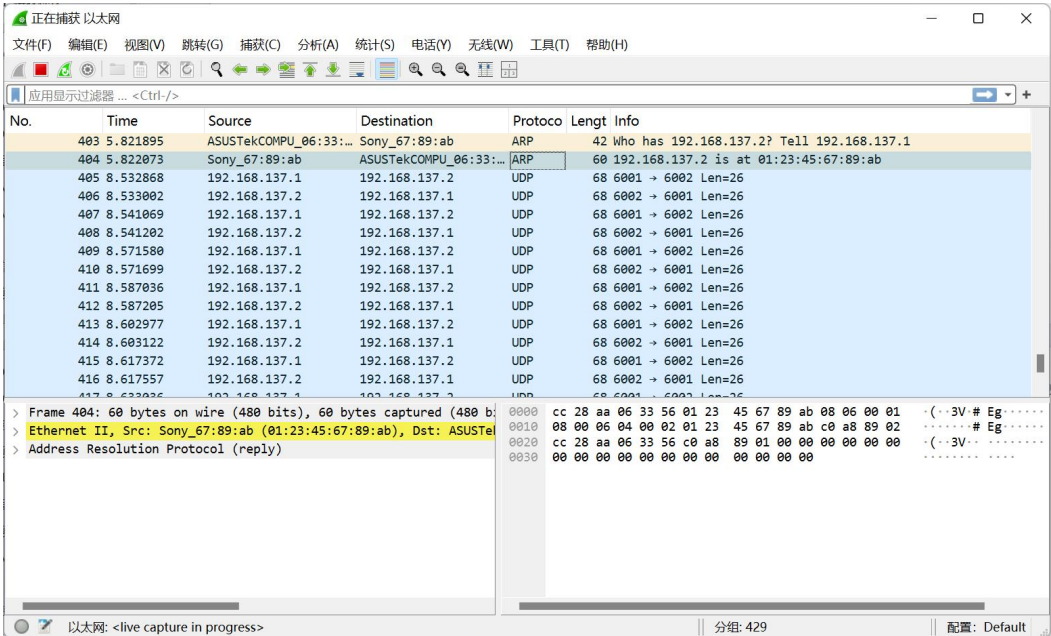
5.3.1 wireshark 抓取 PC 发送的 ARP 广播

由于 ARP 链表中没有缓存 FPGA 的地址, PC 在发送数据时, 找不到 IP 地址对应的 MAC 地址, 于是 PC 会发送 ARP 广播, 请求获取 FPGA 上设置的物理地址。先不要将程序下载进芯片中, 通过 wireshark 抓取 ARP 广播, 抓取到的数据包信息如图所示。



由上图可以得知, 发送方发送的目的 MAC 地址为 48' hff_ff_ff_ff_ff, 类型为 16' h0806, 操作码为 16' h0001, 表示 ARP 请求, 请求的 IP 地址为 192.168.137.2。接着将程序下载进 FPGA 中, 重新发送数据, wireshark 抓取到的数据包如图所示。

可以看到, FPGA 向 PC 发送的 ARP 广播回复了一个 ARP 应答包, 将自己的 MAC 地址发送出去, 操作码为 16' h0002, 表示 ARP 应答。



5.3.2 netassist 收发 UDP 数据包

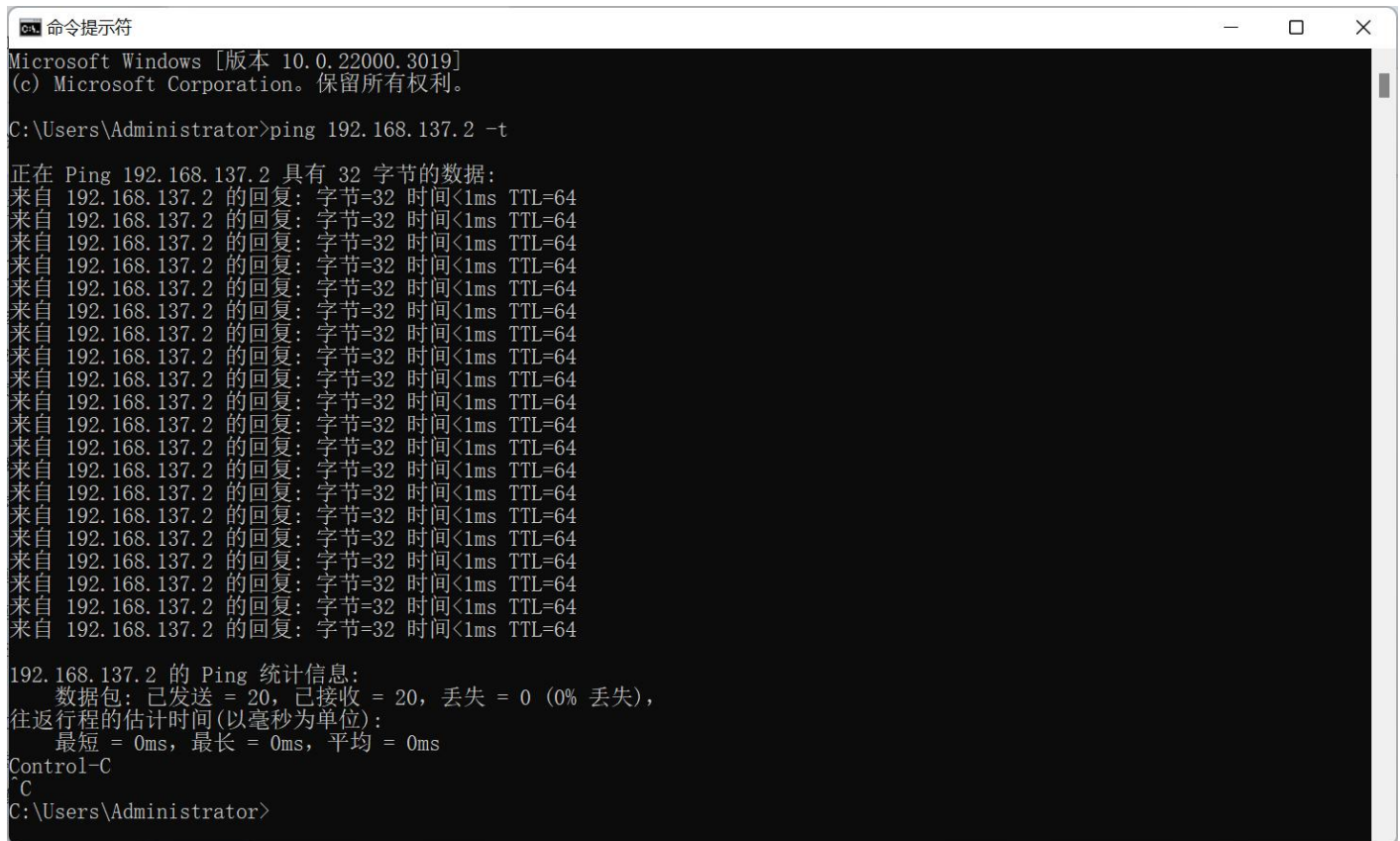
使用 netassist 接收和发送的数据如图所示，可以看出每次发送和传回的数据内容保持一致，并且接收和发送的数据包数量和有效数据数量均相等，表示程序中 UDP 发送和接收的部分功能正确。



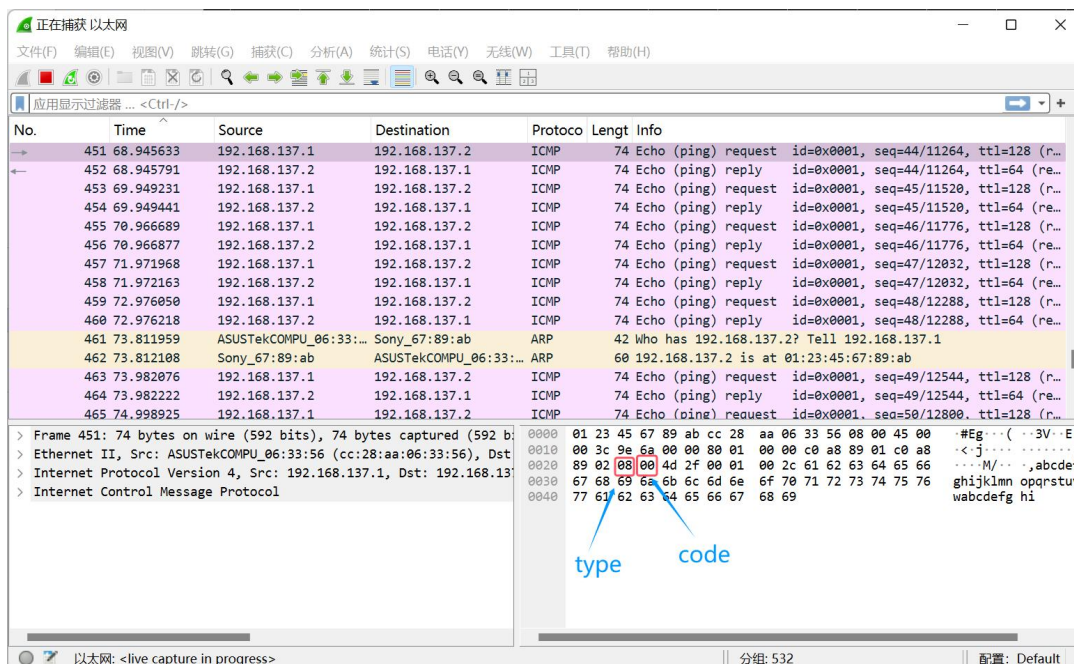
5.3.3 通过 PING 命令发送 ICMP 请求

打开 WINDOWS 命令提示符界面，输入” ping 192.168.137.2 -t” 命令，PC 会向 FPGA 持续发送 ICMP 请求包，

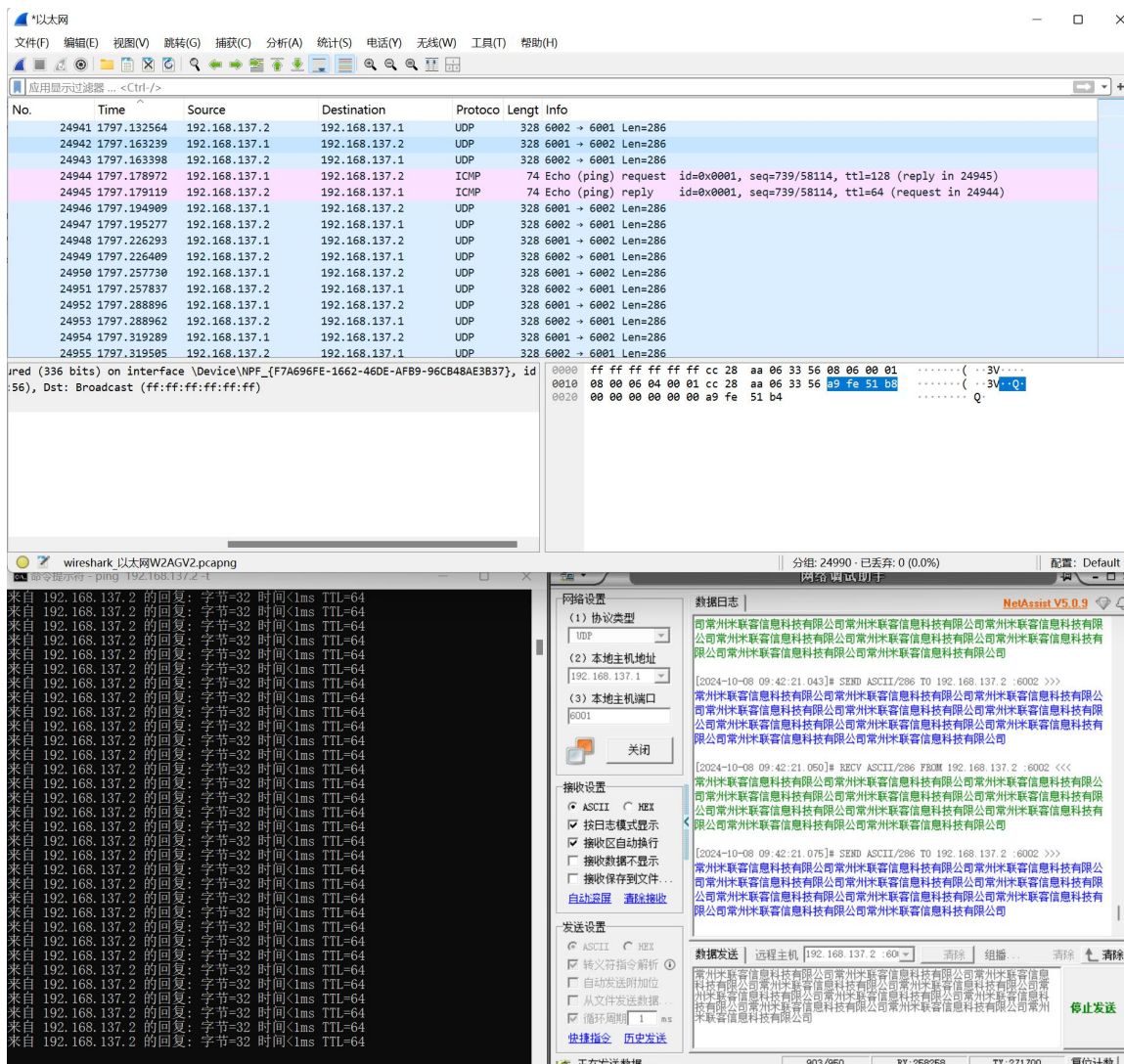
按住键盘上的 control + c 停止发送，如图。



通过 Wireshark 抓取到的 ICMP 请求包如图，其中 type 字段数据为 8'h08，code 字段数据为 8'h00，表示主动请求。



FPGA 接收到 PC 发送的 ICMP 请求包，会回复一个 ICMP 应答包，通过 wireshark 抓取到的 ICMP 应答包如图所示，其中 type 字段数据为 8' h00，code 字段数据为 8' h00，表示回显应答。

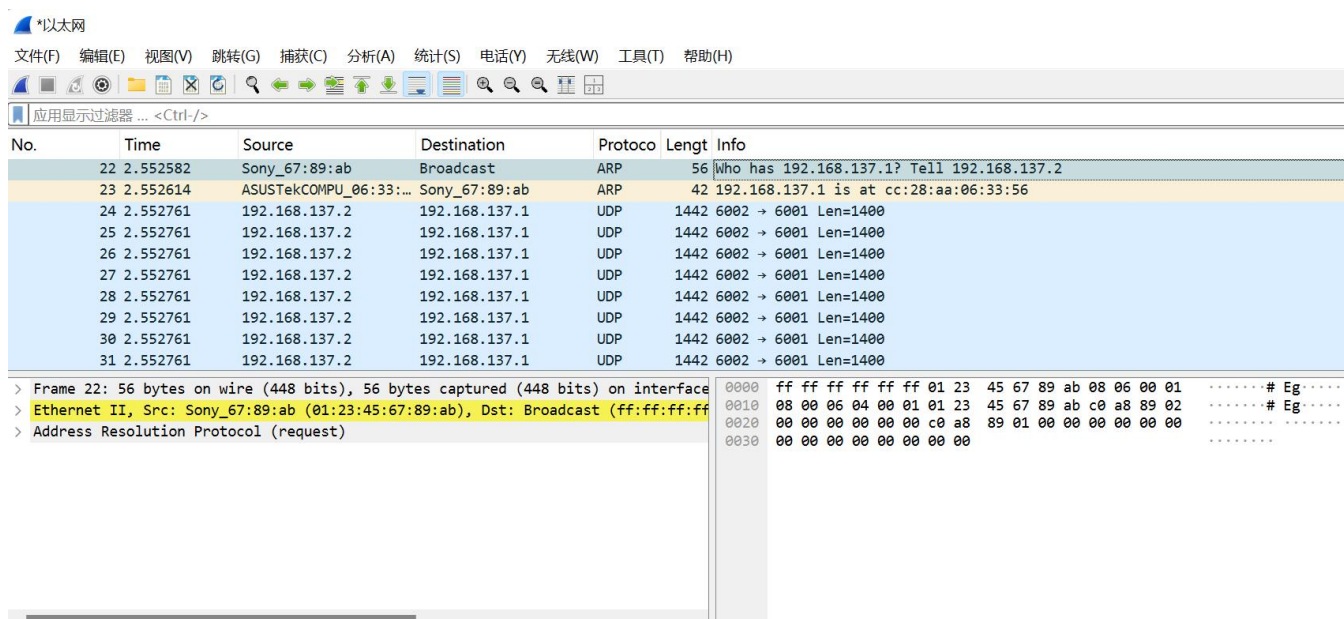


5.4 FPGA 主动发送验证

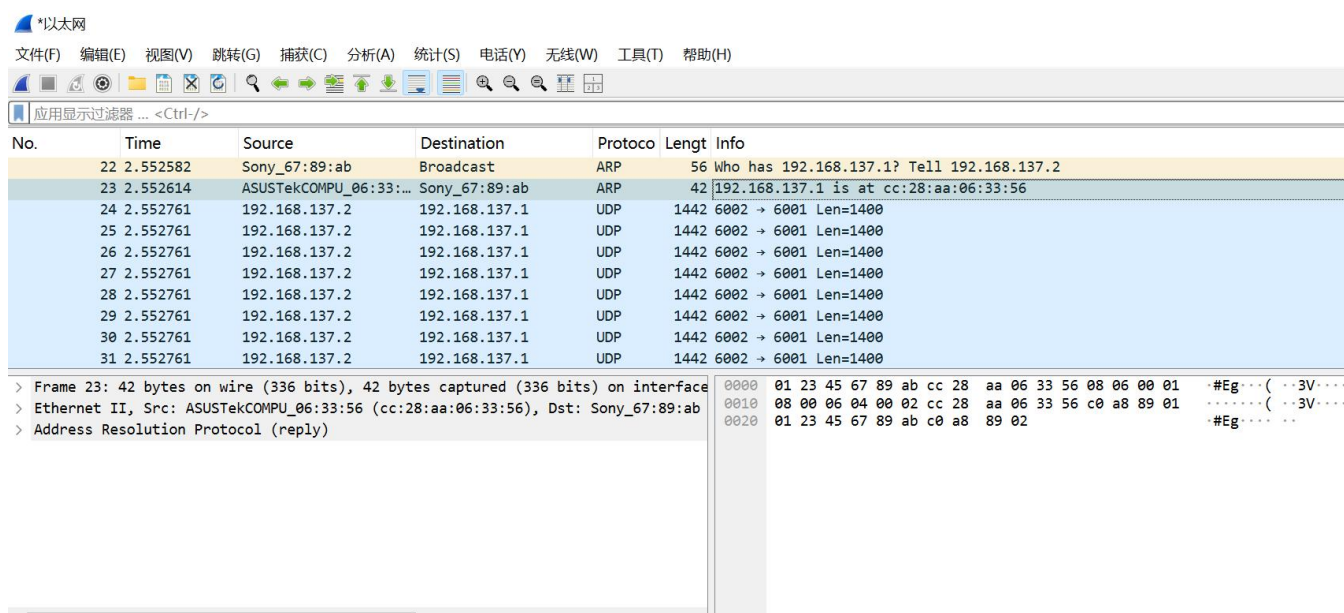
修改顶层文件，每隔一定的时间，FPGA 主动发送一帧数据给 PC，FPGA 会先发送 ARP 广播给 PC，PC 收到广播后，将自己的 MAC 地址通过 ARP 应答包发送给 FPGA，FPGA 将 PC 的 MAC 地址存入 cache 中，开始对 PC 发送数据帧。PC 端通过科来数据包生成器发送 PAUSE 帧给 FPGA，以实现流量控制效果。

5.4.1 FPGA 发送 ARP 主动请求

将程序下载进 FPGA 中，FPGA 会发送 ARP 广播包给 PC，wireshark 抓取的 ARP 广播内容如图所示。



PC 收到广播后，将自己的 MAC 地址打包成 ARP 应答包发送给 FPGA，wireshark 抓取的 ARP 应答包的内容如图所示。之后 FPGA 开始发送数据帧到 PC。



PHY 芯片的一些功能需要通过引脚的硬件连接来配置，YT8531 的引脚功能如下图所示。

No.	Name	Type	Description
22	PHYAD0	O/LI/PD	PHYAD[2:0]. PHY address configuration.
27	PHYAD1	O/LI/PD	
26	PHYAD2	O/LI/PD	
23	PLLOFF	O/LI/PD	In sleep mode, PLL off configuration.
24	TXDLY	O/LI/PD	RGMII Transmit clock timing control. Pull up to add delay to TXC for TXD latching.
25	RXDLY	O/LI/PU	RGMII receiver clock timing control. Pull-up to add 2ns delay on RX_CLK when RX_CLK is 125MHz or, to add 8ns delay on RX_CLK when RX_CLK is 25MHz/2.5MHz, which shall be used to latch RXD.
32	CFG_EXT	O/LI/PU	I/O Pad. External Power Source Mode Configuration. Pull up to use the external power source for the I/O pad. Pull down to use the integrated LDO to transform the desired voltage for the I/O pad.
33	CFG_LDO0	O/LI/PU	CFG_LDO[1:0]. When pulling down CFG_EXT pin, CFG_LDO[1:0] represent internal LDO output voltage setting for I/O pad:
34	CFG_LDO1	O/LI/PD	2'b00: Reserved 2'b01: 2.5V 2'b10 or 2'b11: 1.8V When pulling up CFG_EXT pin, CFG_LDO[1:0] stand for input voltage selection of external power for I/O pad: 2'b00: 3.3V 2'b01: 2.5V 2'b10 or 2'b11: 1.8V

PHYAD 引脚控制 PHY 芯片的物理地址，与 IIC 总线类似，一组 MDIO 总线可以连接多个设备，每个设备都需要设置独立的地址，主机利用这个地址对不同的设备进行访问。通过 4.7 千欧的电阻对 TXDLY 和 RXDLY 进行上拉或下拉，可以控制 PHY 芯片是否对发送时钟和接收时钟进行延时，对 1000Mbps 速率，上拉会给时钟端 2ns 延时，以让时钟的边沿处于数据的中心，更容易采集到稳定的数据。CFG_EXT 引脚控制是否开启芯片内部的 LDO 电源供电，CFG_LDO0 和 CFG_LDO1 引脚控制 IO 电压标准。

一般 PHY 芯片仅通过硬件连接配置即可正常使用，若还需要修改其它功能如速率、半双工等，需要使用软件配置的方法，即通过 MDIO 接口配置 PHY 寄存器。PHY 寄存器的地址空间为 5 位，802.3 协议中规定了地址为 0-15 的寄存器功能，其余寄存器可由厂商自定义。寄存器 0 为 PHY 控制寄存器，该寄存器的功能如下图所示。

Bit	Symbol	Access	Default	Description
15	Reset	RW SC	0x0	PHY Software Reset. Writing 1 to this bit causes immediate PHY reset. Once the operation is done, this bit is cleared automatically. 0: Normal operation 1: PHY reset
14	Loopback	RW SWC	0x0	Internal loopback control 1'b0: disable loopback 1'b1: enable loopback
13	Speed_Selection(LSB)	RW	0x0	LSB of speed_selection[1:0]. Link speed can be selected via either the Auto-Negotiation process, or manual speed selection speed_selection[1:0]. Speed_selection[1:0] is valid when Auto-Negotiation is disabled by clearing bit 0.12 to zero. Bit6 bit13 1 1 = Reserved 1 0 = 1000Mb/s 0 1 = 100Mb/s 0 0 = 10Mb/s
12	Autoneg_En	RW	0x1	1: to enable auto-negotiation; 0: auto-negotiation is disabled.
11	Power_down	RW SWC	0x0	1 = Power down 0 = Normal operation When the port is switched from power down to normal operation, software reset and Auto-Negotiation are performed even bit[15] RESET and bit[9] RESTART_AUTO_NEGOTIATION are not set by the user.
10	Isolate	RW SWC	0x0	Isolate phy from RGMII. 1'b0: Normal mode 1'b1: Isolate mode
9	Re_Autoneg	RW SC SWS	0x0	Auto-Negotiation automatically restarts after hardware or software reset regardless of bit[9] RESTART. 1 = Restart Auto-Negotiation Process 0 = Normal operation
8	Duplex_Mode	RW	0x1	The duplex mode can be selected via either the Auto-Negotiation process or manual duplex selection. Manual duplex selection is allowed when Auto-Negotiation is disabled by setting bit[12] AUTO_NEGOTIATION to 0. 1 = Full Duplex 0 = Half Duplex
7	Collision_Test	RW SWC	0x0	Setting this bit to 1 makes the COL signal
				asserted whenever the TX_EN signal is asserted. 1 = Enable COL signal test 0 = Disable COL signal test
6	Speed_Selection(MSB)	RW	0x1	See bit13.
5:0	Reserved	RO	0x0	Reserved. Write as 0, ignore on read

Reset: 软件复位, 将位 15 设置为 1 来复位 PHY 芯片, 复位过程中该位保持为 1, 复位完成后自动清零, 复位完成前 PHY 芯片不接受写入寄存器的操作。

Loopback: 位 14 默认为 0, 当设置为 1 时 PHY 芯片为回环模式, 从 MAC 发送过来的数据会通过 PCS 端直接回环到 MAC 接收端, 通常应用于故障诊断中。

Autoneg_En: 位 12 为自动协商使能, 设置为 1 时端口的工作模式通过和对端进行自动协商来确定, 设置为 0 时通过其它位的相应配置来确定。

Speed_Selection: 位 13 和位 6 联合控制端口的速率, 前提是自动协商关闭, 该设置才起作用。位 13 为低位, 位 6 为高位, Speed_Selection 设置为 2' b00 时, 端口速率为 10Mbps, 设置为 2' b01 时, 端口速率为 100Mbps, 设置为 2' b10 时, 端口速率为 1000Mbps, 2' b11 保留。

Duplex_Mode: 位 8 控制端口是全双工模式还是半双工模式, 需要自动协商关闭该设置才起作用。

Power_Down: 位 11 设置为 1 时, PHY 芯片掉电, 当从掉电中恢复时, 会进行软件复位和自动协商。

通过配置寄存器 0 就能将 PHY 配置成我们想要的工作状态, 其它寄存器的功能可以在手册中查看。

2 MDIO 接口设计

2.1 MDIO 时序介绍

MDIO 接口在协议 802.3 中定义，主要用于以太网的 MAC 和 PHY 之间，MAC 层器件通过 MDIO 配置 PHY 芯片的工作模式或者获取当前的工作状态。MDIO 又称为 SMI (Serial Management Interface，串行管理接口)，由 MDIO 数据线和 MDC 时钟线两条信号线组成，是串行半双工的数据接口，其接口通信协议规范如下图所示。

	Management Frame Fields							
	Preamble	ST	OP	PHYAD	REGAD	TA	DATA	IDLE
Read	1...1	01	10	AAAAA	RRRRR	Z0	DDDDDDDDDDDDDDDD	Z
Write	1...1	01	01	AAAAA	RRRRR	10	DDDDDDDDDDDDDDDD	Z

Preamble: 32 位前导码，连续发送 32 位高电平用于 PHY 芯片同步。

ST: 2' b01 作为数据开始的标志。

OP: 2 位操作码，2' b10 时表示读操作，2' b01 时表示写操作。

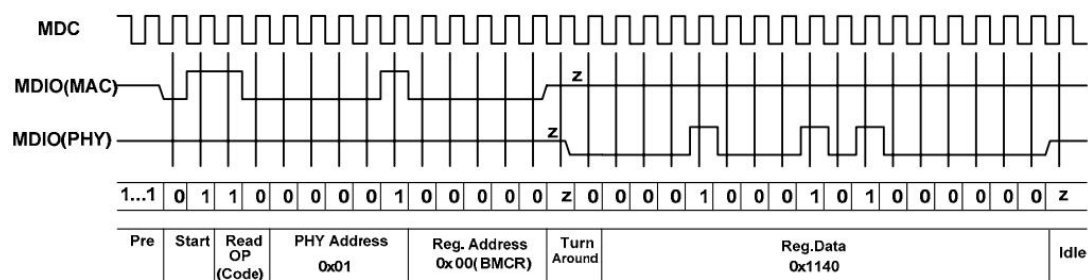
PHYAD: PHY 硬件地址，高 2 位默认为 2' b00，低 3 位由 PHY 芯片引脚的上下拉控制，通过该字段决定和哪个 PHY 芯片通信。

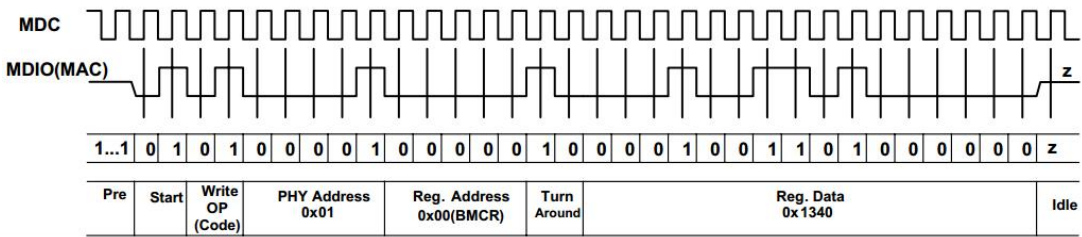
REGAD: 寄存器地址，用于设置该次操作的寄存器对象。

TA: 空闲时间，防止总线上产生竞争，在写操作时，FPGA 驱动信号线传输 2' b10。读操作时，FPGA 释放总线，将 MDIO 驱动成高阻态，在传输第一个比特时，PHY 芯片也驱动信号线为高阻态，传输第二个比特时，PHY 芯片接过总线控制权，将信号线驱动为低电平。

DATA: 16 位数据，写操作时由 FPGA 驱动，读操作时由 PHY 芯片驱动。

以 PHY 芯片硬件地址为 0x01 为例，MDIO 接口的读写时序如下图所示。

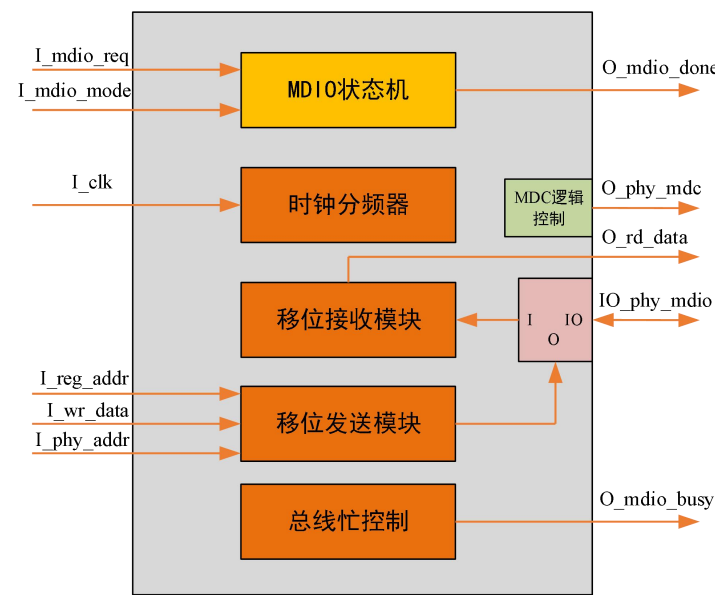




MDC 时钟由 FPGA 侧驱动，时钟下降沿更新数据，上升沿采样数据。当总线处于空闲状态或一次读写操作完成时，FPGA 和 PHY 芯片都必须释放总线，即将 MDIO 信号线驱动为高阻态。

2.2 MDIO 程序设计

MDIO 驱动器主要包含 MDIO 收发数据状态机、MDC 时钟分频器、移位发送模块、移位接收模块、总线忙控制模块。MDIO 和 MDC 的输出逻辑和时序通过状态机控制。



接口信号功能如下：

IO_phy_mdio 为 MDIO 双向数据线。

O_phy_mdc 为 MDC 时钟线。

I_reg_addr 为操作的寄存器地址。

I_phy_addr 为 PHY 的物理地址。

I_wr_data 为写入寄存器的数据。

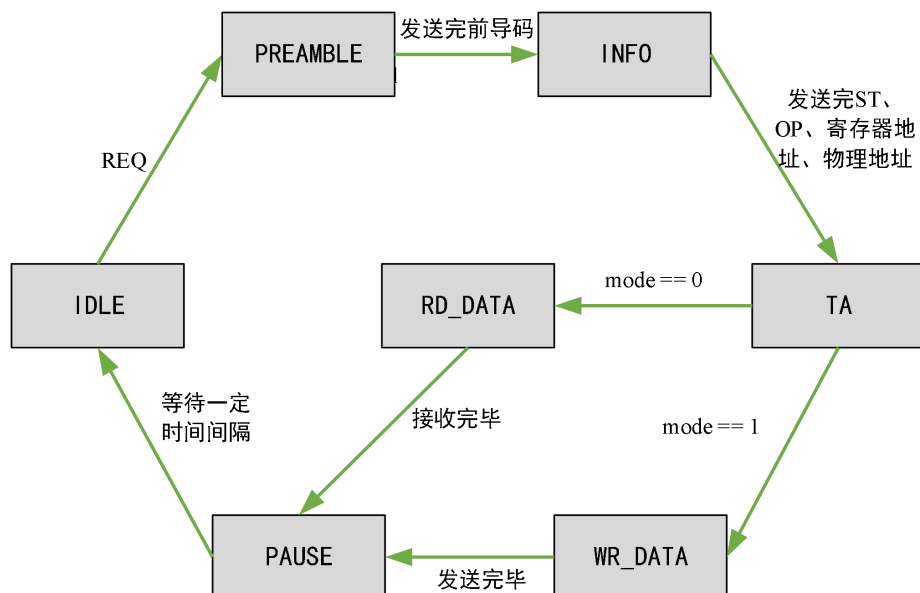
O_rd_data 为从寄存器读出的数据。

I_mdio_mode 为总线的操作模式，置 1 时为写操作，置 0 时为读操作。

I_mdio_req 为握手请求信号。

O_mdio_busy 为握手响应信号，当空闲状态接收到请求时，该信号拉高，直到一次操作完成后，该信号拉低，等待下一次握手请求信号。

MDIO 控制器中的状态机跳转图如下图所示。



IDLE: 空闲状态, 状态机处于空闲状态时, 将总线置为高阻态, 当接收到 REQ 请求时, 开始一次操作, 进入 PREAMBLE 状态。

PREAMBLE: 发送前导码, 发送 32bit 的逻辑 1, 用于同步, 发送完成后进入 INFO 状态

INFO: 发送开始标志、操作码、物理地址、寄存器地址共 14bit 数据, 发送完成后进入 TA 状态。

TA: 2 个时钟周期用来转换总线控制权。

RD_DATA: 将总线上的串行数据转成 16bit 并行数据。

WR_DATA: 将需要写入寄存器的数据由并行转成串行输出。

PAUSE: 一次操作完成后暂停一段时间间隔, 才能允许下一次操作。

```

module uimdio_ctrl#(
    parameter DIV_NUM = 25
)
(
    input wire I_clk, //主时钟
    input wire I_rstn, //复位, 低有效
    input wire [4:0] I_phy_addr, //硬件地址
    input wire [4:0] I_reg_addr, //寄存器地址
    input wire I_mdio_req, //操作请求
    input wire I_mdio_mode, //操作模式, 0write, 1read
  )
  
```

```

    input  wire    [15:0]      I_wr_data      ,//写寄存器数据
    output reg     [15:0]      O_rd_data      ,//读寄存器数据
    output wire                    O_mdio_busy ,//操作忙标志
    output reg                    O_mdio_done ,//读完成标志
    output wire                    O_phy_mdc   ,//MDIO 时钟线
    inout  wire                    IO_phy_mdio //MDIO 数据线
);
localparam IDLE      = 0;//空闲
localparam PREAMBLE  = 1;//前导码
localparam INFO      = 2;//START + OP + PHYADDR + REGADDR
localparam TA        = 3;//控制权转换
localparam WR_DATA    = 4;//写寄存器
localparam RD_DATA    = 5;//读寄存器
localparam PAUSE      = 6;//两次操作间隔
reg [2:0] cur_st      ;
reg [2:0] nxt_st      ;
reg [5:0] st_cnt      ;
reg          div_clk   ;
reg [15:0] div_cnt     ;
reg          mdio_out  ;
reg          mdio_en   ;
assign O_mdio_busy = ~(cur_st == IDLE);
assign O_phy_mdc   = ~div_clk;//时钟反转，数据在时钟的下降沿跳变
assign IO_phy_mdio = mdio_en ? mdio_out : 1'bz;
//时钟分频计数器
always@(posedge I_clk or negedge I_rstn) begin
    if(!I_rstn) begin
        div_cnt <= 0;
        div_clk <= 0;
    end
    else if(div_cnt < DIV_NUM - 1) begin
        div_cnt <= div_cnt + 1;
        div_clk <= div_clk;
    end
    else begin
        div_cnt <= 0;
        div_clk <= ~div_clk;
    end
end
//状态机计数器
always@(posedge div_clk or negedge I_rstn) begin
    if(!I_rstn)
        st_cnt <= 0;
    else if(nxt_st != cur_st)
        st_cnt <= 0;
    else
        st_cnt <= st_cnt + 1;
end
end

```

//三段式状态机

```
always@(posedge div_clk or negedge I_rstn) begin
    if(!I_rstn)
        cur_st    <=    IDLE;
    else
        cur_st    <=    nxt_st;
end
always@(*) begin
    if(!I_rstn)
        nxt_st    <=    IDLE;
    else begin
        case(cur_st)
            IDLE:begin
                if(I_mdio_req)
                    nxt_st    <=    PREAMBLE;
                else
                    nxt_st    <=    IDLE;
            end
            PREAMBLE:begin
                if(st_cnt == 'd31)
                    nxt_st    <=    INFO;
                else
                    nxt_st    <=    PREAMBLE;
            end
            INFO:begin
                if(st_cnt == 'd13)
                    nxt_st    <=    TA;
                else
                    nxt_st    <=    INFO;
            end
            TA:begin
                if(st_cnt == 'd1)
                    if(I_mdio_mode)
                        nxt_st    <=    WR_DATA;
                    else
                        nxt_st    <=    RD_DATA;
                else
                    nxt_st    <=    TA;
            end
            WR_DATA:begin
                if(st_cnt == 'd15)
                    nxt_st    <=    PAUSE;
                else
                    nxt_st    <=    WR_DATA;
            end
            RD_DATA:begin
                if(st_cnt == 'd16)
                    nxt_st    <=    PAUSE;
```

```

        else
            nxt_st    <=  RD_DATA;

        end

    PAUSE:begin
        if(st_cnt == 'd31)
            nxt_st    <=  IDLE;
        else
            nxt_st    <=  PAUSE;
        end

        default:begin
            nxt_st    <=  IDLE;
        end
    endcase
end

end

always@(posedge div_clk or negedge I_rstn) begin
    if(!I_rstn)
        mdio_out    <=  0;
    else if(cur_st == PREAMBLE)
        mdio_out    <=  1;
    else if(cur_st == INFO) begin
        case(st_cnt)
            'd0      :mdio_out    <=  0;
            'd1      :mdio_out    <=  1;
            'd2      :mdio_out    <=  I_mdio_mode ? 0 : 1;
            'd3      :mdio_out    <=  I_mdio_mode ? 1 : 0;
            'd4      :mdio_out    <=  I_phy_addr[4];
            'd5      :mdio_out    <=  I_phy_addr[3];
            'd6      :mdio_out    <=  I_phy_addr[2];
            'd7      :mdio_out    <=  I_phy_addr[1];
            'd8      :mdio_out    <=  I_phy_addr[0];
            'd9      :mdio_out    <=  I_reg_addr[4];
            'd10     :mdio_out    <=  I_reg_addr[3];
            'd11     :mdio_out    <=  I_reg_addr[2];
            'd12     :mdio_out    <=  I_reg_addr[1];
            'd13     :mdio_out    <=  I_reg_addr[0];
        endcase
    end
    else if(cur_st == TA) begin
        case(st_cnt)
            'd0      :mdio_out    <=  I_mdio_mode ? 1 : 1'bz;
            'd1      :mdio_out    <=  I_mdio_mode ? 0 : 1'bz;
        endcase
    end
    else if(cur_st == WR_DATA)
        mdio_out    <=  I_mdio_mode ? I_wr_data[15 - st_cnt] : 0;
    else
        mdio_out    <=  0;

```

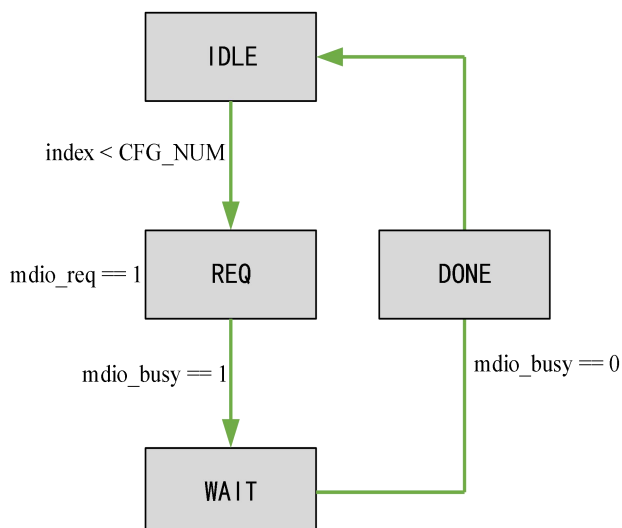
```

end
always@(posedge div_clk or negedge I_rstn) begin
    if(!I_rstn)
        mdio_en <= 0;
    else if(cur_st == IDLE || cur_st == PAUSE)
        mdio_en <= 0;
    else if(cur_st == TA)
        mdio_en <= I_mdio_mode ? 1 : 0;
    else if(cur_st == PREAMBLE)
        mdio_en <= 1;
    else
        mdio_en <= mdio_en;
end
always@(posedge div_clk or negedge I_rstn) begin
    if(!I_rstn)
        O_mdio_done <= 0;
    else if(cur_st == RD_DATA && st_cnt == 'd16)
        O_mdio_done <= 1;
    else
        O_mdio_done <= 0;
end
//MDIO 读数据, 使用 mdc 时钟采样
always@(posedge O_phy_mdc or negedge I_rstn) begin
    if(!I_rstn)
        O_rd_data <= 0;
    else if(cur_st == RD_DATA && st_cnt >= 1)
        O_rd_data <= {O_rd_data[14:0], IO_phy_mdio};
    else
        O_rd_data <= O_rd_data;
end
endmodule

```

为了方便操作 MDIO 驱动器的读写, 需要增加一级模块封装, 该模块完成对请求信号及操作类型的控制, 来对

PHY 寄存器进行配置。在 PHY 寄存器配置完成后, 通过按键可以进行读操作, 将寄存器的配置信息读取出来。



在上电后等待一定的时间间隔，释放复位，通过握手信号控制驱动器的工作状态，当 busy 信号为 0 时，拉高 req 信号，直到 busy 信号为高，代表驱动器响应成功，直到 busy 再次拉低，进行下一次操作。通过计数器控制写操作执行的次数，当达到指定次数时，寄存器配置完成，拉高 cfgdone 信号，表示不再继续进行写操作。

第一次写操作在 bit11 的位置写入 1，让 PHY 芯片掉电，第二次写操作再写入速度模式和全双工模式。

```

module uiphy_cfg#(
    parameter SPEED = 2'b01,
    parameter CLK_FREQ = 32'd25_000_000
)
(
    input wire I_clk,
    input wire I_rstn,
    input wire I_keycap,
    output wire O_phyrst,
    output wire O_phy_mdc,
    inout wire IO_phy_mdio
);
localparam CFG_NUM = 3;
localparam IDLE = 0;
localparam REQ = 1;
localparam DONE = 2;
reg [23:0] cnt;
reg [5:0] index;
reg [23:0] gap_cnt;
reg [1:0] state;
reg cfgdone;
reg mdio_req;
reg mode;
wire [4:0] reg_addr;
wire [15:0] rd_data;
reg [15:0] wr_data;
wire mdio_busy;
wire mdio_done;
wire start_lock;
assign start_lock = cnt == 1250_000;
assign reg_addr = cfgdone ? 5'h0 : 5'h0;
always@(posedge I_clk or negedge O_phyrst) begin
    if(!O_phyrst)
        cnt <= 0;
    else if(cnt < 1250_000)
        cnt <= cnt + 1'b1;
    else
        cnt <= cnt;
end
always@(posedge I_clk or negedge O_phyrst) begin

```

```

    if(!0_phyrst)
        state    <=  IDLE;
    else if(start_lock) begin
        case(state)
            IDLE:begin
                if(cfgdone == IDLE)
                    state    <=  REQ;
                else if(cfgdone && I_keycap)
                    state    <=  REQ;
                else
                    state    <=  IDLE;
            end
            REQ:begin
                if(mdio_busy)
                    state    <=  DONE;
                else
                    state    <=  REQ;
            end
            DONE:begin
                if(!mdio_busy)
                    state    <=  IDLE;
                else
                    state    <=  DONE;
            end
            default:begin
                state    <=  IDLE;
            end
        endcase
    end
end

always@(posedge I_clk or negedge 0_phyrst) begin
    if(!0_phyrst)
        mdio_req    <=  0;
    else if(mdio_req && mdio_busy)
        mdio_req    <=  0;
    else if(state == REQ)
        mdio_req    <=  1;
    else
        mdio_req    <=  0;
end

always@(posedge I_clk or negedge 0_phyrst) begin
    if(!0_phyrst)
        mode    <=  0;
    else if(state == REQ && cfgdone == 1)
        mode    <=  0;
    else if(state == REQ)
        mode    <=  1;
    else

```

```

        mode    <=    mode;
end
always@(posedge I_clk or negedge O_phyrst) begin
    if(!O_phyrst)
        cfgdone <=    0;
    else if(index == CFG_NUM - 1 && !mdio_busy)
        cfgdone <=    1;
end
always@(posedge I_clk or negedge O_phyrst) begin
    if(!O_phyrst)
        index    <=    0;
    else if(state == DONE && !mdio_busy && !cfgdone)
        index    <=    index + 1;
    else
        index    <=    index;
end
always@(*) begin
    case(index)
        0:wr_data    <=    16'h0800;
        1:wr_data    <=    {2'b00, SPEED[0], 6'b0000_10, SPEED[1], 6'b0000_00};
        2:wr_data    <=    {2'b10, SPEED[0], 6'b0000_10, SPEED[1], 6'b0000_00};
        default:wr_data <=    16'b8000;
    endcase
end
uiphyrst#(
    .CLK_FREQ          (CLK_FREQ)
)
uiphyrst
(
    .I_CLK              (I_clk          ),
    .I_rstn             (I_rstn         ),
    .I_phyrst           (I_rstn         ),
    .O_phyrst           (O_phyrst       )
);
uimdio_ctrl #(
    .DIV_NUM            (50)
)
uimdio_ctrl
(
    .I_clk              (I_clk          ),
    .I_rstn             (O_phyrst       ),
    .I_phy_addr         (5'b00100      ),
    .I_reg_addr         (reg_addr       ),
    .I_mdio_req         (mdio_req       ),
    .I_mdio_mode        (mode           ),
    .I_wr_data          (wr_data        ),
    .O_rd_data          (rd_data        ),
    .O_mdio_busy        (mdio_busy     ),

```

```

        .O_mdio_done      (mdio_done      ),
        .O_phy_mdc       (O_phy_mdc      ),
        .IO_phy_mdio     (IO_phy_mdio     )
    );
endmodule

```

仿真测试源码如下：

```

`timescale 1ns/1ps
module cfg_tb;

reg clk,rst;
wire      phyrst,phy_mdc,phy_mdio;
pullup(phy_mdio);
uiphy_cfg #(
    .SPEED(2'b01),
    .CLK_FREQ(32'd25_000_000)
) inst_uiphy_cfg (
    .I_clk      (clk),
    .I_rstn     (~rst),
    .O_phyrst   (phyrst),
    .O_phy_mdc  (phy_mdc),
    .IO_phy_mdio (phy_mdio)
);
always #20 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    #200;
    rst = 0;
end

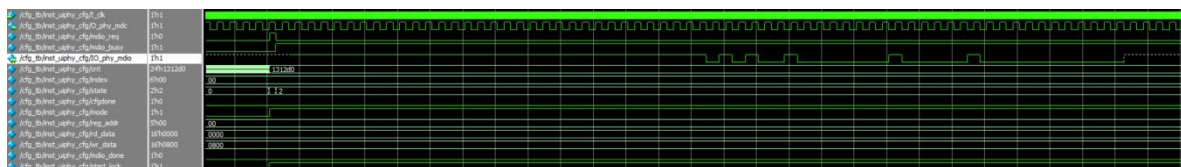
endmodule

```

通过 modelsim 进行仿真验证，抓取 mdio_req 为高电平的时间段，可见 mdio_req 拉高后，MDIO 驱动器将 mdio_busy 也拉高进行响应，当 mdio_busy 拉高时，mdio_req 立即拉低，握手成功。



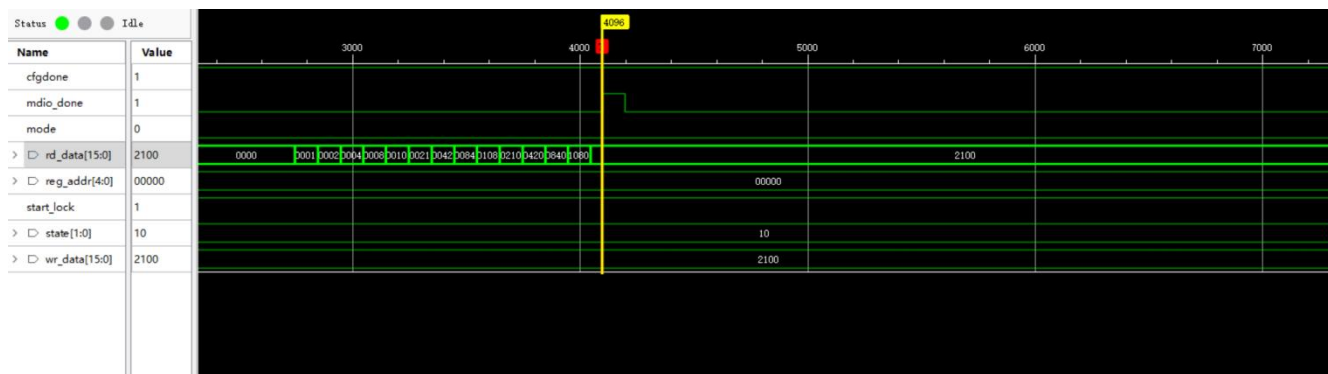
mdio 信号线上的数据如下图所示，空闲时总线为高阻态，在 mdio_busy 拉高后，进行写操作，总线先发送 32bit 的逻辑 1，接着按顺序发送开始标志 2' b01、操作码 2' b01、物理地址 5' b00100、寄存器地址 5' b00000、TA2' b10 共 16bit 数据，然后发送数据 16' h0800，全部发送完成后，总线置为高阻态。



通过 MDIO 将 PHY 芯片的工作模式设置为全双工、100Mbps 速率，在 PC 中的“网络和 Internet”设置中查看以太网的配置信息，可以看到链接速度为 100Mbps，证明 PHY 芯片配置成功。



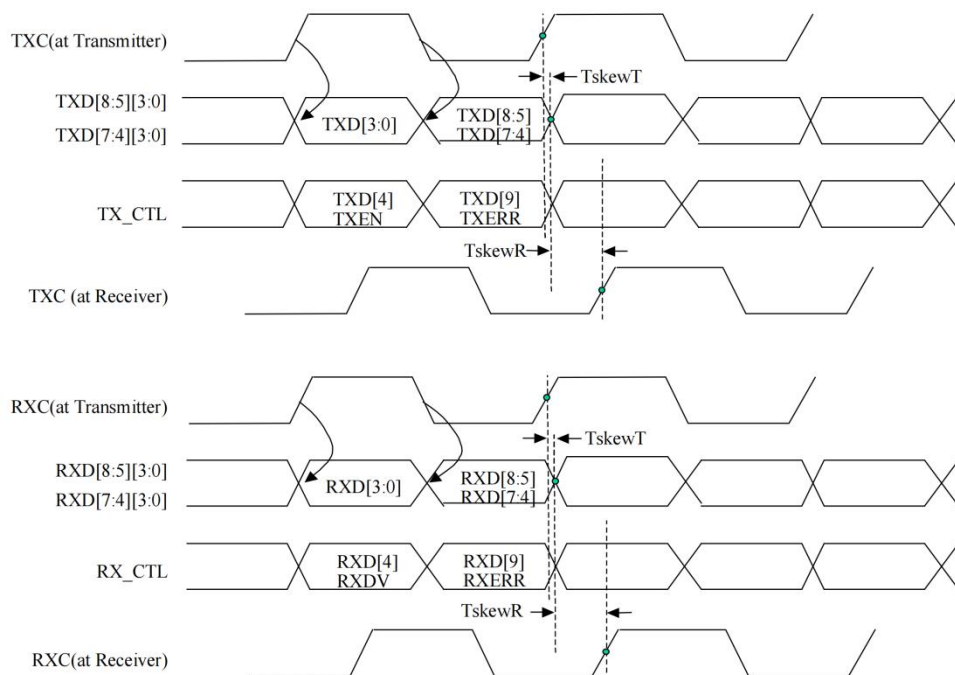
然后在程序中设置按键控制读寄存器 0 操作，通过 chipwatcher 抓取波形，上板验证效果如下图所示。读出寄存器 0 的配置信息为 16'h2100，即为写入的配置信息。



3 RGMII 接口设计

3.1 RGMII 接口时序

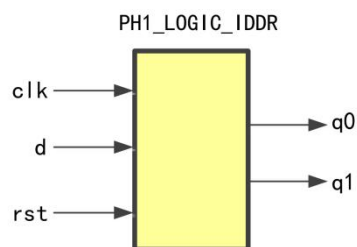
在 2.2 节我们对 RGMII 接口的功能及引脚做了介绍，RGMII 接口的时序如下图所示。



在普通模式下，数据在时钟的边沿变化，如果不做额外处理，数据无法被稳定采样，所以 PHY 芯片还提供了延迟模式来保证数据在稳定的区间内被采样。发送端或接受端被设置成延时模式时，时钟在 PHY 芯片内部会添加 2ns 的延时，让时钟的边沿对齐数据有效区间的中心。延时模式通过 PHY 芯片的引脚上下拉控制。

RGMII 接口是双沿采样的，每个时钟周期传输一个字节的的数据。传输数据时，在时钟上升沿传输一个字节的低 4bit，在时钟下降沿传输一个字的高 4bit，所以千兆速率下 RGMII 接口的采样时钟为 125MHz，而接口速率为 1000Mbps。FPGA 内部逻辑难以处理双沿数据，需要在接收引脚或发送引脚的 IOB 上将双沿数据转换成单沿数据或将单沿数据转换成双沿数据，IOB 中的 IDDR 和 ODDR 可以用来实现这些功能。

PH1_LOGIC_IDDR 是安路 PH1 系列 FPGA IO 上将双沿采样时钟数据变换为单沿数据的单元。具有数据对齐的双速率 D 触发器，并具有同步/异步复位功能。可以将输入的 DDR 数据转为 1 位 SDR 数据同步到内部时钟网络。



该单元的引脚功能如下：

clk：采样时钟。

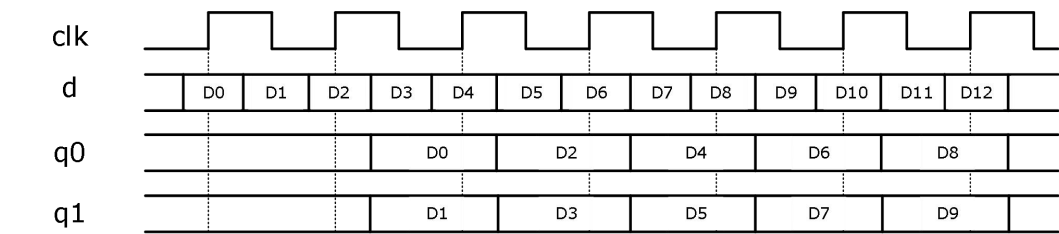
d：IO 上输入的 DDR 数据。

rst：复位信号，高有效。

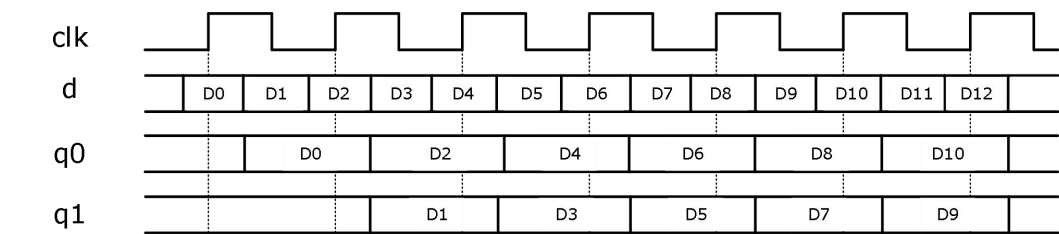
q0：1 位上升沿数据输出。

q1：1 位下降沿数据输出。

该单元可以配置成 PIPED 模式或 NONE 模式，PIPED 模式下时序如下图所示。

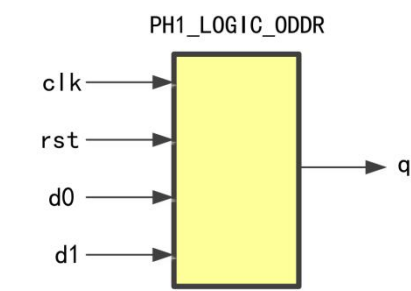


NONE 模式下时序图如下图所示。



工作在 PIPED 模式下时，q0 和 q1 对应一个完整字节的数据，所以在对 RGMII 接口采样时使用 PIPED 模式。

PH1_LOGIC_ODDR 时 FPGA IO 上将内部单沿数据变换为双沿数据后输出的单元，具有同步/异步复位功能。可以将数据双倍率输出到 IO 口上。



该单元的引脚功能如下：

clk：同步时钟。

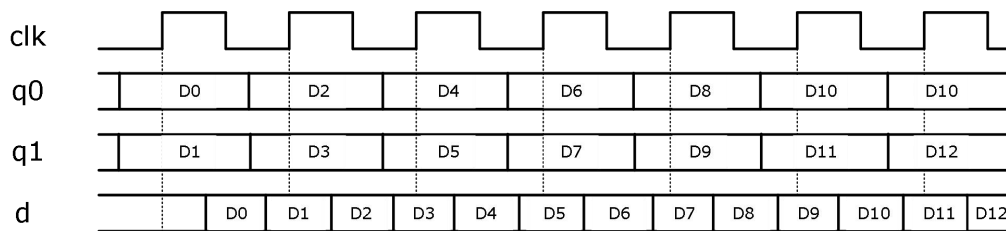
d0：1 位上升沿输入数据。

d1：1 位下降沿输入数据。

rst：复位信号，高有效。

q：1 位 DDR 双沿输出数据。

该单元的工作时序如下图所示。



3.2 RGMII 模块设计

RGMII 接口输入输出模块 rgmii_interface 的代码如下：

```
module rgmii_interface (
// 指示当前运行速度为 10/100
input          speed_10_100,    //设置当前的运行速率
//RGMII 发送时序调整，当 PHY 接收来自 FPGA 的 TX 信号，没有使用内部 2ns 延迟情况需要在 fpga 端设置 2ns 延迟
input          gmii_tx_clk_d,
input          gmii_tx_reset_d,
// 以下端口是 RGMII 物理接口：这些端口将位于 FPGA 的引脚上
output [3:0]   rgmii_txd,
output         rgmii_tx_ctl,
output         rgmii_txc,
input [3:0]    rgmii_rxd,
input          rgmii_rx_ctl,
input          rgmii_rxc,
// 以下端口连接到 TEMAC 核 的 内部 GMII 接口模块
input          gmii_tx_reset,
input          gmii_tx_clk,      // ggmmii_tx_clk: 125mhz
input [7:0]    gmii_txd,
input          gmii_tx_en,
input          gmii_tx_er,
input          gmii_rx_reset,
output         gmii_rx_clk,      //output: 125mhz(1gbps) 25mhz(100mbps) 2.5mhz(10mbps)
output reg [7:0] gmii_rxd,
output reg     gmii_rx_dv,
output reg     gmii_rx_er,
output         gmii_crs,
output         gmii_col,
// 以下信号为 RGMII 状态信号
output reg     link_status,
output reg [1:0] clock_speed,
output reg     duplex_status
);
//-----
// 模块 内部 信号
//-----
reg [3:0] gmii_txd_falling;      // gmii_txd 信号在 gmii_tx_clk 的下降沿锁存。
```

```

wire            rgmii_txc_odelay;                // RGMII 接收器时钟 ODDR 输出.
wire            rgmii_tx_ctl_odelay;            // RGMII 控制信号 ODDR 输出.
wire    [3:0]   rgmii_txd_odelay;                // RGMII 数据 ODDR 输出.
wire            rgmii_tx_ctl_int;                // 内部 RGMII 传输控制信号.
wire            rgmii_rx_ctl_delay;
wire    [3:0]   rgmii_rxd_delay;
wire            rgmii_rx_ctl_reg;                // 内部 RGMII 接收器控制信号.
reg            tx_en_to_ddr;
wire            gmii_rx_dv_reg;
wire            gmii_rx_er_reg;
wire    [7:0]   gmii_rxd_reg;
wire            inband_ce;                        //RGMII 带内状态寄存器 使能输出信号
wire            rgmii_rxc_int;
//=====
// RGMII 发送逻辑
//=====
//-----
// RGMII 发送器时钟管理: rgmii_txc
//-----
// 产生 rgmii_txc 时钟.
PH1_LOGIC_ODDR    rgmii_txc_ddr
(
    .q            (rgmii_txc_odelay), //output 125mhz (1gbps) 25mhz(100mbps) 2.5mhz(10mbps)
    .clk          (gmii_tx_clk_d),
    .d0           (1'b1),
    .d1           (1'b0),
    .rst          (gmii_tx_reset_d)
);
assign rgmii_txc = rgmii_txc_odelay;

//-----
// RGMII 发送逻辑 : rgmii_txd
//-----
// 1Gbps gmii_txd 8 位有效; rgmii_txc 双沿使能发送 8 位
// 10/100Mbps gmii_txd 仅低四位有效, rgmii_txc 双沿使能重复发送低四位
// 1Gbps 时, 125mhz 的 rgmii_txc, 上升沿发送低四位, 下降沿发送高四位。 一个时钟周期 8bit, 125*8=1Gbps
// 100Mbps 时, 25mhz 的 rgmii_txc, 上升沿发送低四位, 下降沿发送低四位。 相当于一个时钟周期只发一个 4bit,
25*4=100mbps
// 10Mbps 时同 100mbps, 数据有效位在每 byte 的低四位, 虽然一个时钟周期重复发低四位, 相当于一个时钟周期只发一个低 4 位。

always @ (speed_10_100, gmii_txd)begin
    if (speed_10_100 == 1'b0) // 1Gbps gmii_txd 8 位有效
        gmii_txd_falling    <= gmii_txd[7:4];
    else // 10/100Mbps gmii_txd 高四位无效, rgmii_txc 双沿使能发送低四位
        gmii_txd_falling    <= gmii_txd[3:0];
end
genvar i;

```

```

generate for (i=0; i<4; i=i+1)begin : txdata_out_bus
    PH1_LOGIC_ODDR    rgmii_txd_out
    (
        .q                (rgmii_txd_odelay[i]),
        .clk                (gmii_tx_clk),
        .d0                (gmii_txd[i]),
        .d1                (gmii_txd_falling[i]),
        .rst                (gmii_tx_reset)
    );
assign rgmii_txd[i] = rgmii_txd_odelay[i];
end
endgenerate

//-----
// RGMII 发送逻辑 : rgmii_tx_ctl
//-----
// 编码 rgmii_ctl 信号
assign rgmii_tx_ctl_int = gmii_tx_en ^ gmii_tx_er;
// 需要逻辑以确保 错误信号 将在整个时钟相位内, 将 tx_ctl 发送为低电平
always @(speed_10_100 or gmii_tx_en or gmii_tx_er)begin
    if (speed_10_100)
        tx_en_to_ddr = gmii_tx_en & (!gmii_tx_er );
    else
        tx_en_to_ddr = gmii_tx_en;
end

// oDDR primitive
PH1_LOGIC_ODDR    ctl_output
(
    .q                (rgmii_tx_ctl_odelay),
    .clk                (gmii_tx_clk),
    .d0                (tx_en_to_ddr),
    .d1                (rgmii_tx_ctl_int),
    .rst                (gmii_tx_reset)
);
assign rgmii_tx_ctl = rgmii_tx_ctl_odelay;
//=====
// RGMII 接收逻辑
//=====
//-----
// RGMII 接收逻辑: rgmii_rxc
//-----
PH1_PHY_IOCLK u_rgmii_rxc_int
(
    .clk_in (rgmii_rxc),
    .clk_out (rgmii_rxc_int)
);
PH1_PHY_SCLK_V2 u_rgmii_rxc
(

```

```

.ce(1'b1),
.clkin (rgmii_rxc),
.clkout (gmii_rx_clk)
);

//-----
// RGMII 接收逻辑: rgmii_rxd ----> gmii_rxd_reg
//-----

// 1Gbps gmii_rxd 8位有效; rgmii_rxc 双沿使能接收8位
// 10/100Mbps gmii_rxd 仅低四位有效, rgmii_rxc 双沿使能接收高低四位数据相同
// 1Gbps时, 125mhz的 rgmii_rxc, 上升沿接收四位数据(对应低4bit), 下降沿接收四位数据(对应高4bit)。一个时钟周期8bit, 125mhz*8=1Gbps
// 100Mbps时, 25mhz的 rgmii_rxc, 上升沿接收四位数据, 下降沿接收四位数据(高四位和低四位数据相同)。相当于单沿采样, 一个时钟周期只收一个4bit(数据有效位在每byte的低四位: 高低四位重复), 25mhz*4=100Mbps
// 10Mbps时同100Mbps, 虽然一个时钟周期上下沿重复接收相同四位数据, 相当于一个时钟周期只收一个4bit。
genvar j;
generate for (j=0; j<4; j=j+1)begin : rxdata_bus
assign rgmii_rxd_delay[j]=rgmii_rxd[j];
end
endgenerate
// Instantiate Double Data Rate Input flip-flops.
// DDR_CLK_EDGE attribute specifies output data alignment from IDDR component
genvar k;
generate for (k=0; k<4; k=k+1)begin : rxdata_in_bus
PH1_LOGIC_IDDR #(
.PIPEMODE ("PIPED")
)
rgmii_rx_data_in
(
.q0 (gmii_rxd_reg[k]),
.q1 (gmii_rxd_reg[k+4]),
.clk (rgmii_rxc_int),
.d (rgmii_rxd_delay[k]),
.rst (1'b0)
);

end
endgenerate

//-----
// RGMII 接收逻辑: rgmii_rx_ctl -----> gmii_rx_dv, gmii_rx_er
//-----

assign rgmii_rx_ctl_delay = rgmii_rx_ctl;

PH1_LOGIC_IDDR #(
.PIPEMODE ("PIPED")
)
rgmii_rx_ctl_in

```

```

(
.q0          (gmii_rx_dv_reg),
.q1          (rgmii_rx_ctl_reg),
.clk         (rgmii_rxc_int),
.d           (rgmii_rx_ctl_delay),
.rst         (1'b0)
);
// 解码 gmii_rx_er signal
assign gmii_rx_er_reg = gmii_rx_dv_reg ^ rgmii_rx_ctl_reg;
//-----
// 接收逻辑: 内部信号给到输出端口: gmii_rxd、gmii_rx_dv、gmii_rx_er、gmii_col、gmii_crs
//-----
always @ (posedge gmii_rx_clk )begin
gmii_rxd      <= gmii_rxd_reg;
gmii_rx_dv    <= gmii_rx_dv_reg;
gmii_rx_er    <= gmii_rx_er_reg;
end
// 从 RGMII 创建 GMII 格式的冲突和载波侦听信号
assign gmii_col = (gmii_tx_en | gmii_tx_er) & (gmii_rx_dv_reg | gmii_rx_er_reg);
assign gmii_crs = (gmii_tx_en | gmii_tx_er) | (gmii_rx_dv_reg | gmii_rx_er_reg);
//=====
// RGMII 状态寄存器
//=====
// 在帧间间隔期间启用带内状态寄存器
assign inband_ce = !(gmii_rx_dv_reg || gmii_rx_er_reg);
always @ (posedge gmii_rx_clk or posedge gmii_rx_reset)begin
    if (gmii_rx_reset) begin
        link_status          <= 1'b0;
        clock_speed[1:0]     <= 2'b0;
        duplex_status        <= 1'b0;
    end
    else if (inband_ce) begin
        link_status          <= gmii_rxd_reg[0];
        clock_speed[1:0]     <= gmii_rxd_reg[2:1];
        duplex_status        <= gmii_rxd_reg[3];
    end
end
endmodule

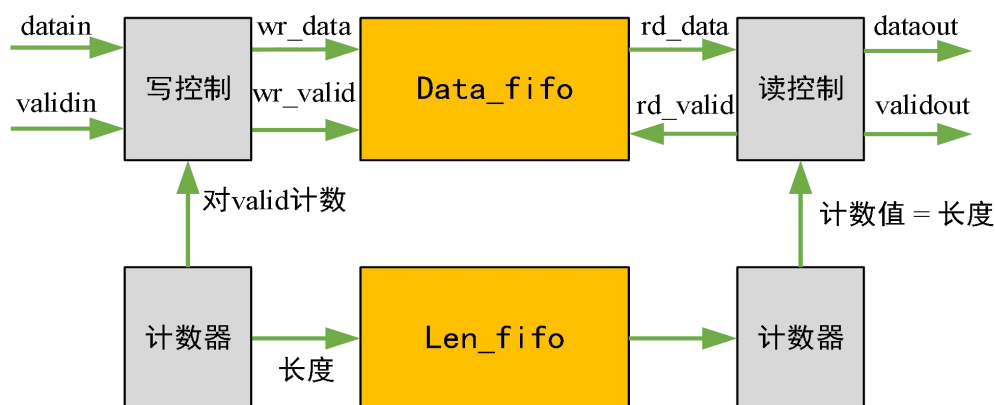
```

3.3 数据转换模块设计

rgmii_interface 模块只实现了千兆速率下的 RGMII 接口收发，而 100Mbps 和 10Mbps 速率的采样时钟分别为 25MHz 和 2.5MHz，数据单沿传输。100M 和 10M 以太网的内部逻辑时钟为 12.5MHz 和 1.25MHz，数据位宽为 8 位，这就需要额外设计一个模块进行跨时钟域处理，并且实现 8bit 数据和 4bit 数据的相互转换，称为 rgmii_cdc

模块。

rgmii_cdc 模块的设计思路与 mac 层类似，在发送端和接收端各使用两个异步 FIFO，一个用来缓存数据，即 data_fifo，一个用来缓存长度队列，即 len_fifo。对写入 data_fifo 的数据数量进行计数，当写入最后一个数据时，将计数的值也写入 len_fifo 中。当 len_fifo 为非空时，代表有一帧数据等待读出，这时候将缓存的长度信息读取出来，然后通过计数器读取出一个完的帧。



rgmii_cdc 模块的代码如下：

```
module rgmii_interface (
// 指示当前运行速度为 10/100
input          speed_10_100,    //设置当前的运行速率
//RGMII 发送时序调整，当 PHY 接收来自 FPGA 的 TX 信号，没有使用内部 2ns 延迟情况需要在 fpga 端设置 2ns 延迟
input          gmii_tx_clk_d,
input          gmii_tx_reset_d,
// 以下端口是 RGMII 物理接口：这些端口将位于 FPGA 的引脚上
output [3:0]   rgmii_txd,
output        rgmii_tx_ctl,
output        rgmii_txc,
input  [3:0]   rgmii_rxd,
input        rgmii_rx_ctl,
input        rgmii_rxc,
// 以下端口连接到 TEMAC 核 的 内部 GMII 接口模块
input        gmii_tx_reset,
input        gmii_tx_clk,      // ggmmii_tx_clk: 125mhz
input [7:0]   gmii_txd,
input        gmii_tx_en,
input        gmii_tx_er,
input        gmii_rx_reset,
output        gmii_rx_clk,     //output: 125mhz(1gbps) 25mhz(100mbps) 2.5mhz(10mbps)
output reg [7:0] gmii_rxd,
output reg    gmii_rx_dv,
output reg    gmii_rx_er,
output        gmii_crs,
```

```

output          gmii_col,
// 以下信号为 RGMII 状态信号
output reg      link_status,
output reg [1:0] clock_speed,
output reg      duplex_status
);
//-----
// 模块 内部 信号
//-----
reg      [3:0] gmii_txd_falling;           // gmii_txd 信号在 gmii_tx_clk 的下降沿锁存。
wire      rgmii_txc_odelay;                // RGMII 接收器时钟 ODDR 输出。
wire      rgmii_tx_ctl_odelay;             // RGMII 控制信号 ODDR 输出。
wire      [3:0] rgmii_txd_odelay;          // RGMII 数据 ODDR 输出。
wire      rgmii_tx_ctl_int;                // 内部 RGMII 传输控制信号。
wire      rgmii_rx_ctl_delay;
wire      [3:0] rgmii_rxd_delay;
wire      rgmii_rx_ctl_reg;                // 内部 RGMII 接收器控制信号。
reg      tx_en_to_ddr;
wire      gmii_rx_dv_reg;
wire      gmii_rx_er_reg;
wire      [7:0] gmii_rxd_reg;
wire      inband_ce;                       //RGMII 带内状态寄存器 使能输出信号
wire      rgmii_rxc_int;
//=====
// RGMII 发送逻辑
//=====
//-----
// RGMII 发送器时钟管理: rgmii_txc
//-----
// 产生 rgmii_txc 时钟.
PH1_LOGIC_ODDR   rgmii_txc_ddr
(
.q              (rgmii_txc_odelay), //output 125mhz (1gbps)  25mhz(100mbps)      2.5mhz(10mbps)
.clk            (gmii_tx_clk_d),
.d0             (1'b1),
.d1             (1'b0),
.rst            (gmii_tx_reset_d)
);
assign rgmii_txc = rgmii_txc_odelay;

//-----
// RGMII 发送逻辑 : rgmii_txd
//-----
// 1Gbps gmii_txd   8 位有效; rgmii_txc 双沿使能发送 8 位
// 10/100Mbps gmii_txd 仅低四位有效, rgmii_txc 双沿使能重复发送低四位
// 1Gbps 时, 125mhz 的 rgmii_txc, 上升沿发送低四位, 下降沿发送高四位。 一个时钟周期 8bit, 125*8=1Gbps
// 100Mbps 时, 25mhz 的 rgmii_txc, 上升沿发送低四位, 下降沿发送低四位。 相当于一个时钟周期只发一个 4bit,
25*4=100mbps

```

// 10Mbps 时同 100Mbps, 数据有效位在每 byte 的低四位, 虽然一个时钟周期重复发低四位, 相当于一个时钟周期只发一个低 4 位。

```

always @ (speed_10_100, gmii_txd)begin
    if (speed_10_100 == 1'b0) // 1Gbps gmii_txd 8 位有效
        gmii_txd_falling    <= gmii_txd[7:4];
    else // 10/100Mbps gmii_txd 高四位无效, rgmii_txc 双沿使能发送低四位
        gmii_txd_falling    <= gmii_txd[3:0];
end
genvar i;
generate for (i=0; i<4; i=i+1)begin : txdata_out_bus
    PH1_LOGIC_ODDR    rgmii_txd_out
    (
        .q                (rgmii_txd_odelay[i]),
        .clk                (gmii_tx_clk),
        .d0                (gmii_txd[i]),
        .d1                (gmii_txd_falling[i]),
        .rst                (gmii_tx_reset)
    );
assign rgmii_txd[i] = rgmii_txd_odelay[i];
end
endgenerate
//-----
// RGMII 发送逻辑 : rgmii_tx_ctl
//-----
// 编码 rgmii_ctl 信号
assign rgmii_tx_ctl_int = gmii_tx_en ^ gmii_tx_er;
// 需要逻辑以确保 错误信号 将在整个时钟相位内, 将 tx_ctl 发送为低电平
always @(speed_10_100 or gmii_tx_en or gmii_tx_er)begin
    if (speed_10_100)
        tx_en_to_ddr = gmii_tx_en & (!gmii_tx_er );
    else
        tx_en_to_ddr = gmii_tx_en;
end

// oDDR primitive
PH1_LOGIC_ODDR    ctl_output
(
    .q                (rgmii_tx_ctl_odelay),
    .clk                (gmii_tx_clk),
    .d0                (tx_en_to_ddr),
    .d1                (rgmii_tx_ctl_int),
    .rst                (gmii_tx_reset)
);
assign rgmii_tx_ctl = rgmii_tx_ctl_odelay;
//=====
// RGMII 接收逻辑
//=====

```

```

//-----
// RGMII 接收逻辑: rgmii_rxc
//-----
PH1_PHY_IOCLK u_rgmii_rxc_int
(
.clkin (rgmii_rxc),
.clkout (rgmii_rxc_int)
);
PH1_PHY_SCLK_V2 u_rgmii_rxc
(
.ce(1'b1),
.clkin (rgmii_rxc),
.clkout (gmii_rx_clk)
);

//-----
// RGMII 接收逻辑: rgmii_rxd ----> gmii_rxd_reg
//-----
// 1Gbps gmii_rxd 8位有效; rgmii_rxc 双沿使能接收8位
// 10/100Mbps gmii_rxd 仅低四位有效, rgmii_rxc 双沿使能接收高低四位数据相同
// 1Gbps时, 125mhz的 rgmii_rxc, 上升沿接收四位数据(对应低4bit), 下降沿接收四位数据(对应高4bit)。一个时钟周期8bit, 125mhz*8=1Gbps
// 100Mbps时, 25mhz的 rgmii_rxc, 上升沿接收四位数据, 下降沿接收四位数据(高四位和低四位数据相同)。相当于单沿采样, 一个时钟周期只收一个4bit(数据有效位在每byte的低四位: 高低四位重复), 25mhz*4=100Mbps
// 10Mbps时同100Mbps, 虽然一个时钟周期上下沿重复接收相同四位数据, 相当于一个时钟周期只收一个4bit。
genvar j;
generate for (j=0; j<4; j=j+1)begin : rxdata_bus
assign rgmii_rxd_delay[j]=rgmii_rxd[j];
end
endgenerate
// Instantiate Double Data Rate Input flip-flops.
// DDR_CLK_EDGE attribute specifies output data alignment from IDDR component
genvar k;
generate for (k=0; k<4; k=k+1)begin : rxdata_in_bus
PH1_LOGIC_IDDR #(
.PIPEMODE ("PIPED")
)
rgmii_rx_data_in
(
.q0 (gmii_rxd_reg[k]),
.q1 (gmii_rxd_reg[k+4]),
.clk (rgmii_rxc_int),
.d (rgmii_rxd_delay[k]),
.rst (1'b0)
);
end
endgenerate

```

```

//-----
// RGMII 接收逻辑: rgmii_rx_ctl -----> gmii_rx_dv、gmii_rx_er
//-----
assign rgmii_rx_ctl_delay = rgmii_rx_ctl;

PH1_LOGIC_IDDR #(
.PIPEMODE          ("PIPED")
)
rgmii_rx_ctl_in
(
.q0                (gmii_rx_dv_reg),
.q1                (rgmii_rx_ctl_reg),
.clk               (rgmii_rxc_int),
.d                (rgmii_rx_ctl_delay),
.rst               (1'b0)
);
// 解码 gmii_rx_er signal
assign gmii_rx_er_reg = gmii_rx_dv_reg ^ rgmii_rx_ctl_reg;
//-----
// 接收逻辑: 内部信号给到输出端口: gmii_rxd、gmii_rx_dv、gmii_rx_er、gmii_col、gmii_crs
//-----
always @ (posedge gmii_rx_clk )begin
gmii_rxd          <= gmii_rxd_reg;
gmii_rx_dv        <= gmii_rx_dv_reg;
gmii_rx_er        <= gmii_rx_er_reg;
end
// 从RGMII创建GMII格式的冲突和载波侦听信号
assign gmii_col = (gmii_tx_en | gmii_tx_er) & (gmii_rx_dv_reg | gmii_rx_er_reg);
assign gmii_crs = (gmii_tx_en | gmii_tx_er) | (gmii_rx_dv_reg | gmii_rx_er_reg);
//=====
// RGMII 状态寄存器
//=====
// 在帧间间隔期间启用带内状态寄存器
assign inband_ce = !(gmii_rx_dv_reg || gmii_rx_er_reg);
always @ (posedge gmii_rx_clk or posedge gmii_rx_reset)begin
    if (gmii_rx_reset) begin
        link_status          <= 1'b0;
        clock_speed[1:0]     <= 2'b0;
        duplex_status        <= 1'b0;
    end
    else if (inband_ce) begin
        link_status          <= gmii_rxd_reg[0];
        clock_speed[1:0]     <= gmii_rxd_reg[2:1];
        duplex_status        <= gmii_rxd_reg[3];
    end
end
end
endmodule

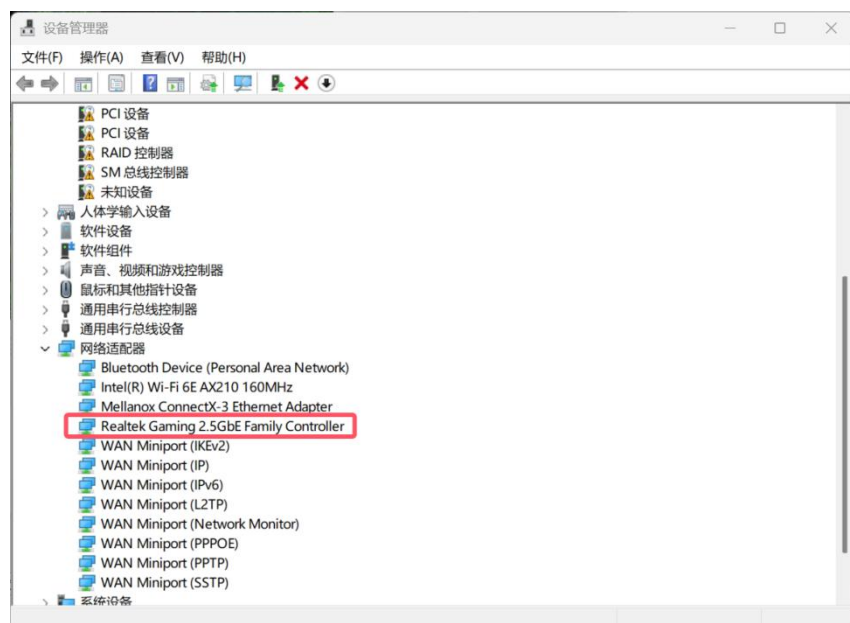
```

4 三速以太网回环测试

4.1 100M 以太网测试

udp 协议栈的逻辑时钟为 12.5MHz，RGMII 接口的时钟为 25MHz，将参数 speed_10_100 设置成 1' b1。

打开设备管理器，在网络适配器选项中选择自己的以太网卡。



在高级选项中将网卡的连接速度和工作模式设置成 100Mbps 全双工。



连接好网线，并将程序下载进开发板中。此时开发板网卡自协商的速度为 100Mbps。使用网络调试助手发送数据帧，FPGA 将报文解析完成后，通过发送端传回 PC，结果如下图所示。



4.2 10M 以太网测试

udp 协议栈的逻辑时钟为 1.25MHz，RGMII 接口的时钟为 2.5MHz，PH1A90SBG484 芯片的 PLL 输出的最低时钟频率为 6.25MHz，所以这两个时钟要由分频逻辑生成。选择 125MHz 时钟进行计数分配，代码如下：

```
reg      clk_1_25;
reg      clk_2_5;
reg      clk_2_5_90;
reg [7:0] clk_cnt;
always@(posedge clk_125 or negedge pll_locked) begin
    if(!pll_locked)
        clk_cnt <= 0;
    else if(clk_cnt == 99)
        clk_cnt <= 0;
    else
        clk_cnt <= clk_cnt + 1;
end
always@(posedge clk_125 or negedge pll_locked) begin
    if(!pll_locked)
        clk_1_25 <= 0;
    else
        case(clk_cnt)
            49 :clk_1_25 <= 1;
            99 :clk_1_25 <= 0;
            default:clk_1_25 <= clk_1_25;
        endcase
end
```

```

end
always@(posedge clk_125 or negedge pll_locked) begin
    if(!pll_locked)
        clk_2_5 <= 0;
    else
        case(clk_cnt)
            24 :clk_2_5 <= 1;
            49 :clk_2_5 <= 0;
            74 :clk_2_5 <= 1;
            99 :clk_2_5 <= 0;
            default:clk_2_5 <= clk_2_5;
        endcase
    end
end
always@(posedge clk_125 or negedge pll_locked) begin
    if(!pll_locked)
        clk_2_5_90 <= 0;
    else
        case(clk_cnt)
            12 :clk_2_5_90 <= 1;
            37 :clk_2_5_90 <= 0;
            62 :clk_2_5_90 <= 1;
            87 :clk_2_5_90 <= 0;
            default:clk_2_5_90 <= clk_2_5_90;
        endcase
    end
end

```

将网卡的连接速度和工作模式设置成 10Mbps 全双工。上电开发板，PC 设置中显示网络连接速度为 10Mbps。



将程序下载进开发板中，通过网络调试助手进行数据回环，结果如下图所示。



ping 包测试结果如下图所示。

```
>ping 192.168.137.2 -t
```

```
正在 Ping 192.168.137.2 具有 32 字节的数据:
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
来自 192.168.137.2 的回复: 字节=32 时间<1ms TTL=128
```

```
192.168.137.2 的 Ping 统计信息:
```

```
数据包: 已发送 = 22, 已接收 = 22, 丢失 = 0 (0% 丢失),
```

```
往返行程的估计时间(以毫秒为单位):
```

```
最短 = 0ms, 最长 = 0ms, 平均 = 0ms
```

03 UDP 巨型帧设计

软件版本: Anlogic -TD_6.2.2_Release_2026.1

操作系统: WIN10/WIN11 64bit

硬件平台: 适用安路(Anlogic)FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 巨型帧介绍

以太网数据帧的长度为 6 字节目的 MAC 地址+6 字节源 MAC 地址+2 字节类型+有效载荷+4 字节 FCS。IEEE802.3 协议规定,以太网帧长最小为 64 字节,最大为 1518 字节。MTU (Maximum Transmission Unit, 最大传输单元)指以太网的有效载荷部分的长度的最大值,为 1500 字节。

巨型帧 (Jumbo Frame) 是指 MTU 超过 1500 字节的以太网帧,通过分片进行数据传输。只有当传输第一个分片时,需要添加 UDP 头部,其他分片不添加,这样减少了非数据片段的开销,提高了带宽利用率。但巨型帧也会带来更大的延时,当巨型帧占用网络时,其它协议需要排队等待访问 MAC 的时间更长,当巨型帧出现错误时,数据重传也需要更长的时间。

在实际使用中,需要相互通讯的两个端口同时支持巨型帧,否则巨型帧会被滤除。

2 巨型帧程序设计

2.1 巨型帧帧格式

巨型帧由 IP 包头部的第 7 和第 8 这两个字节控制,其中高 3bit 为标志字段,低 13bit 为分片偏移字段。标志字段的功能如下:

bit1: 保留位,不使用。

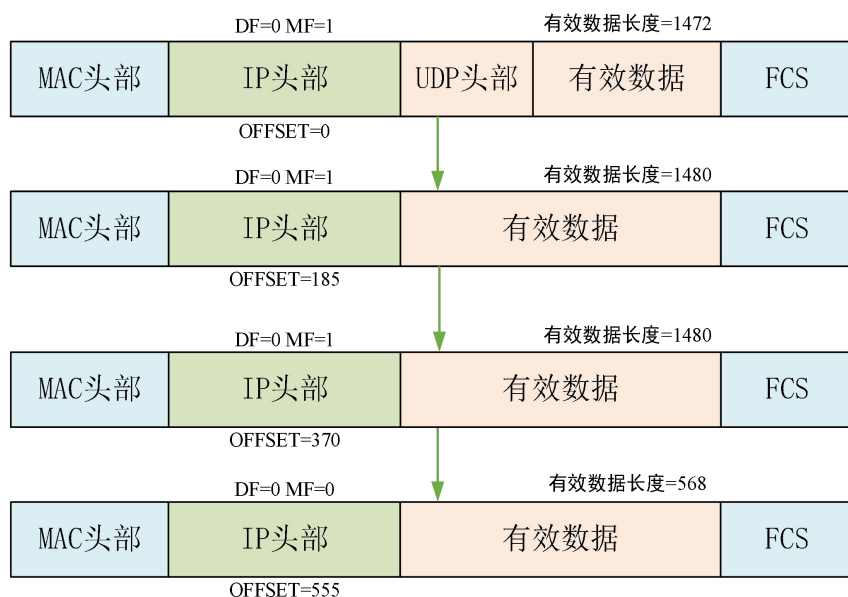
bit2: Don't Fragment, 不分片标志,置 1 时表示不分片,置 0 时表示分片。

bit3: More Fragment, 更多分片标志,置 1 时表示下一个数据包是巨型帧的一个分片,一般在巨型帧除了最后一个分片的数据包都置 1。

分片偏移表示该分片在巨型帧中的位置,由于该字段只有 13 个 bit,而 IP 和 UDP 总长度字段有 16 个 bit,所以该

字段每加 1 表示 8 个字节的数据，比如当一个分片的 UDP 字段长度为 1480 时，第 1 个分片的分片偏移为 0，第 2 个分片的分片偏移为 185 (1480/8)。

假设一次传输 5000 字节的数据，巨型帧发送的示意图如下图所示。



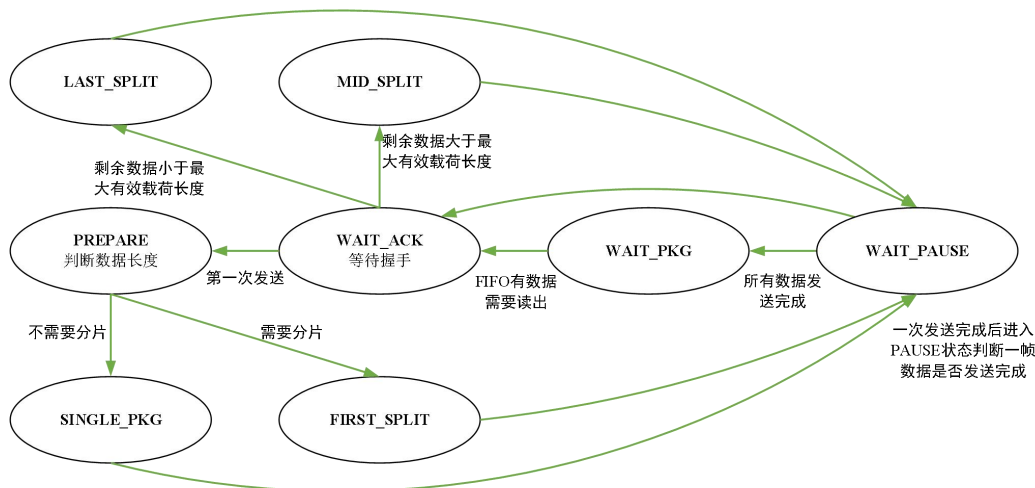
巨型帧的处理需要在 UDP 层和 IP 层进行。

2.2 发送端设计

当开启巨型帧时，如果数据长度小于 (IP 包有效载荷长度-UDP 首部长度的)，则作为普通帧发送，否则开启巨型帧分片。

UDP 层例化两个 FIFO，1 个用于缓存客户端数据，1 个用于缓存长度队列，当用户数据写入数据 FIFO 完成后，再把长度信息写入长度 FIFO，用长度 FIFO 的空信号来判断是否有数据等待发送。

程序中通过状态机控制数据的发送，由于 UDP 层需要向 IP 层握手才能发送数据，状态机的控制会有些复杂。状态机的转换图如下图所示。



WAIT_PKG: 初始状态, 等待需要发送的数据包, 当有数据包需要发送时, 向 IP 层发送请求信号, 状态机跳转至 WAIT_ACK。

WAIT_ACK: 等待 IP 层握手响应, 当发送第一包数据时, 状态机跳转至 PREPARE 状态; 当发送的不是第一包数据时, 判断剩余的数据数量, 如果剩余数据量小于最大有效载荷长度, 则跳转至 LAST_SPLIT 状态, 否则跳转至 MID_SPLIT 状态。

PREPARE: 通过数据长度判断是否需要分片, 需要分片则进入 FIRST_SPLIT 状态, 否则进入 SINGLE_PKG 状态。

FIRST_SPLIT: 发送第一个分片并且封装 UDP 头部, 发送完成后进入 WAIT_PAUSE 状态。

SINGLE_SPLIT: 发送一个完整的普通包, 发送完成后进入 WAIT_PAUSE 状态。

MID_SPLIT: 发送不包括第一个和末尾的分片, 发送完成后进入 WAIT_PAUSE 状态。

LAST_SPLIT: 发送最后一个分片, 发送完成后进入 WAIT_PAUSE 状态。

WAIT_PAUSE: 判断数据是否发送完成, 发送完成则进入 WAIT_PKG 状态, 否则继续向 IP 层发送请求信号, 进入 WAIT_ACK 状态。

UDP 层还需将 DF 标志、MF 标志和分片偏移发送到 IP 层, 在封装 IP 头部和计算 IP 首部校验时需要用到这些数据。

设置了两个顶层参数用来控制巨型帧, JUMBO_ENABLE 为巨型帧使能, 置 1 时打开巨型帧, 置 0 时关闭;

JUMBO_PACKET 为巨帧一个分片的长度。

uiudp_tx 模块的代码如下:

```

module uiudp_tx#(
    parameter JUMBO_ENABLE = 1'b1,
    parameter JUMBO_PACKET = 16'd4048

```

```

)
(
    input    wire    [15:0]          I_udp_local_port,    //UDP 本地端口
    input    wire    [15:0]          I_udp_dest_port,      //UDP 目的端口

    input    wire                                I_reset,
    input    wire                                I_W_udp_clk,        //UDP 写数据时钟
    input    wire                                I_W_udp_req,        //UDP 写请求
    input    wire                                I_W_udp_valid,      //UDP 写有效
    input    wire    [7:0]              I_W_udp_data,          //UDP 写数据
    input    wire    [15:0]             I_W_udp_len,            //UDP 写长度
    output   reg                                O_W_udp_busy,      //UDP 写忙

    input    wire                                I_udp_ip_tbusy,    //ip 层发送忙
    output   reg                                O_udp_ip_treq,      //udp 请求发送数据到 IP 层
    output   reg                                O_udp_ip_tvalid,    //udp 发送 udp 包到 ip 层有效
    output   reg    [7:0]                  O_udp_ip_tdata,        //udp 发送 udp 包到 ip
    output   reg    [15:0]                 O_udp_ip_tpkg_len,      //udp 发送 udp 包长度
    output   wire    [15:0]                 O_udp_ip_flag         //3bit flag + 13bit offset
);
localparam MAX_FRAME_LEN    =    JUMBO_PACKET - 8;
localparam OFFSET          =    JUMBO_PACKET>>8;
localparam CHECKSUM        =    16'h0000; //假如 UDP 包不使用校验和功能, 校验和部分需全部置 0, UDP 发
送不对校验和计算
reg    [1:0] SWFIFO;
reg    [2:0] SRFIFO;

localparam WAIT_PKG        =    2'd0;
localparam PRE             =    2'd1;
localparam WRITE_FIFO      =    2'd2;

localparam WAIT_FIFO       =    'd0;
localparam WAIT_PAUSE      =    'd1;
localparam PREPARE         =    'd2;
localparam FIRST_SPLIT     =    'd3;
localparam MID_SPLIT       =    'd4;
localparam LAST_SPLIT      =    'd5;
localparam SINGLE_PKG      =    'd6;
localparam WAIT_ACK        =    'd7;

reg            fifo_data_wren            ;
reg    [8:0]    fifo_data_din            ;
reg            fifo_data_rden            ;
wire [8:0]      fifo_data_dout           ;
wire            fifo_data_full           ;
wire            fifo_data_empty          ;
wire [16:0]     fifo_data_count          ;

```

```

reg          fifo_info_wren          ;
reg  [15:0]  fifo_info_din           ;
reg          fifo_info_rden          ;
wire [15:0]  fifo_info_dout          ;
wire          fifo_info_full         ;
wire          fifo_info_empty        ;
wire [4:0]   fifo_info_count         ;

wire          udp_send_ready         ;
reg           udp_in_valid           ;
reg  [7:0]    udp_in_data             ;
reg  [15:0]   udp_in_len              ;
wire          udp_in_last            ;

reg  [15:0]   st_cnt                  ;
reg  [15:0]   udp_length              ;
reg           split_flag              ;
reg           more_fragments          ;
reg  [12:0]   udp_offset              ;
reg           udp_out_last            ;

assign  udp_send_ready  =  (fifo_data_count > 'd20000 || fifo_info_count > 'd12) ? 0 : 1;
assign  udp_in_last     =  udp_in_valid && ~I_W_udp_valid;
assign  0_udp_ip_flag   =  {1'b0, ~split_flag, more_fragments, udp_offset};
always @(posedge I_W_udp_clk) begin
    udp_in_valid    <=  I_W_udp_valid;
    udp_in_data     <=  I_W_udp_data;
    udp_in_len      <=  I_W_udp_len;
end
always @(posedge I_W_udp_clk) begin
    if(I_reset) begin
        fifo_data_wren    <=  1'b0;
        fifo_data_din     <=  65'd0;
        fifo_info_wren    <=  1'b0;
        fifo_info_din     <=  65'd0;
        0_W_udp_busy      <=  0;
        SWFIFO            <=  WAIT_PKG;
    end
    else begin
        case(SWFIFO)
            WAIT_PKG:begin
                if(I_W_udp_req && !0_W_udp_busy && udp_send_ready) begin
                    0_W_udp_busy    <=  1;
                    SWFIFO          <=  PRE;
                end
            else begin
                fifo_data_wren    <=  1'b0;
                fifo_data_din     <=  65'd0;
            end
        endcase
    end
end

```

```

        fifo_info_wren    <= 1'b0;
        fifo_info_din     <= 65'd0;
        O_W_udp_busy      <= 0;
        SWFIFO             <= WAIT_PKG;
    end
end
PRE:begin
    if(udp_in_valid) begin
        fifo_info_wren    <= 1;
        fifo_info_din     <= I_W_udp_len;
        fifo_data_wren    <= 1;
        fifo_data_din     <= {udp_in_last, udp_in_data};
        SWFIFO             <= WRITE_FIFO;
    end
end
WRITE_FIFO:begin
    fifo_info_wren    <= 0;
    fifo_info_din     <= 0;
    fifo_data_wren    <= 1;
    fifo_data_din     <= {udp_in_last, udp_in_data};
    if(udp_in_valid && ~I_W_udp_valid) begin
        O_W_udp_busy      <= 0;
        SWFIFO             <= WAIT_PKG;
    end
end
default:begin
    fifo_data_wren    <= 1'b0;
    fifo_data_din     <= 65'd0;
    fifo_info_wren    <= 1'b0;
    fifo_info_din     <= 65'd0;
    O_W_udp_busy      <= 0;
    SWFIFO             <= WAIT_PKG;
end
endcase
end
end
udp_data_fifo udp_data_fifo
(
    .clk                (I_W_udp_clk),
    .srst                (I_reset),
    .din                (fifo_data_din),
    .wr_en              (fifo_data_wren),
    .rd_en              (fifo_data_rden),
    .dout               (fifo_data_dout),
    .full               (fifo_data_full),
    .empty              (fifo_data_empty),
    .data_count         (fifo_data_count)
);

```

```

udp_info_fifo udp_info_fifo
(
    .clk                (I_W_udp_clk),
    .srst               (I_reset),
    .din                (fifo_info_din),
    .wr_en              (fifo_info_wren),
    .rd_en              (fifo_info_rden),
    .dout               (fifo_info_dout),
    .full               (fifo_info_full),
    .empty              (fifo_info_empty),
    .data_count         (fifo_info_count)
);

always@(posedge I_W_udp_clk) begin
    if(I_reset)
        st_cnt    <= 0;
    else if(SRFIFO == WAIT_PKG)
        st_cnt    <= 0;
    else if(SRFIFO == FIRST_SPLIT || SRFIFO == SINGLE_PKG || SRFIFO == MID_SPLIT || SRFIFO == LAST_SPLIT)
        st_cnt    <= st_cnt + 1;
    else
        st_cnt    <= 0;
end

always@(posedge I_W_udp_clk) begin
    if(I_reset)
        SRFIFO    <= WAIT_PKG;
    else begin
        case(SRFIFO)
            WAIT_PKG:begin
                if(!fifo_info_empty && !I_udp_ip_tbusy)
                    SRFIFO <= WAIT_ACK;
                else
                    SRFIFO <= WAIT_PKG;
            end
            WAIT_PAUSE:begin
                if(!split_flag)
                    SRFIFO <= WAIT_PKG;
                else if(!I_udp_ip_tbusy && split_flag)
                    SRFIFO <= WAIT_ACK;
                else
                    SRFIFO <= WAIT_PAUSE;
            end
            WAIT_ACK:begin
                if(I_udp_ip_tbusy)
                    if(!split_flag)
                        SRFIFO <= PREPARE;
                    else if((udp_offset + OFFSET) > (udp_length >> 3))
                        SRFIFO <= LAST_SPLIT;
                    else

```

```

                                SRFIFO    <=    MID_SPLIT;

                                else

                                SRFIFO    <=    WAIT_ACK;

                                end

                                PREPARE:begin
                                    if(fifo_info_dout > MAX_FRAME_LEN && JUMBO_ENABLE)
                                        SRFIFO    <=    FIRST_SPLIT;
                                    else
                                        SRFIFO    <=    SINGLE_PKG;
                                    end
                                FIRST_SPLIT, MID_SPLIT, LAST_SPLIT, SINGLE_PKG:begin
                                    if(udp_out_last)
                                        SRFIFO    <=    WAIT_PAUSE;
                                    else
                                        SRFIFO    <=    SRFIFO;
                                    end
                                default:begin
                                    SRFIFO    <=    WAIT_PKG;
                                end
                                endcase
                                end
                                end

                                always@(posedge I_W_udp_clk) begin
                                    if(I_reset)
                                        O_udp_ip_treq    <=    0;
                                    else if(I_udp_ip_tbusy)
                                        O_udp_ip_treq    <=    0;
                                    else if(SRFIFO == WAIT_PAUSE && !I_udp_ip_tbusy && split_flag)
                                        O_udp_ip_treq    <=    1;
                                    else if(SRFIFO == WAIT_PKG && !fifo_info_empty && !I_udp_ip_tbusy)
                                        O_udp_ip_treq    <=    1;
                                    else
                                        O_udp_ip_treq    <=    O_udp_ip_treq;
                                    end
                                end

                                always@(posedge I_W_udp_clk) begin
                                    if(I_reset)
                                        fifo_info_rden    <=    0;
                                    else if(SRFIFO == WAIT_PKG && !fifo_info_empty && !I_udp_ip_tbusy)
                                        fifo_info_rden    <=    1;
                                    else
                                        fifo_info_rden    <=    0;
                                    end
                                end

                                always@(posedge I_W_udp_clk) begin
                                    if(I_reset)
                                        fifo_data_rden    <=    0;
                                    else if(fifo_data_dout[8] && fifo_data_rden)
                                        fifo_data_rden    <=    0;
                                    else if(st_cnt == JUMBO_PACKET && JUMBO_ENABLE)

```

```

        fifo_data_rden    <= 0;
    else if((SRFIFO == FIRST_SPLIT || SRFIFO == SINGLE_PKG) && st_cnt == 'd8)
        fifo_data_rden    <= 1;
    else if((SRFIFO == MID_SPLIT || SRFIFO == LAST_SPLIT) && st_cnt == 'd0)
        fifo_data_rden    <= 1;
    else
        fifo_data_rden    <= fifo_data_rden;
end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        udp_length    <= 0;
    else if(SRFIFO == PREPARE)
        udp_length    <= fifo_info_dout + 16'd8;
    else
        udp_length    <= udp_length;
end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        split_flag    <= 0;
    else if(SRFIFO == LAST_SPLIT && udp_out_last)
        split_flag    <= 0;
    else if(SRFIFO == PREPARE && fifo_info_dout > MAX_FRAME_LEN && JUMBO_ENABLE)
        split_flag    <= 1;
    else
        split_flag    <= split_flag;
end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        more_fragments    <= 0;
    else if(SRFIFO == FIRST_SPLIT || SRFIFO == MID_SPLIT)
        more_fragments    <= 1;
    else
        more_fragments    <= 0;
end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        udp_out_last    <= 0;
    else if(fifo_data_dout[8] && fifo_data_rden)
        udp_out_last    <= 1;
    else if(st_cnt == JUMBO_PACKET && JUMBO_ENABLE)
        udp_out_last    <= 1;
    else
        udp_out_last    <= 0;
end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        O_udp_ip_tdata    <= 0;
    else if(SRFIFO == FIRST_SPLIT || SRFIFO == SINGLE_PKG)

```

```

        case(st_cnt)
            1      :0_udp_ip_tdata <= I_udp_local_port[15:8];
            2      :0_udp_ip_tdata <= I_udp_local_port[7:0];
            3      :0_udp_ip_tdata <= I_udp_dest_port[15:8];
            4      :0_udp_ip_tdata <= I_udp_dest_port[7:0];
            5      :0_udp_ip_tdata <= udp_length[15:8];
            6      :0_udp_ip_tdata <= udp_length[7:0];
            7      :0_udp_ip_tdata <= CHECKSUM[15:8];
            8      :0_udp_ip_tdata <= CHECKSUM[7:0];
            default :0_udp_ip_tdata <= fifo_data_dout[7:0];
        endcase
    else
        0_udp_ip_tdata <= fifo_data_dout[7:0];
    end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        0_udp_ip_tvalid <= 0;
    else if(udp_out_last)
        0_udp_ip_tvalid <= 0;
    else if(SRFIFO == FIRST_SPLIT || SRFIFO == SINGLE_PKG || SRFIFO == MID_SPLIT || SRFIFO == LAST_SPLIT)
        0_udp_ip_tvalid <= st_cnt > 0;
    else
        0_udp_ip_tvalid <= 0_udp_ip_tvalid;
    end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        0_udp_ip_tpkg_len <= 0;
    else if(fifo_info_rden)
        0_udp_ip_tpkg_len <= JUMBO_ENABLE ? fifo_info_dout > MAX_FRAME_LEN ? JUMBO_PACKET :
        fifo_info_dout + 8 : fifo_info_dout + 8;
    else if(SRFIFO == WAIT_PAUSE && (udp_offset + OFFSET) > (udp_length >> 3))
        0_udp_ip_tpkg_len <= udp_length - (udp_offset << 3);
    else if(SRFIFO == WAIT_PAUSE )
        0_udp_ip_tpkg_len <= JUMBO_PACKET;
    else
        0_udp_ip_tpkg_len <= 0_udp_ip_tpkg_len;
    end
always@(posedge I_W_udp_clk) begin
    if(I_reset)
        udp_offset <= 0;
    else if(SRFIFO == WAIT_PKG)
        udp_offset <= 0;
    else if(SRFIFO != LAST_SPLIT && udp_out_last && 0_udp_ip_tvalid)
        udp_offset <= udp_offset + OFFSET;
    else
        udp_offset <= udp_offset;
    end
endmodule

```

2.3 接收端设计

首先在 IP 层把 IP 包头的 MF 标志解析出来，发送到 UDP 层用于判断数据包是否为分片，如果是分片，则继续将下一个数据包写入 FIFO，等待全部数据包写入 FIFO 完成后，再将完整的有效数据段读出。

uiudp_rx 代码如下：

```
module uiudp_rx
(
    input wire I_reset,
    input wire I_R_udp_clk, //udp 接收时钟,和 ip_layer 接收的数据时
    钟相同
    output reg O_R_udp_valid, //输出到用户层 udp 读数据有效
    output reg [7:0] O_R_udp_data, //输出到用户层 udp 数据
    output reg [15:0] O_R_udp_len, //输出到用户层读 udp 数据长度
    output reg [15:0] O_R_udp_src_port, //输出到用户层接收到的 UDP 数据包的端口号

    input wire I_udp_ip_rvalid, //udp 接收来自 ip 层的数据有效信号
    input wire [7:0] I_udp_ip_rdata, //udp 接收来自 ip 层数据
    input wire I_udp_ip_rmf
);
reg fifo_data_rden;
wire [8:0] fifo_data_dout;
wire fifo_data_full;
wire fifo_data_empty;
wire [16:0] fifo_data_count;

reg fifo_info_rden;
wire [15:0] fifo_info_dout;
wire fifo_info_full;
wire fifo_info_empty;
wire [4:0] fifo_info_count;

reg rec_data_valid;
reg [7:0] rec_data;
reg rec_data_mf; //打拍信号

reg [15:0] rx_length;
reg info_lock;
reg [15:0] rx_cnt;
reg [15:0] fifo_cnt;
reg [15:0] udp_pkg_len;
reg [15:0] udp_length;
always@(posedge I_R_udp_clk) begin
    rec_data_valid <= I_udp_ip_rvalid;
    rec_data <= I_udp_ip_rdata;
    rec_data_mf <= I_udp_ip_rmf;
```

```

end
udp_data_fifo udp_data_fifo
(
    .clk                (I_R_udp_clk),
    .srst               (I_reset),
    .din                ({1'b0, I_udp_ip_rdata}),
    .wr_en              (I_udp_ip_rvalid),
    .rd_en              (fifo_data_rden),
    .dout               (fifo_data_dout),
    .full               (fifo_data_full),
    .empty              (fifo_data_empty),
    .data_count         (fifo_data_count)
);
udp_info_fifo udp_info_fifo
(
    .clk                (I_R_udp_clk),
    .srst               (I_reset),
    .din                (rx_length),
    .wr_en              (rec_data_valid && ~I_udp_ip_rvalid && ~rec_data_mf),
    .rd_en              (fifo_info_rden),
    .dout               (fifo_info_dout),
    .full               (fifo_info_full),
    .empty              (fifo_info_empty),
    .data_count         (fifo_info_count)
);
always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        fifo_cnt      <= 0;
    else if(I_udp_ip_rvalid)
        fifo_cnt      <= fifo_cnt + 1;
    else
        fifo_cnt      <= 0;
end
always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        info_lock     <= 0;
    else if(fifo_cnt == 6)
        info_lock     <= I_udp_ip_rmfm ? 1 : 0;
    else
        info_lock     <= info_lock;
end
always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        rx_length     <= 0;
    else if((fifo_cnt == 4 || fifo_cnt == 5) && !info_lock)
        rx_length     <= {rx_length[7:0], I_udp_ip_rdata[7:0]};
    else
        rx_length     <= rx_length;
end

```

```

end

always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        udp_length <= 0;
    else if(fifo_info_rden)
        udp_length <= fifo_info_dout < 26 ? 26 : fifo_info_dout;
    else
        udp_length <= udp_length;
end

always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        udp_pkg_len <= 0;
    else if(fifo_info_rden)
        udp_pkg_len <= fifo_info_dout;
    else
        udp_pkg_len <= udp_pkg_len;
end

always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        fifo_info_rden <= 0;
    else if(fifo_info_rden || fifo_data_rden)
        fifo_info_rden <= 0;
    else if(!fifo_info_empty)
        fifo_info_rden <= 1;
    else
        fifo_info_rden <= 0;
end

always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        fifo_data_rden <= 0;
    else if(rx_cnt == udp_length - 1)
        fifo_data_rden <= 0;
    else if(fifo_info_rden)
        fifo_data_rden <= 1;
    else
        fifo_data_rden <= fifo_data_rden;
end

always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        rx_cnt <= 0;
    else if(fifo_data_rden)
        rx_cnt <= rx_cnt + 1;
    else
        rx_cnt <= 0;
end

always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)

```

```

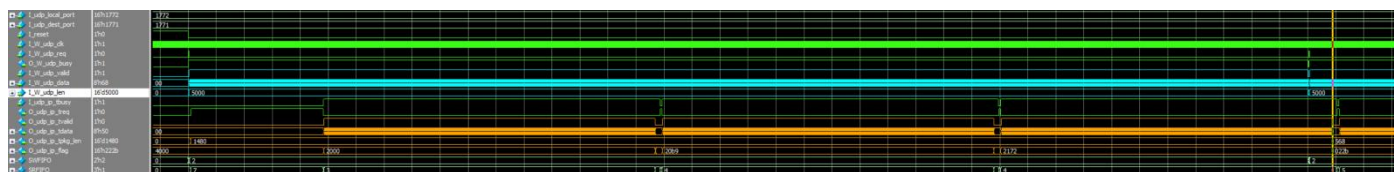
        O_R_udp_valid    <=  0;
    else if(rx_cnt > 7 && rx_cnt < udp_pkg_len)
        O_R_udp_valid    <=  1;
    else
        O_R_udp_valid    <=  0;
end
always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        O_R_udp_data     <=  0;
    else if(rx_cnt > 7 && rx_cnt < udp_pkg_len)
        O_R_udp_data     <=  fifo_data_dout[7:0];
    else
        O_R_udp_data     <=  0;
end
always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        O_R_udp_len <=  0;
    else if(fifo_info_rden)
        O_R_udp_len <=  fifo_info_dout - 8;
    else
        O_R_udp_len <=  O_R_udp_len;
end
always@(posedge I_R_udp_clk or posedge I_reset) begin
    if(I_reset)
        O_R_udp_src_port <=  0;
    else if(fifo_data_rden && (rx_cnt == 0 || rx_cnt == 1))
        O_R_udp_src_port <=  {O_R_udp_src_port[7:0], fifo_data_dout[7:0]};
    else
        O_R_udp_src_port <=  O_R_udp_src_port;
end
endmodule

```

3 巨型帧验证

3.1 仿真验证

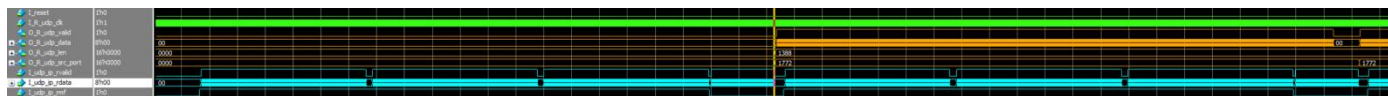
首先对设计完成的 uiudp_tx 模块进行仿真验证，设置 JUMBO_ENABLE 为 1，JUMBO_PACKET 为 1480，用户端持续发送一段长度为 5000 字节的测试数据，仿真波形图如下图所示。



该数据被分成了 4 个分片依次发送出去，前 3 个分片的 UDP 长度为 1480，最后 1 个分片的 UDP 长度为 568，

则数据发送的长度正确 ($1480 \times 3 + 568 - 8 = 5000$)。标志位分别为 3' b001、3' b001、3' b001、3' b000，标志位都正确。偏移量的计算为 (片偏移字段 * 8)，则偏移量分别为 0、1480、2960、4440，偏移量都正确。

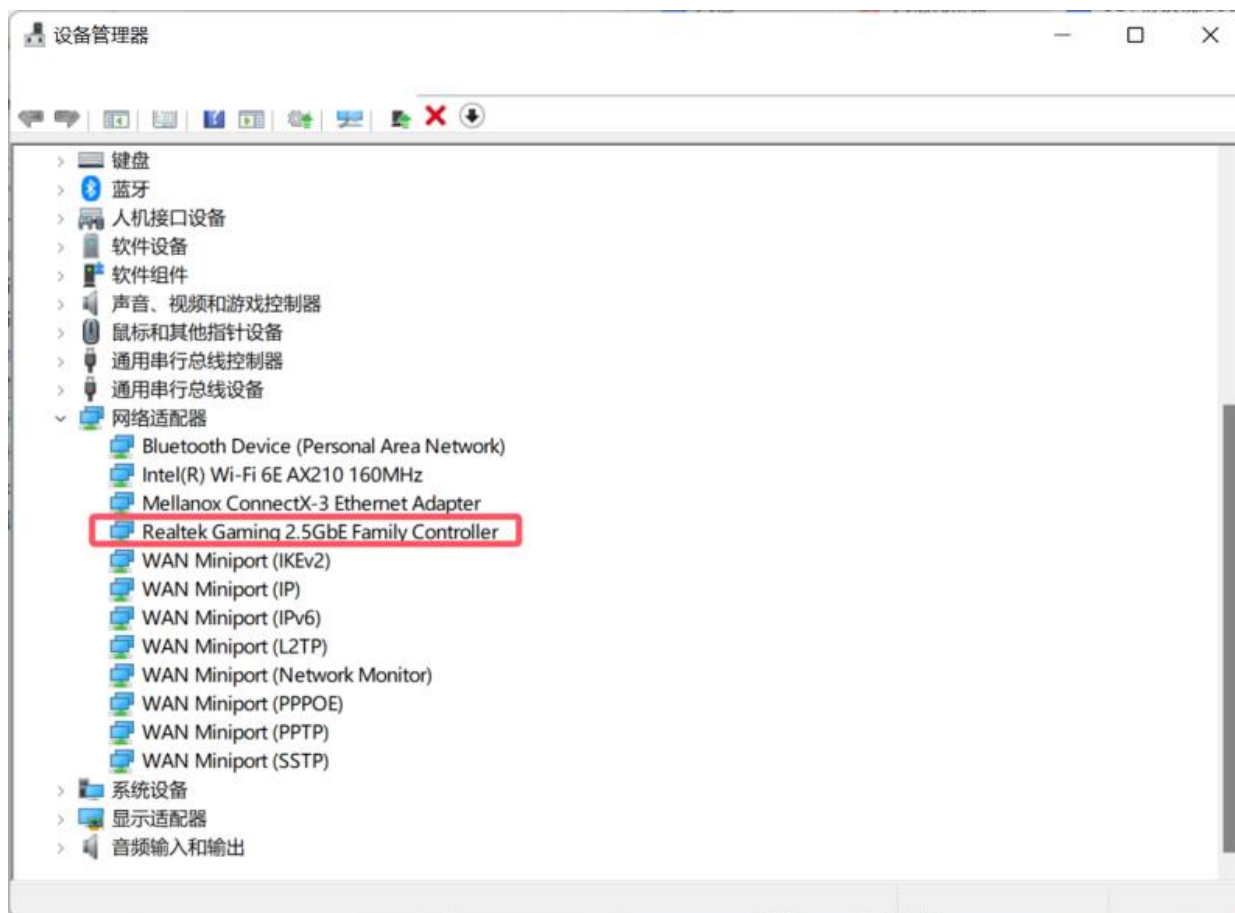
将发送端发送的巨型帧通过 MAC 回环进接收端，uiudp_rx 模块的仿真波形图如下图所示。



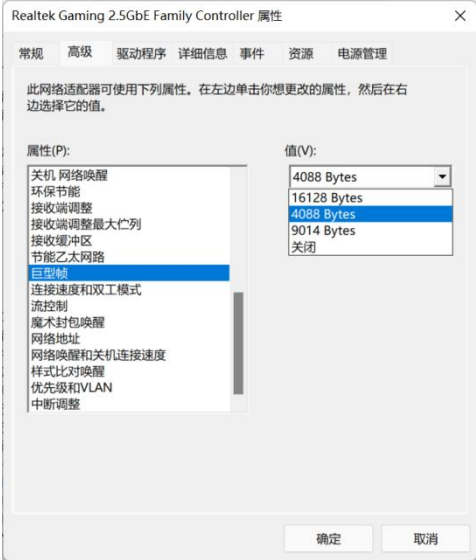
IP 层解析出 MF 信号用于 UDP 层判断数据是独立的还是巨帧分片，当 4 个巨帧分片都写入 FIFO 后，将 FIFO 中的数据读出重组成为完整的 5000 字节的数据流。

3.2 上板验证

首先在电脑“设备管理器”中展开“网络适配器”选项，找到自己的网卡，双击进入设置。



在高级选项卡中，找到巨型帧选项开启，不同厂家设置的传输单元大小可能有所不同，本次验证选择 4088Bytes。

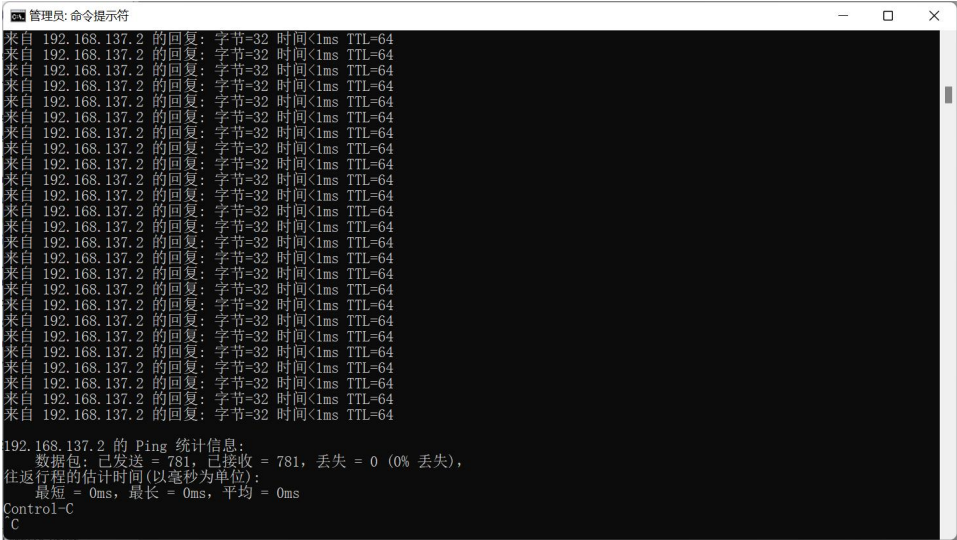


首先使用网络调试助手发送一个巨型帧，通过 wireshark 软件抓取。

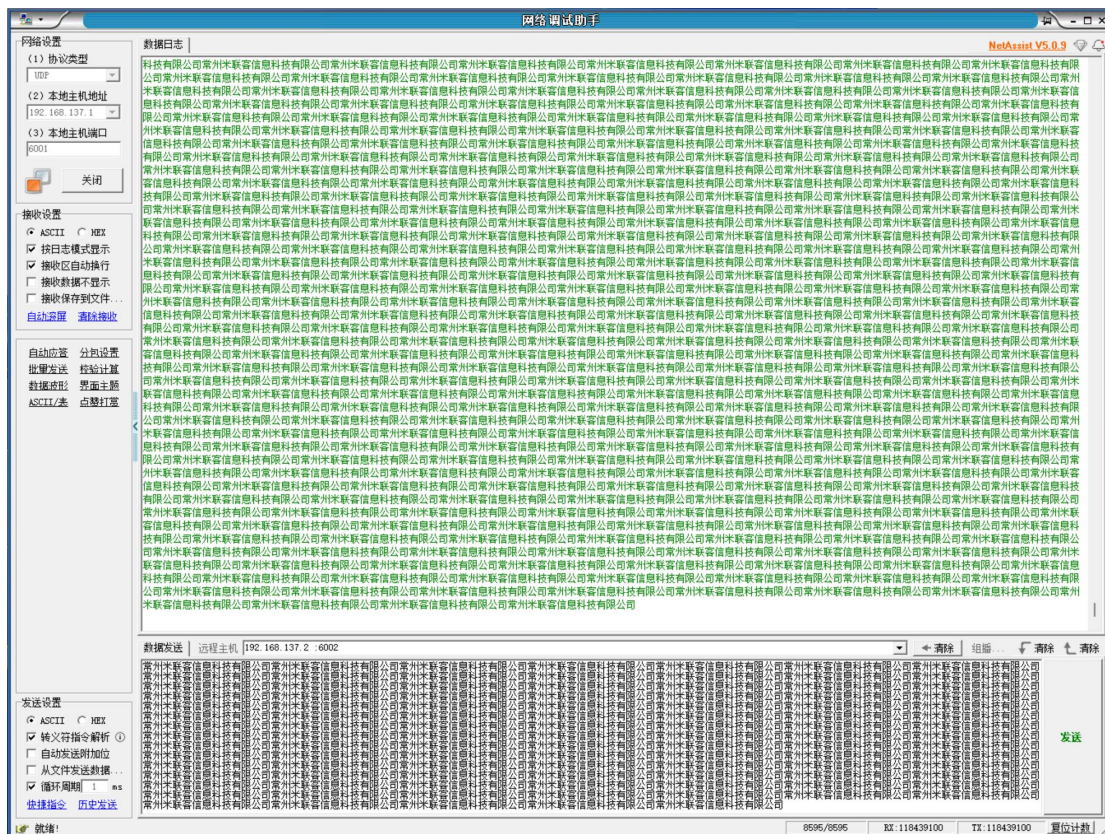
716	41.341132	192.168.137.1	192.168.137.2	IPv4	4082	Fragmented IP protocol (proto=UDP 17, off=0, ID=5989) [Reassembled in #719]
717	41.341132	192.168.137.1	192.168.137.2	IPv4	4082	Fragmented IP protocol (proto=UDP 17, off=4048, ID=5989) [Reassembled in #719]
718	41.341132	192.168.137.1	192.168.137.2	IPv4	4082	Fragmented IP protocol (proto=UDP 17, off=8096, ID=5989) [Reassembled in #719]
719	41.341132	192.168.137.1	192.168.137.2	UDP	1678	6001 → 6002 Len=13780

可以抓到每一帧的长度为 4082，则 JUMBO_FRAME 的参数设置为 4048 (4082-20 (IP 首部) -14 (MAC 首部))。

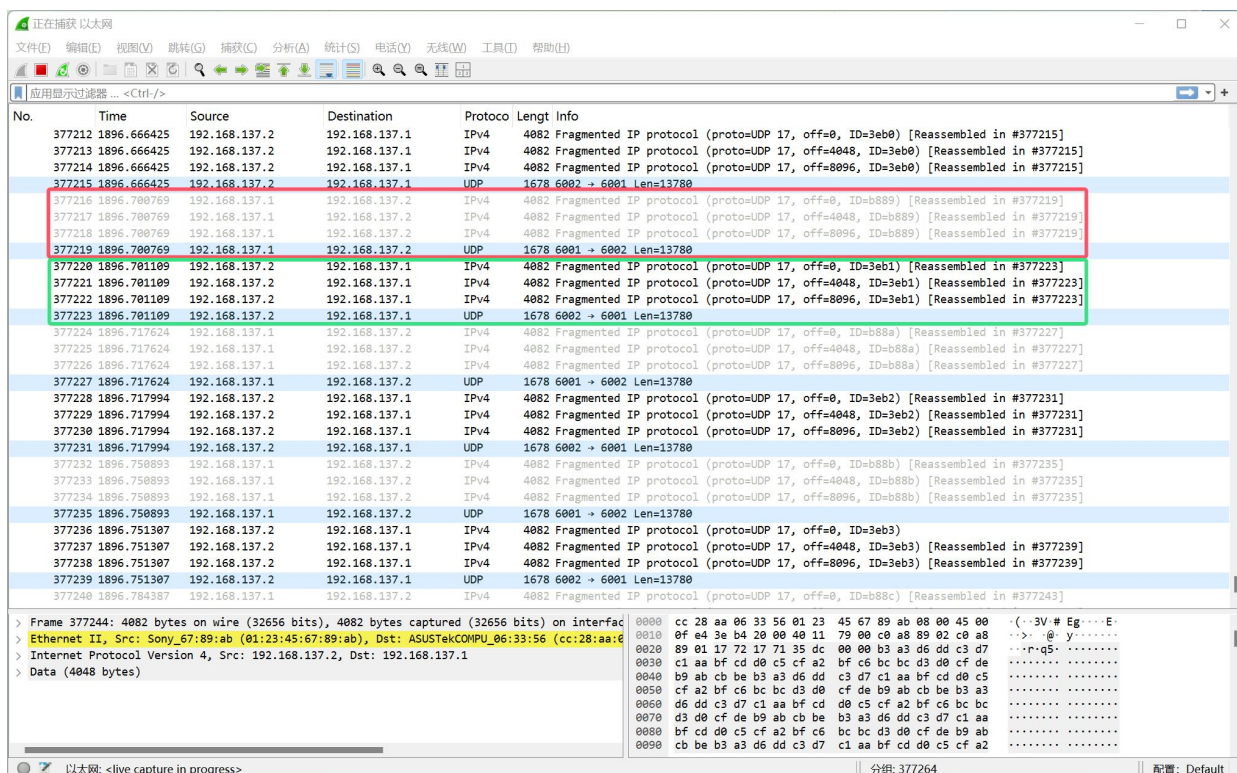
将程序下载进开发板中，首先通过 ping 包测试网络是否通畅。



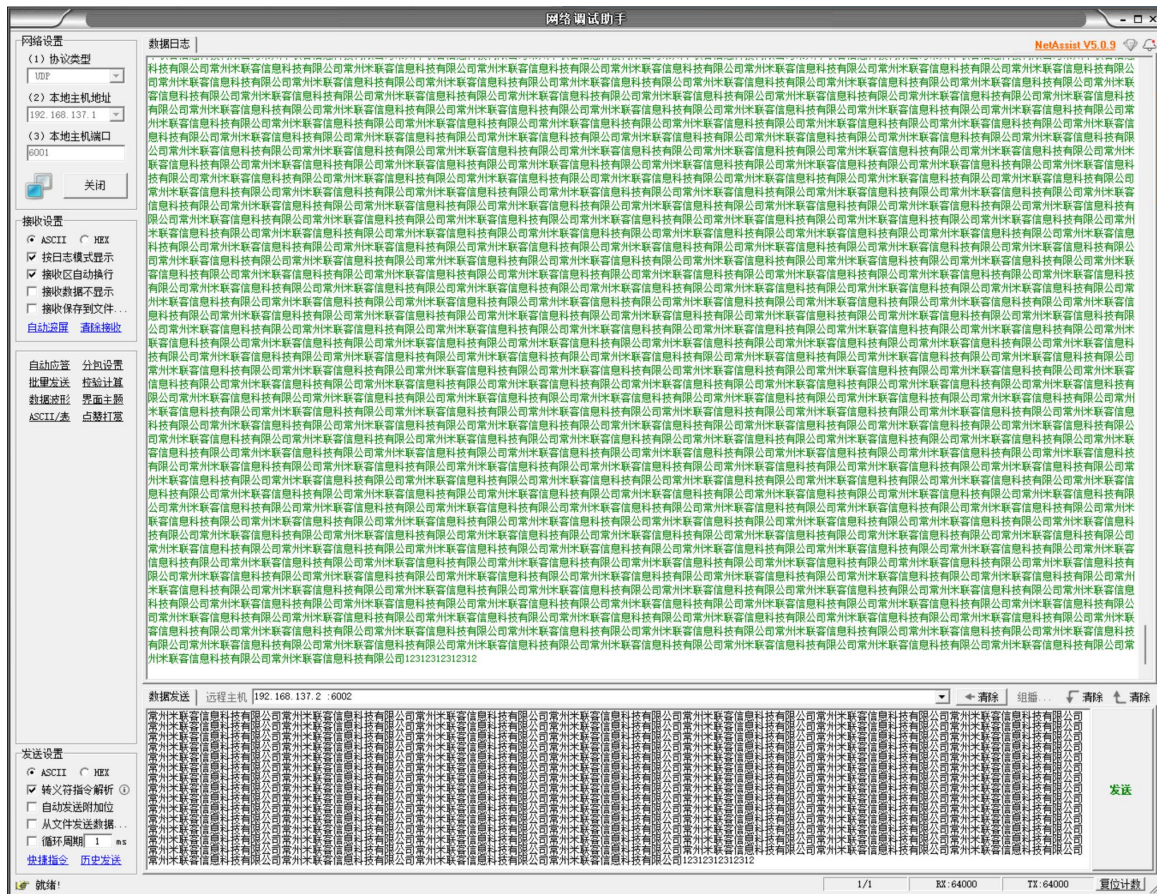
接着通过网络调试助手进行回环测试。



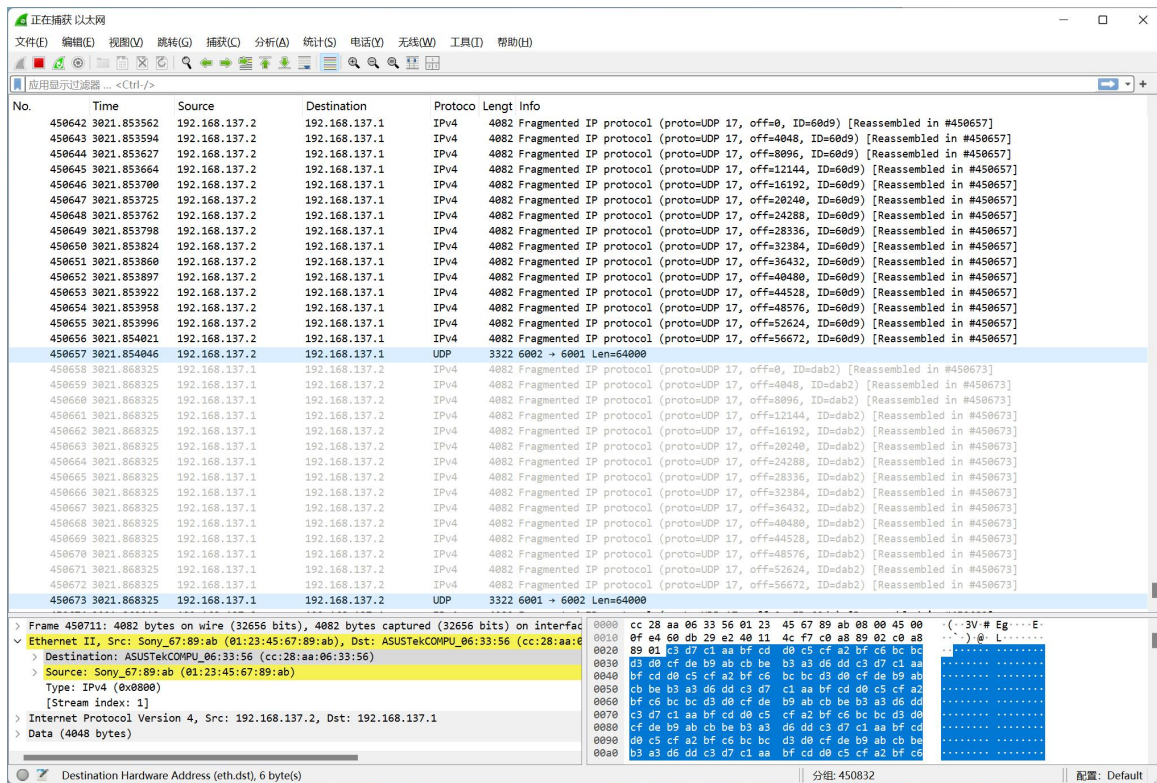
可以看到收发数据量一致，并且环回的数据正确。通过 wireshark 抓取的数据包如下图所示，红框为电脑向开发板发送的巨帧分片，在最后一个分片时重组成为巨型帧。



在网络调试助手发送 64000 字节长度的数据，验证巨型帧最大帧长。网络调试助手接收到的数据如下图所示。



wireshark 抓取的数据如下图所示。



04 基于 UDP 的 AD7606 采集方案

软件版本: Anlogic -TD_6.2.2_Release_2026.1

操作系统: WIN10/WIN11 64bit

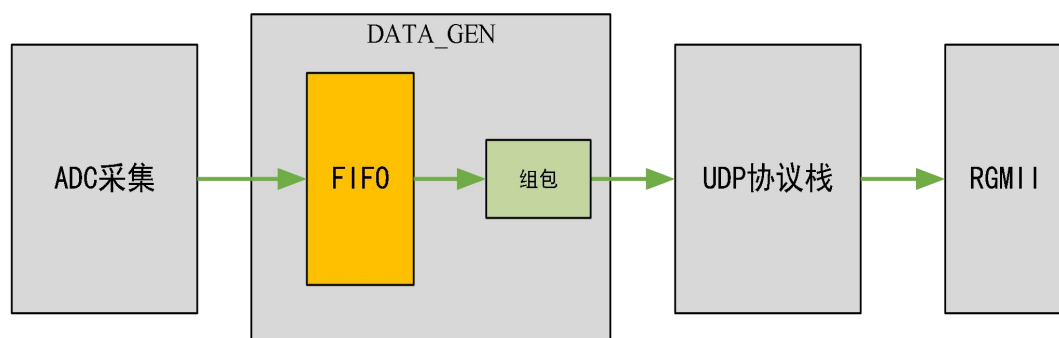
硬件平台: 适用安路(Anlogic)FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频教程、答疑解惑!

1 系统框图

本方案通过把 AD9248 采集到的数据, 通过千兆以太网 UDP 协议发送到 PC, 最终在上位机中显示波形图。

AD9248 的工作时序和驱动设计这里不再展开讲解, 如有需要可以在米联客 FPGA 教程基础篇中学习, 本方案的系统框图如下图所示。



通过 ADC 采集的数据先进入 FIFO 中缓存, 当 FIFO 中的数据量达到一定的阈值时, 将数据读出, 并且添加识别码和信息到读出的数据段之前作为头部, 把组好的数据段发送至 UDP 协议栈中进行组帧。将组好的 MAC 帧由单沿转为双沿, 发送至 PHY 芯片中, PHY 芯片完成编码和串化后由网线传输到 PC 端。

上位机数据段的格式如下图所示。

UDP首部	ID号	帧序列号	通道数	符号	长度	数据
	8Byte	2Byte	6bit	2bit	4Byte	

各字段的功能定义如下:

ID 号: 上位机识别码, 固定为 64' hAA55_AA55_AA55_AA55。

帧序列号: 每发完一帧加 1。

通道数: 波形采样的通道数, 默认为 8。

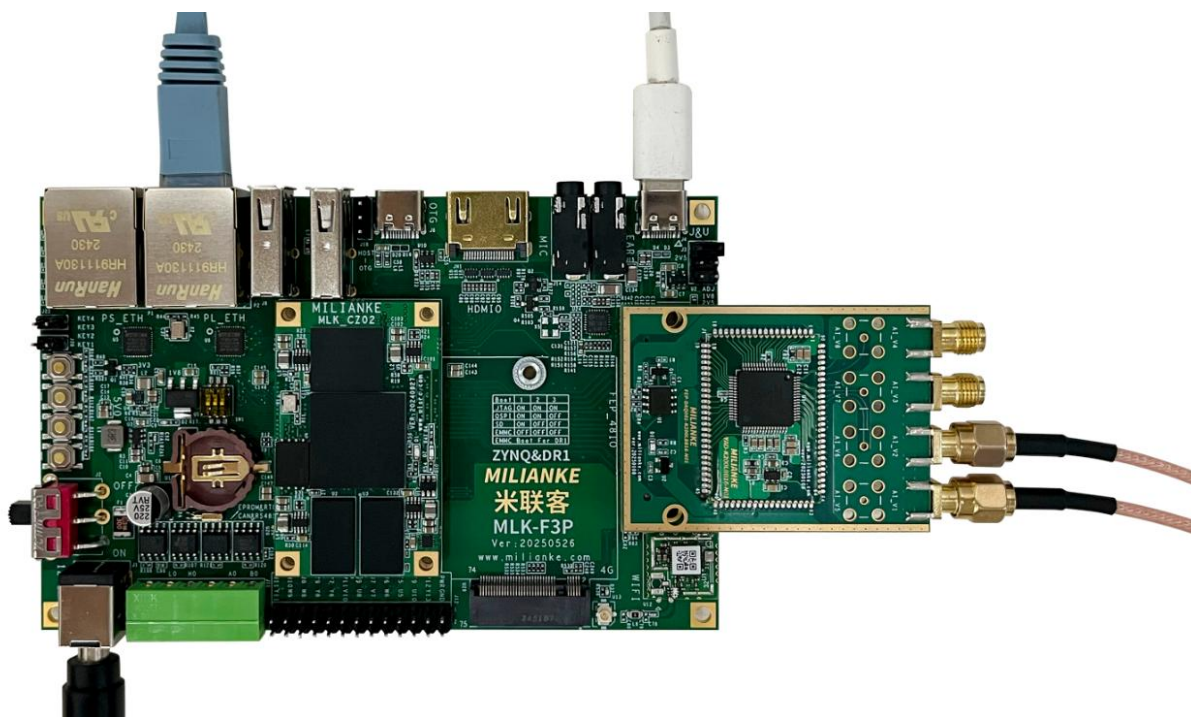
符号: 是否有符号位, 默认为 0, 没有符号位。

长度：每通道数据的长度。

注意数据段的头部和数据部分都是小端序，即先发送低字节，再发送高字节。

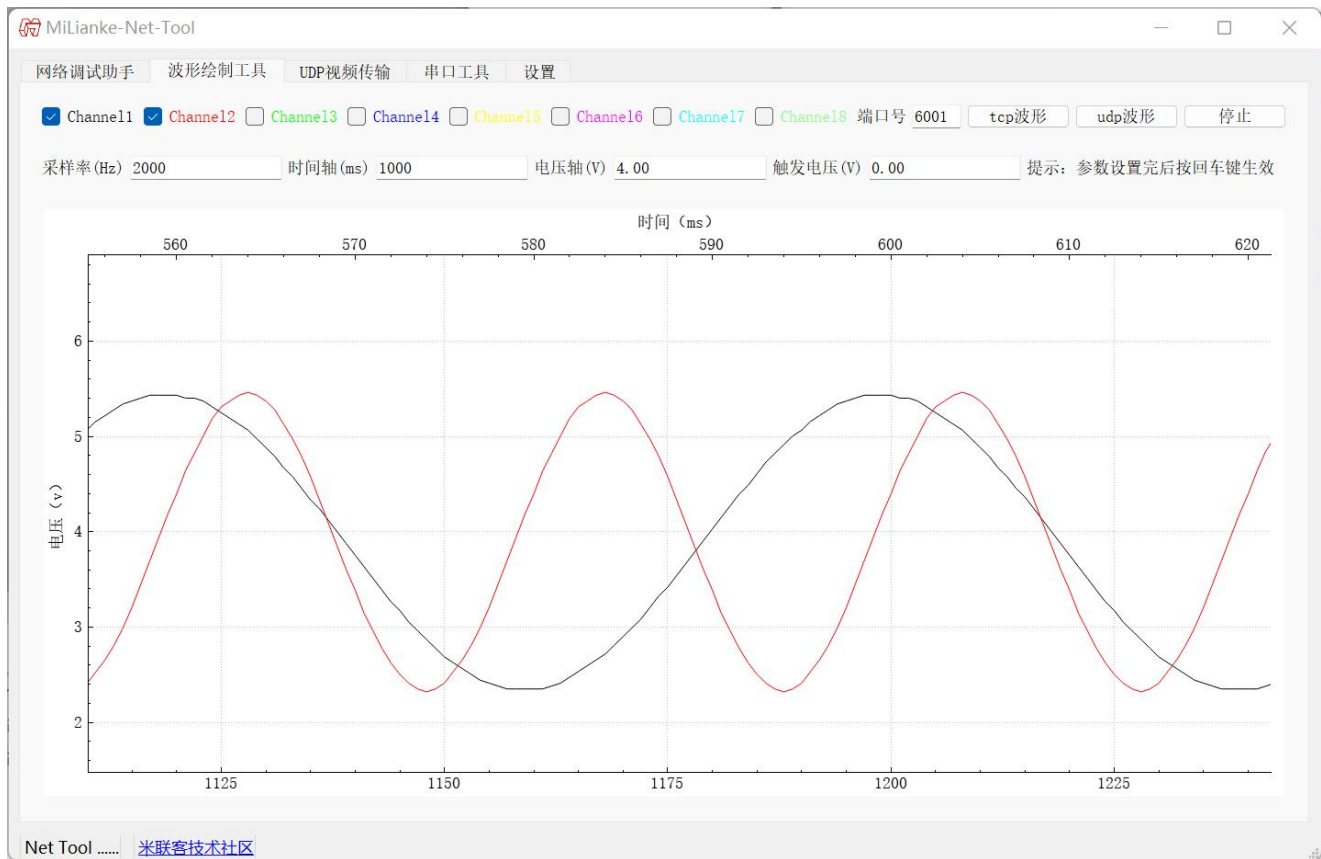
2 下载演示

2.1 硬件连接



2.2 运行结果

波形采集的效果如下图所示。



05 基于 UDP 的 AD9248 采集方案

软件版本: Anlogic -TD_6.2.2_Release_2026.1

操作系统: WIN10/WIN11 64bit

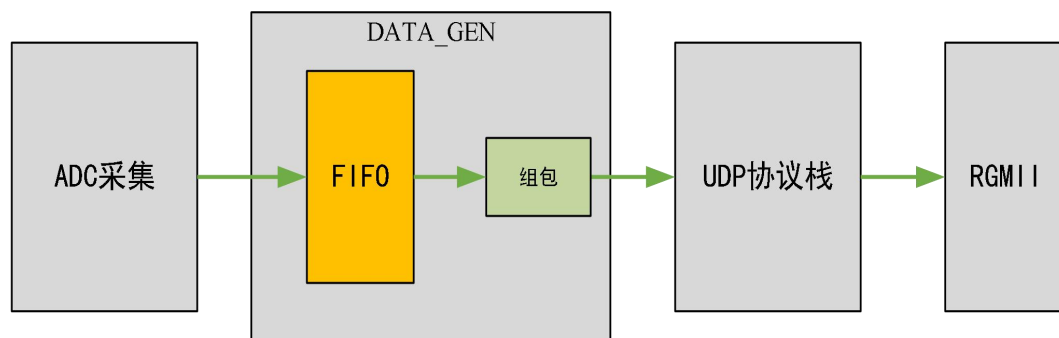
硬件平台: 适用安路(Anlogic)FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 系统框图

本方案通过把 AD9248 采集到的数据, 通过千兆以太网 UDP 协议发送到 PC, 最终在上位机中显示波形图。

AD9248 的工作时序和驱动设计这里不再展开讲解, 如有需要可以在米联客 FPGA 教程基础篇中学习, 本方案的系统框图如下图所示。



通过 ADC 采集的数据先进入 FIFO 中缓存, 当 FIFO 中的数据量达到一定的阈值时, 将数据读出, 并且添加识别码和信息到读出的数据段之前作为头部, 把组好的数据段发送至 UDP 协议栈中进行组帧。将组好的 MAC 帧由单沿转为双沿, 发送至 PHY 芯片中, PHY 芯片完成编码和串化后由网线传输到 PC 端。

上位机数据段的格式如下图所示。

UDP首部	ID号	帧序列号	通道数	符号	长度	数据
	8Byte	2Byte	6bit	2bit	4Byte	

各字段的功能定义如下:

ID 号: 上位机识别码, 固定为 64' hAA55_AA55_AA55_AA55。

帧序列号: 每发完一帧加 1。

通道数: 波形采样的通道数, 默认为 8。

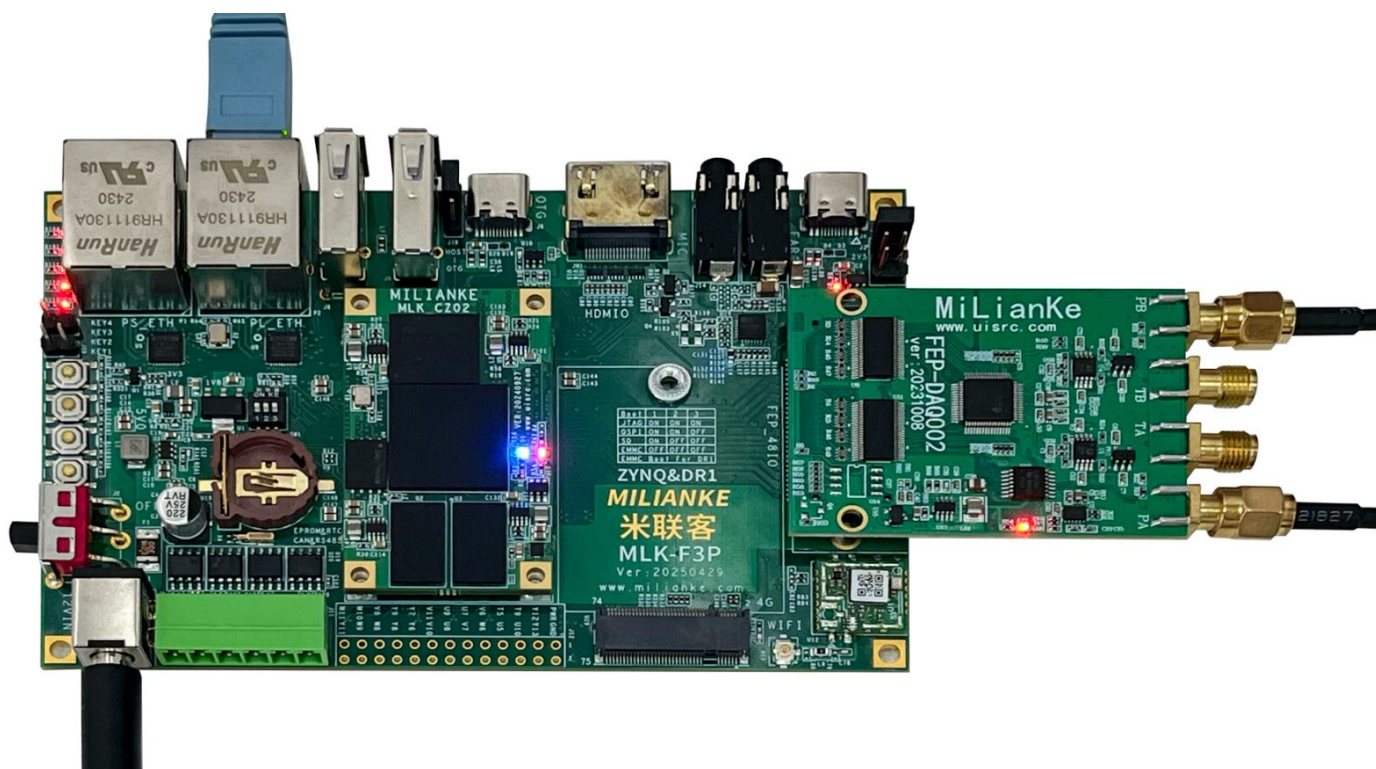
符号: 是否有符号位, 默认为 0, 没有符号位。

长度：每通道数据的长度。

注意数据段的头部和数据部分都是小端序，即先发送低字节，再发送高字节。

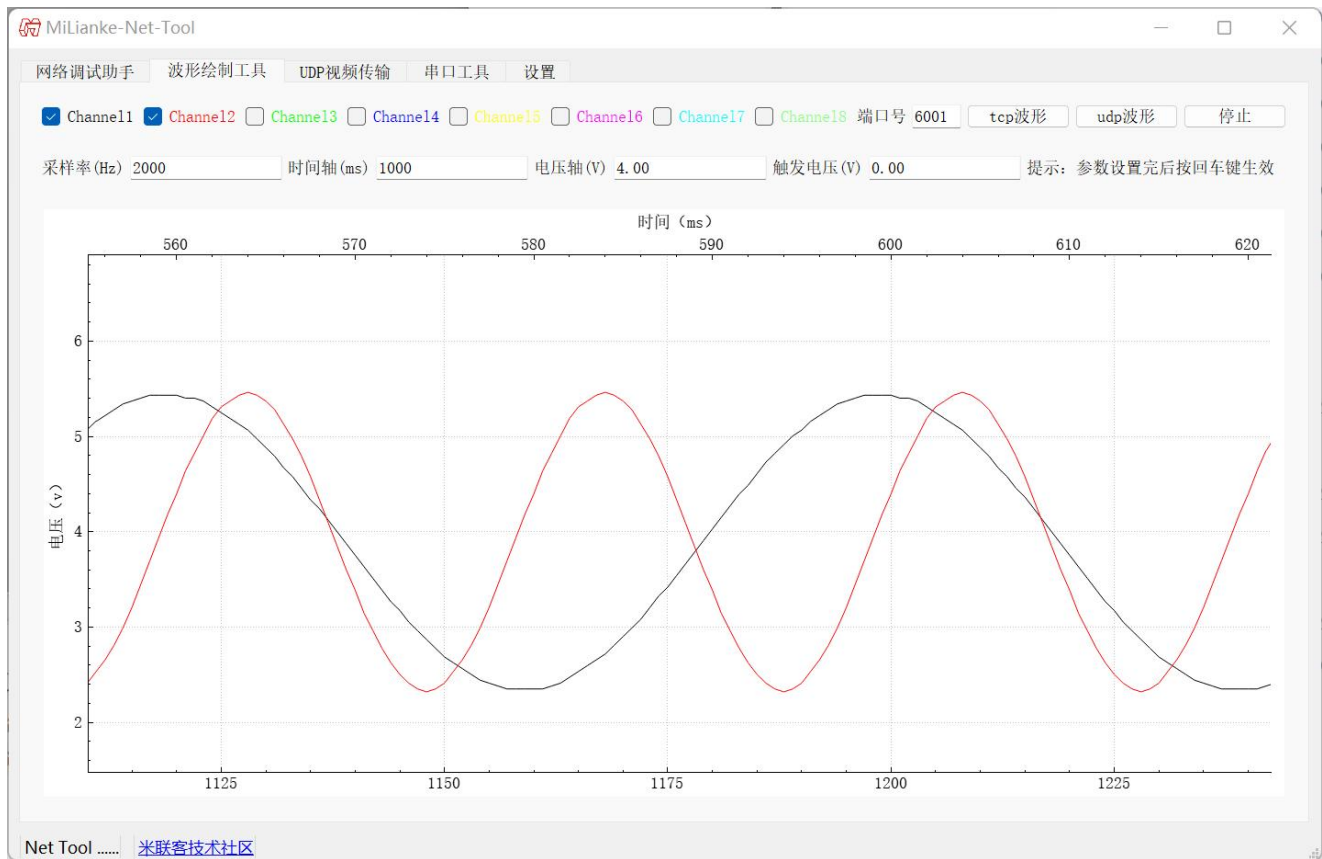
2 下载演示

2.1 硬件连接



2.2 运行结果

波形采集的效果如下图所示。



06 DDS 信号采集方案演示

软件版本: Anlogic -TD_6.2.2_Release_2026.1

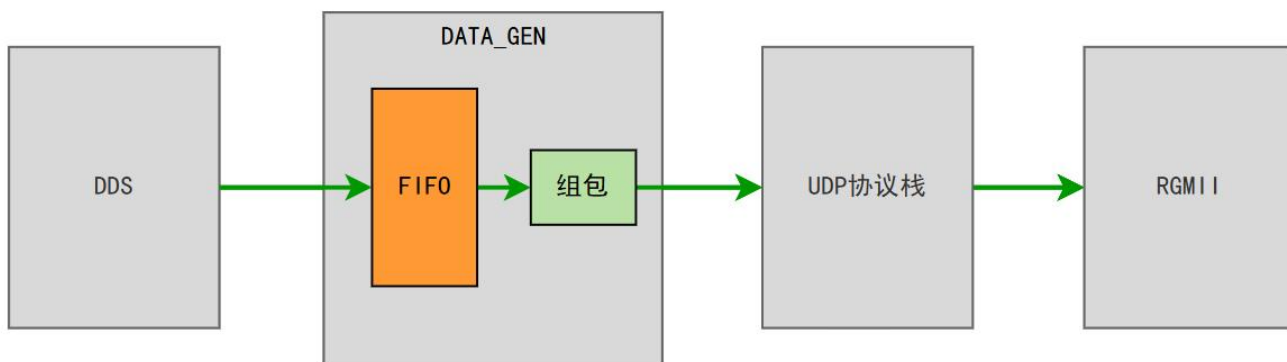
操作系统: WIN10/WIN11 64bit

硬件平台: 适用安路(Anlogic)FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 系统框图

本方案使用米联客自主编写的 DDS 任意信号发生器产生数据，通过千兆以太网 UDP 协议发送到 PC，最终在上位机中显示波形图。DDS 任意信号发生器原理这里不再展开讲解，如有需要可以在米联客 FPGA 教程基础篇中学习，本方案的系统框图如下图所示。



通过 ADC 采集的数据先进入 FIFO 中缓存，当 FIFO 中的数据量达到一定的阈值时，将数据读出，并且添加识别码和信息到读出的数据段之前作为头部，把组好的数据段发送至 UDP 协议栈中进行组帧。将组好的 MAC 帧由单沿转为双沿，发送至 PHY 芯片中，PHY 芯片完成编码和串化后由网线传输到 PC 端。

上位机数据段的格式如下图所示。

UDP首部	ID号	帧序列号	通道数	符号	频率	长度	数据
	8Byte	2Byte	6bit	2bit	2Byte	4Byte	

各字段的功能定义如下：

ID 号：上位机识别码，固定为 64' hAA55_AA55_AA55_AA55。

帧序列号：每发完一帧加 1。

通道数：波形采样的通道数，默认为 8。

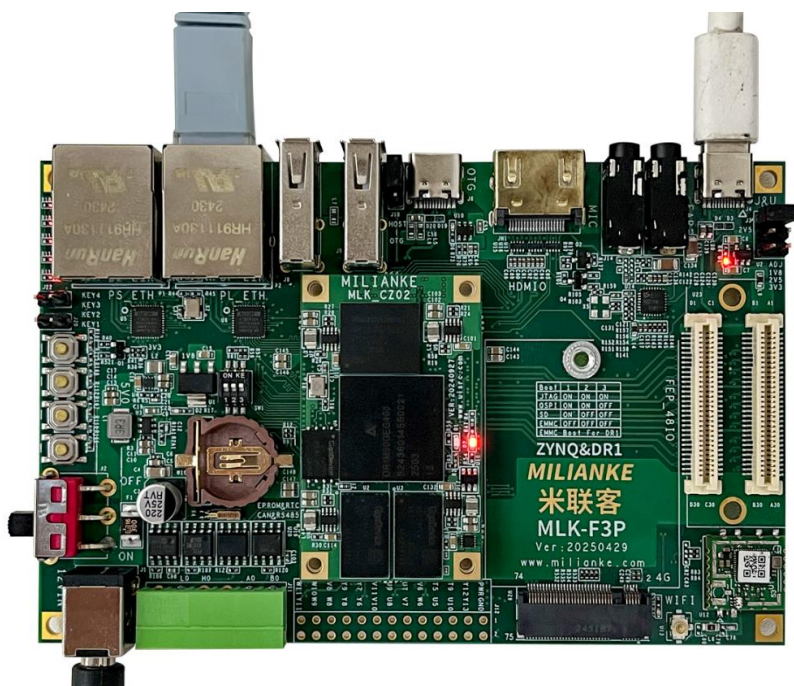
符号：是否有符号位，默认为 0，没有符号位。

长度：每通道数据的长度。

注意数据段的头部和数据部分都是小端序，即先发送低字节，再发送高字节。

2 下载演示

2.1 硬件连接

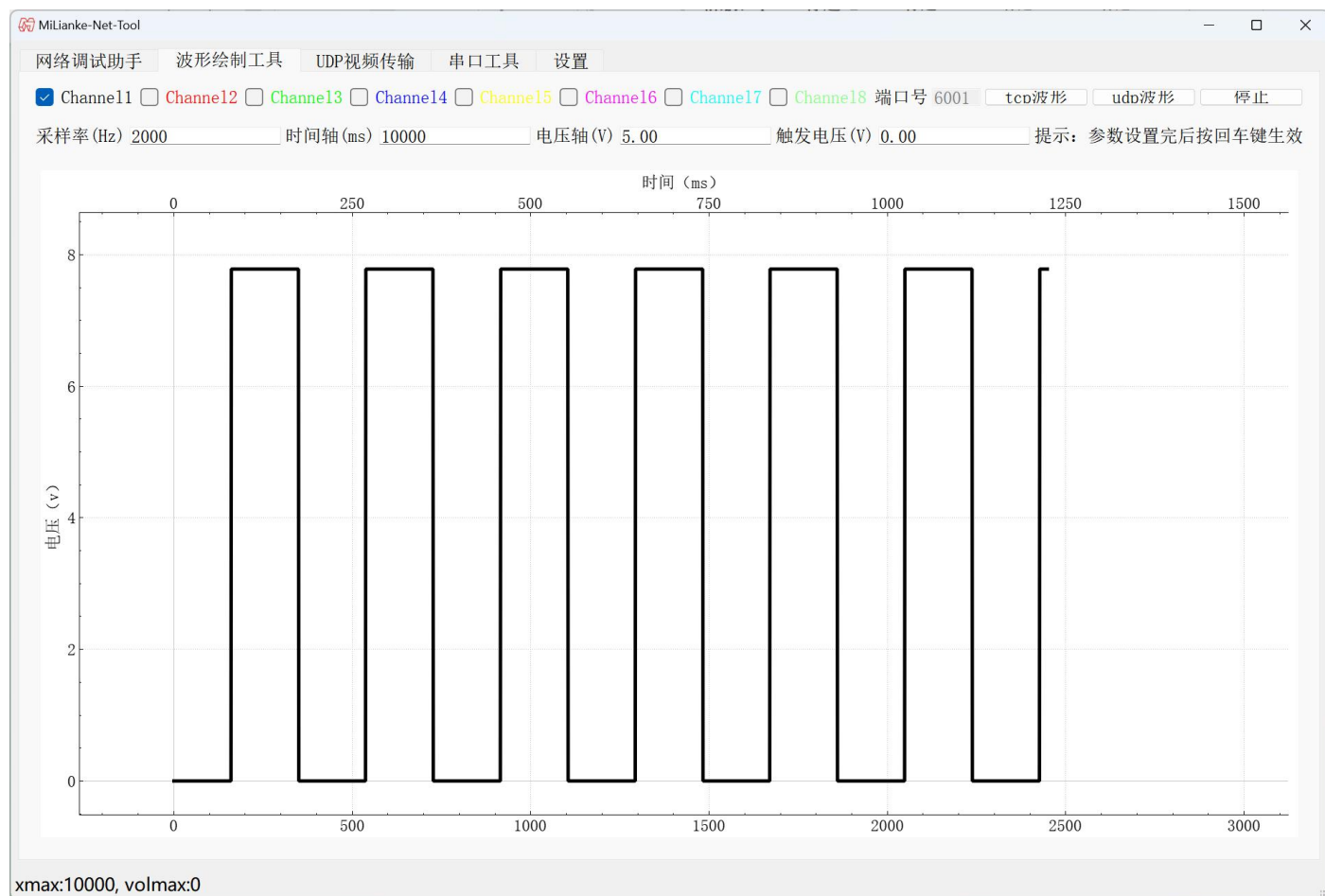
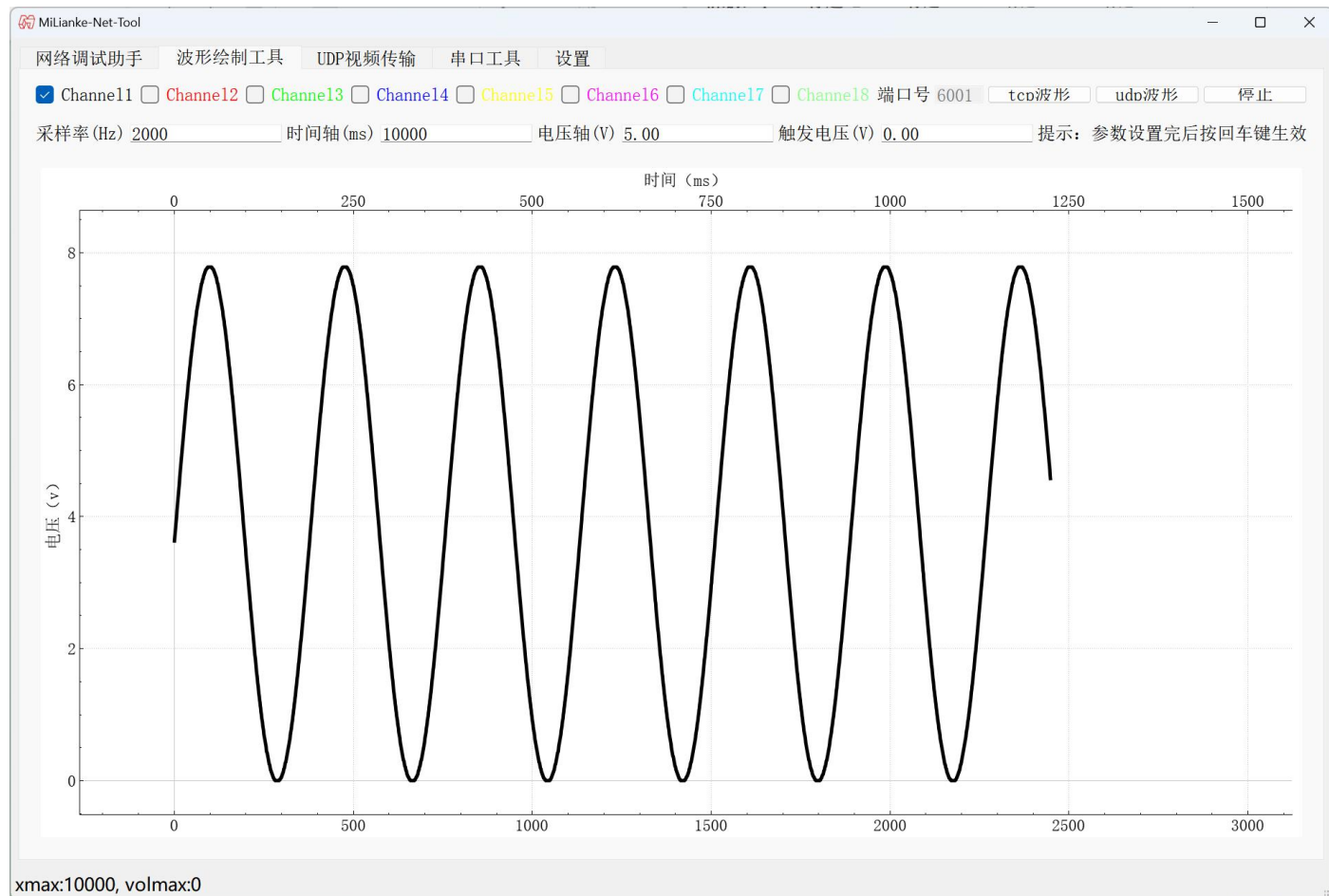


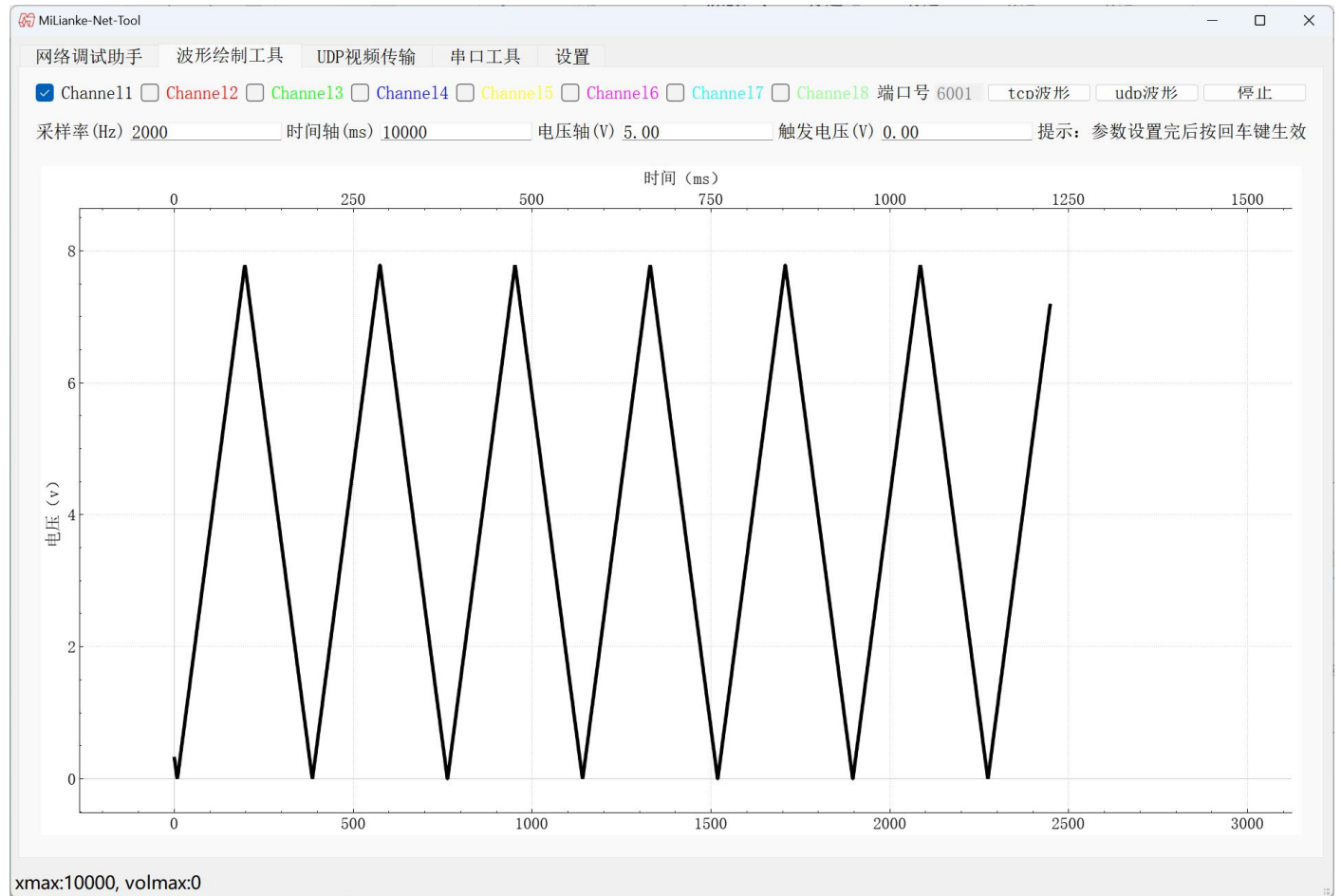
2.2 运行结果

使用米联客波形绘制工具，界面配置及波形采集的效果如下图所示，点击 UDP 波形，右击显示窗口，选择“自适应大小”，即可显示波形。

KEY1 切换波形

KEY2 增加频率





07 图像采集方案演示

软件版本: Anlogic -TD_6.2.2_Release_2026.1

操作系统: WIN10/WIN11 64bit

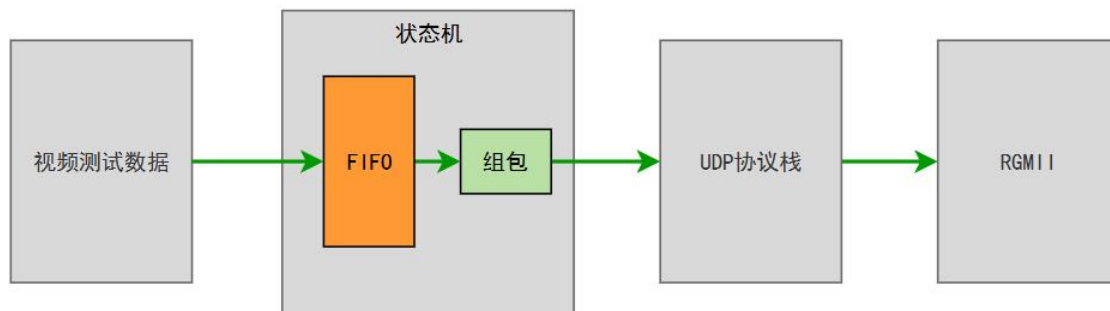
硬件平台: 适用安路(Anlogic)FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

本方案通过把生成的视频测试数据,通过千兆以太网 UDP 协议发送到 PC,最终在上位机中显示不同颜色的图片。视频时序这里不再展开讲解,如有需要可以在米联客 FPGA 教程基础篇中学习。

1 程序设计

本方案的系统框图如下图所示。



采集的图像测试数据先进入 FIFO 中缓存,当 FIFO 中的数据量达到一定的阈值时,将数据读出,并且信息到读出的数据段之前作为头部,把组好的数据段发送至 UDP 协议栈中进行组帧。将组好的 MAC 帧由单沿转为双沿,发送至 PHY 芯片中,PHY 芯片完成编码和串化后由网线传输到 PC 端。

上位机数据段的格式如下图所示。

UDP首部	图像头	图像宽度	图像高度	图片序列号	帧序列号	图像有效数据长度	图像数据
	4Byte	2Byte	2Byte	2Byte	2Byte	4Byte	1440Byte

数据有效长度: 每帧中图像数据的字节数, 1440 字节。

帧序列号: 标志一张图片的第几帧, 每发完一帧加 1, 一张图片发送完成后置 0。

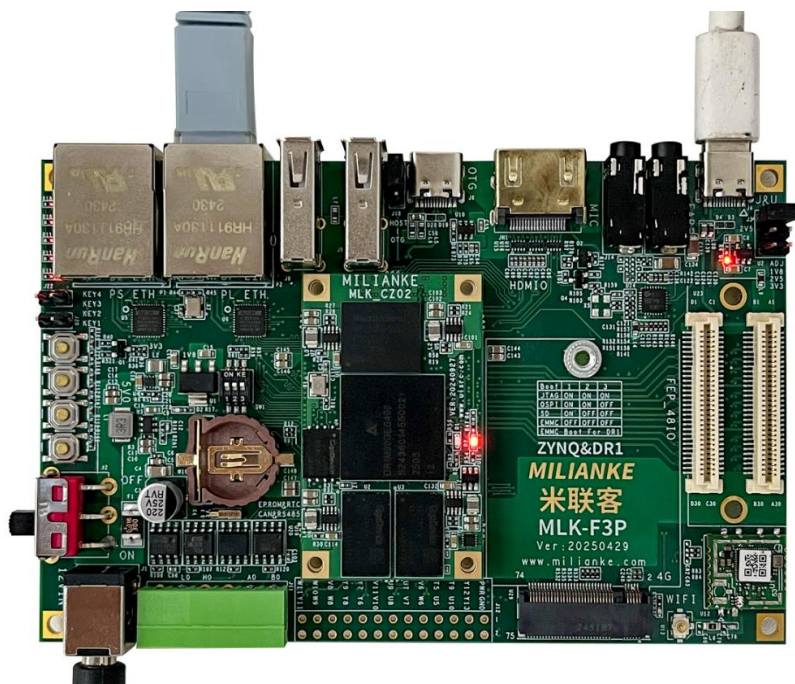
图片序列号: 每发送完一张图片加 1。

图像头: 固定为 32'hAA0055FF, 用于识别图像帧。

通过状态机生成符合上位机数据段格式的用户数据, 代码如下:

2 上板调试

2.1 硬件连接



2.1 实验结果

使用米联客 UDP 视频传输工具，界面配置如下图所示，点击开始接收，即可显示图像。



08 摄像头采集方案演示

软件版本: Anlogic -TD_6.2.2_Release_2026.1

操作系统: WIN10/WIN11 64bit

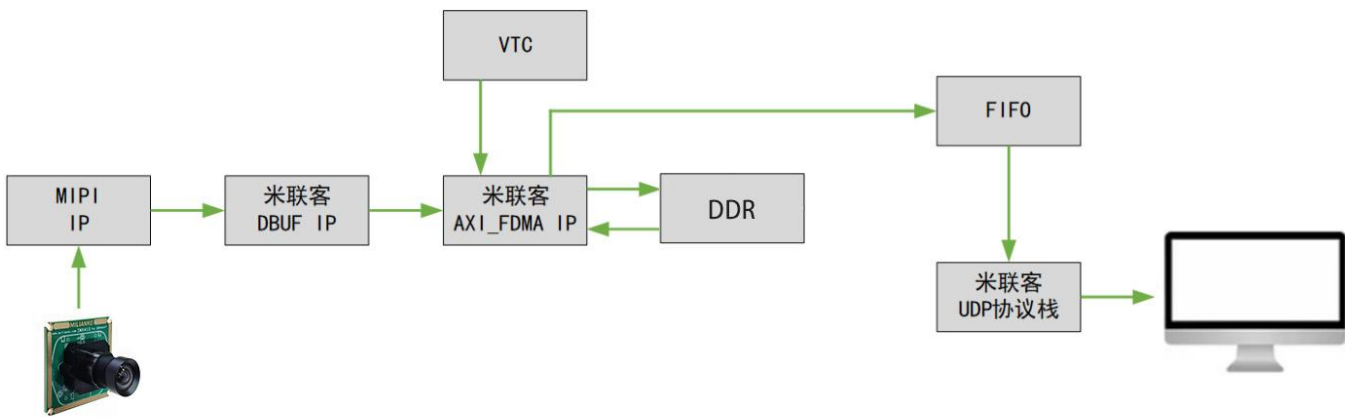
硬件平台: 适用安路(Anlogic)FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 系统框图

本方案在安路上实现了 MIPI 接口采集摄像头数据, 经过 DDR 缓存之后通过千兆以太网接口输出到上位机显示的方案。

本方案采用了米联客开源 UDP 协议栈, 方便用户在不同厂家的 FPGA 之间完成移植, 并且提供了开源上位机供客户实现功能。系统框图如下图所示。



上位机数据段的格式如下图所示。

UDP首部	图像有效数据长度	帧序列号	图片序列号	图像高度	图像宽度	图像头	图像数据
	4Byte	2Byte	2Byte	2Byte	2Byte	4Byte	1440Byte

数据有效长度: 每帧中图像数据的字节数, 1440 字节。

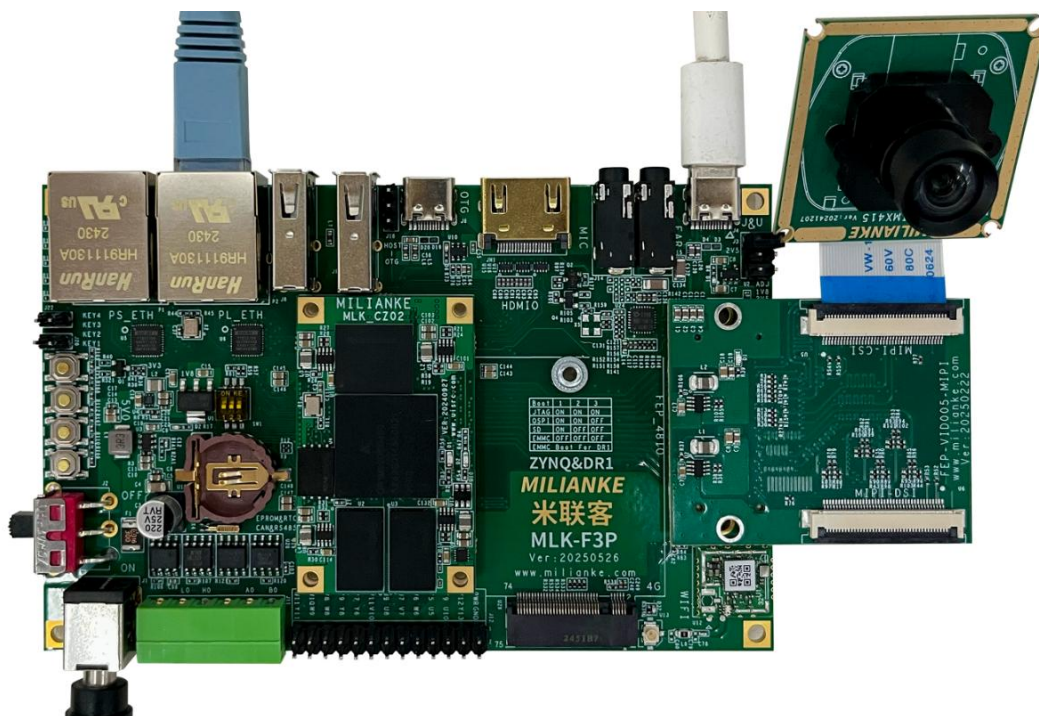
帧序列号: 标志一张图片的第几帧, 每发完一帧加 1, 一张图片发送完成后置 0。

图片序列号: 每发送完一张图片加 1。

图像头: 固定为 32'hAA0055FF, 用于识别图像帧。

2 下载演示

2.1 硬件连接



2.2 运行结果

视频采集的效果如下图所示。

